

On a novel fully decoupled, linear and second-order accurate numerical scheme for the Cahn–Hilliard–Darcy system of two-phase Hele–Shaw flow

Masterthesis
Mathematical Modeling, Simulation and Optimization

Farin Farrokhseresht
Matr.No. 218101039

February 11, 2026

First Examiner: Dr. C. Kahle
Mathematical department
University of Koblenz

Second Examiner: Prof. Dr. M. Hinze
Mathematical department
University of Koblenz

This thesis is dedicated to my sister for her lifelong support.

Declaration of authorship

I herewith confirm that I alone have authored this thesis, that I did not use any resources other than those I have cited — in particular no online sources not included in the bibliography section — and that I have not previously submitted this thesis in association with any other examination procedure.

Farin Farrokhseresht
Koblenz, February 11, 2026

Abstract

In this thesis we study the fully decoupled Scalar Auxiliary Variable (SAV) scheme of Yang (2021) for the Cahn–Hilliard–Darcy phase-field system modeling two-phase Hele–Shaw flow. The scheme decouples all equations through two auxiliary variables that exploit the zero-energy-contribution property, achieving linear systems, unconditional energy stability, and second-order accuracy without nonlinear iterations. We provide detailed mathematical analysis, develop a Python implementation with GPU acceleration, and validate through numerical experiments on droplet coarsening, spinodal decomposition, and convergence studies. Results confirm theoretical predictions and demonstrate practical efficiency, providing a foundation for extensions to three-dimensional simulations and viscous fingering applications.

Contents

Nomenclature	ix
1. Introduction	1
2. Background and Mathematical Foundations	4
2.1. Phase-Field Methods for Interface Dynamics	4
2.1.1. Overview of Phase-Field Approaches	4
2.1.2. The Cahn–Hilliard Equation	4
2.1.3. Surface Tension and Interfacial Effects	5
2.2. Darcy’s Law and Flow in Porous Media	6
2.2.1. Fundamentals of Darcy Flow	6
2.2.2. Hele–Shaw Cells and Two-Phase Hele–Shaw Flow	6
2.3. The Cahn–Hilliard–Darcy System	7
2.3.1. Mathematical Formulation	7
2.3.2. Boundary and Initial Conditions	8
2.4. Numerical Challenges and Existing Approaches	9
2.4.1. Key Challenges	9
2.4.2. Existing Numerical Strategies	9
2.5. The Scalar Auxiliary Variable Method	10
2.5.1. General Framework	10
2.5.2. Advantages and Applications	10
2.6. The Decoupling Strategy	11
3. The Decoupled SAV Numerical Scheme	12
3.1. Motivation and Key Challenges Revisited	12
3.1.1. Challenges for Standard Schemes	12
3.1.2. Linearization Problem and Existing Approaches	13
3.2. The Scalar Auxiliary Variable (SAV) Approach	13
3.2.1. Introduction of the First Auxiliary Variable	13
3.2.2. Reformulation Using the Auxiliary Variable	14
3.3. The Zero-Energy-Contribution Property and the Second Auxiliary Variable	15
3.3.1. Understanding Zero-Energy-Contribution	15
3.3.2. Introduction of the Second Auxiliary Variable Q	16
3.4. The Complete Transformed System	16
3.4.1. Reformulation of the CHD System	16

3.4.2.	Energy Dissipation of the Transformed System	17
3.5.	Time Discretization Strategy	17
3.5.1.	The Backward Differentiation Formula (BDF2)	17
3.5.2.	Stabilization and the Extra Term	18
3.5.3.	Full Time-Marching Scheme (Yang 2021)	18
3.6.	The Fully Decoupled Implementation Strategy	19
3.6.1.	Decoupling Through Linear Combinations	19
3.6.2.	Solution Algorithm Overview	20
3.7.	Theoretical Properties	21
3.7.1.	Unconditional Energy Stability	21
3.7.2.	Second-Order Accuracy	27
3.8.	Comparison with Other Schemes	28
3.8.1.	Advantages over Existing Methods	28
3.9.	Initialization and Practical Considerations	29
3.9.1.	First Time Step Initialization	29
3.9.2.	Choice of Parameters	29
3.9.3.	Time Step Selection	30
3.10.	Summary	30
4.	Numerical Implementation of the Fully-Decoupled SAV Scheme	32
4.1.	Spatial Discretization Strategy	32
4.1.1.	Choice of Spectral Methods	32
4.1.2.	Fourier Discretization and Wavenumber Grid	33
4.1.3.	Avoiding Aliasing and Dealiasing	34
4.2.	Linear Elliptic Solvers for Each Substep	34
4.2.1.	General Structure of Implicit Equations	34
4.2.2.	Phase-Field Equation	35
4.2.3.	Velocity and Pressure (Darcy Step)	35
4.2.4.	Auxiliary Variables U and Q	37
4.3.	Overall Time-Stepping Algorithm	37
4.3.1.	Algorithm Structure	37
4.4.	Implementation Details and Practical Considerations	38
4.4.1.	FFT Backend and Parallelization	38
4.4.2.	Initialization and First Time Step	39
4.4.3.	Parameter Choices and Tuning	39
4.4.4.	Energy and Mass Conservation Monitoring	40
4.5.	GPU Acceleration and Computational Infrastructure	40
4.5.1.	Why GPU Acceleration?	40
4.5.2.	Implementation Platform: AMD Ryzen 7 7800X3D and NVIDIA GeForce RTX 4070 SUPER	41
4.6.	CFL-Like Constraints and Time-Step Stability	42
4.6.1.	Absence of CFL Constraints in Energy-Stable Schemes	42
4.6.2.	Accuracy vs Stability Trade-off	43
4.6.3.	Balancing Temporal and Spatial Errors	43

4.6.4. Stability Verification via Energy Plots	43
4.7. Summary	44
5. Numerical Results and Validation Studies	45
5.1. Overview of Numerical Experiments	45
5.2. Coarsening Test: Two-Droplet Dynamics	46
5.2.1. Initial Condition and Parameters	46
5.2.2. Results and Physical Interpretation	46
5.3. Spinodal Decomposition: Phase Separation Dynamics	47
5.3.1. Problem Formulation	47
5.3.2. Symmetric Case: $\phi_{\text{avg}} = 0$	47
5.3.3. Asymmetric Case: $\phi_{\text{avg}} = 0.3$	48
5.4. Temporal Convergence Analysis	49
5.4.1. Methodology	49
5.4.2. Results and Interpretation	50
5.5. Comparison with Literature Results	51
5.6. Computational Efficiency and Scalability	52
5.6.1. Wall-Clock Time Comparison	52
5.7. Summary	52
6. Conclusions and Future Perspectives	54
6.1. Summary of Main Contributions	54
6.1.1. Theoretical Framework	54
6.1.2. Numerical Implementation	54
6.1.3. Numerical Validation	55
6.2. Three-Dimensional Extensions and Fingering Phenomena	55
6.2.1. Three-Dimensional Simulations	55
6.2.2. Viscous Fingering and Hele–Shaw Flow Instabilities	56
6.3. Recommendations for Future Work	57
6.4. Closing Remarks	58
Bibliography	59
A. Python Implementation Code	63
A.1. Performance Notes from Chapter 5	63
A.2. CPU Version (NumPy/SciPy)	64
A.3. GPU Version (CuPy/CUDA with CPU Fallback)	81
A.4. Key Implementation Features	99
A.5. Dependencies	99
A.5.1. Required (CPU Only)	99
A.5.2. Optional (GPU Acceleration)	99
A.5.3. Optional (Performance Enhancement)	100

Nomenclature

Table 1.: Key symbols used throughout the thesis.

Symbol	Description
Parameters	
Δt	Time step size
N_x, N_y	Number of grid points in the x and y directions
ϕ_{avg}	Average composition (mean phase-field value)
ϵ	Interface width (controls diffuse interface thickness)
τ	Darcy relaxation parameter (time scale in the velocity update)
α	Drag coefficient in the Darcy step
M	Mobility in the Cahn–Hilliard equation
λ	Surface tension parameter in the free energy
S	Stabilization parameter in the DSAV scheme
B	Positive constant in the SAV definition (keeps the square root well-defined)
State/Auxiliary Variables	
ϕ	Phase-field variable
μ	Chemical potential
U	SAV auxiliary variable for the nonlinear free-energy term
Q	Auxiliary variable enforcing the zero-energy-contribution property

1. Introduction

Two-phase flow appears in many natural and industrial processes and underpins applications in hydrology, oil recovery, and chemical processing [24]. In such systems, two immiscible fluids—for example air and water or oil and water—share a region, interact through surface tension and pressure, and evolve over time [9, 28]. The interface can deform, merge, or split, so accurate mathematical descriptions remain challenging and are still an active research topic [28, 1].

Concrete examples include wastewater treatment and fermentation reactors, where gas is sparged to enhance oxygen transfer and mixing [25, 16]. Bubble motion, breakup, and coalescence strongly affect interfacial mass transfer and mixing [16].

Another important example is flow in porous media and Hele–Shaw cells, where injected fluids displace resident fluids and the interface evolves under viscous and capillary forces [3, 28, 9]. Predicting interface dynamics is essential for understanding displacement efficiency in porous media [9]. Related multiphase models also appear in biological settings such as tissue growth and tumor modeling [17].

Classical numerical approaches track the interface explicitly as it evolves. When the interface undergoes complex geometric changes—such as strong deformations, splitting, or merging—explicit tracking becomes difficult to implement [11]. Diffuse-interface or phase-field methods avoid this by representing the interface with a smoothly varying scalar field that transitions between two bulk values across a narrow region [41, 31]. This implicit representation can handle large deformations and topological changes such as breakup and coalescence without direct interface reconstruction [1].

A widely used diffuse-interface model is the Cahn–Hilliard equation, which is based on a free energy functional that enforces a smooth but thin interface between the two phases [33, 4]. Although originally introduced in the context of phase separation in alloys, the model is now commonly applied to interfacial fluid problems [33, 4]. When the Cahn–Hilliard equation is combined with Darcy’s law, which describes slow viscous flow in porous media or thin channels [46, 9], the result is the Cahn–Hilliard–Darcy (CHD) system [44]. This system captures the combined effects of diffusion, interfacial forces, and advection by the fluid flow, making it a flexible and effective model for interface dynamics in many of the examples described above [44].

Despite its conceptual simplicity, the CHD system is challenging to simulate numerically. The model contains nonlinearities arising from the free energy, a fourth-order differential operator in the Cahn–Hilliard equation, coupling between the phase field and the fluid velocity, and an incompressibility constraint through Darcy’s law [44]. Furthermore, the CHD system satisfies an energy dissipation law, and numerical schemes should ideally preserve this structure to ensure stability, especially in long-time simulations [44, 18, 38].

A variety of numerical strategies have been developed to address these challenges, including fully implicit methods, convex splitting schemes, and stabilized semi-implicit approaches [18, 38, 2]. While these methods can provide good stability properties, they often require solving nonlinear or tightly coupled systems at each time step, which can be computationally expensive and difficult to implement [18, 2].

A significant advancement in this area is the numerical scheme developed by Yang (2021), which applies the Scalar Auxiliary Variable (SAV) method to the CHD system [44]. The SAV approach introduces an auxiliary variable for the nonlinear part of the free energy, enabling linear and energy-stable time stepping [37, 39]. Yang’s method extends SAV with a second auxiliary variable that preserves a zero-energy-contribution identity, yielding a fully linear, fully decoupled, second-order accurate, and unconditionally energy-stable scheme [44]. Each variable can be updated via linear elliptic solves with constant coefficients, making the scheme efficient and practical for Python implementations [44].

The goal of this thesis is to study the CHD system from both a mathematical and numerical perspective and to implement Yang’s decoupled SAV scheme in two dimensions using Python. We develop the model and its energy structure, explain the SAV-based decoupling strategy, describe the implementation in detail, and present numerical experiments that illustrate accuracy, stability, and efficiency.

The thesis is organized as follows:

Chapter 2 introduces the mathematical formulation of the Cahn–Hilliard–Darcy model, including the phase-field variable, free energy functional, chemical potential, Darcy flow, and energy dissipation property.

Chapter 3 discusses the numerical challenges associated with the CHD system and outlines the ideas behind Yang’s decoupled SAV scheme, with an emphasis on linearization and energy stability.

Chapter 4 describes the numerical implementation in detail, including spatial discretization, solution strategies for elliptic equations, and the overall algorithm used for time integration.

Chapter 5 presents numerical simulations demonstrating the performance and stability

of the method, including qualitative interface evolution and convergence plots.

Chapter 6 concludes the thesis and discusses potential directions for future research, such as three-dimensional simulations or more complex coupled systems.

2. Background and Mathematical Foundations

2.1. Phase-Field Methods for Interface Dynamics

2.1.1. Overview of Phase-Field Approaches

Traditional approaches to modeling two-phase flows track a sharp interface explicitly [11, 30]. This becomes difficult to implement when the interface undergoes strong deformations or topological changes such as pinch-off or splitting [13, 1]. Phase-field methods provide an alternative by replacing the sharp interface with a diffuse transition region represented by a smoothly varying scalar phase field [11, 41, 31].

In a phase-field representation, the order parameter $\phi(\mathbf{x}, t)$ takes distinct values in each phase—for example, $\phi = 1$ in fluid 1 and $\phi = -1$ in fluid 2—with a smooth transition across a narrow interfacial region of width $\mathcal{O}(\epsilon)$ [41, 31, 10]. Key advantages include automatic handling of topological changes and a natural incorporation of surface tension through the free-energy functional [13, 1, 31]. Phase-field models are compatible with standard spatial discretizations, including finite elements and spectral methods, and fit within thermodynamically consistent frameworks [13, 30, 31].

Phase-field models are typically constructed within a thermodynamically consistent framework, where the evolution equations are derived from a free energy functional and satisfy energy dissipation principles [31, 30, 12]. This structure motivates energy-stable discretizations for long-time simulations [12, 18].

2.1.2. The Cahn–Hilliard Equation

The **Cahn–Hilliard equation** is one of the most widely used phase-field models and was originally introduced to describe phase separation and spinodal decomposition in binary alloys [33, 4]. Given a phase field ϕ , the equation is typically written as:

$$\frac{\partial \phi}{\partial t} = \nabla \cdot (M \nabla \mu), \quad (2.1)$$

where $M > 0$ is a mobility parameter and μ is the **chemical potential** defined through the variational derivative of the free energy:

$$\mu = \frac{\delta E}{\delta \phi} = \lambda (-\Delta \phi + f(\phi)). \quad (2.2)$$

Substituting μ into the evolution equation shows that the Cahn–Hilliard equation is fourth order in space. This fourth-order operator leads to stiffness for explicit time stepping [12]. Here, $\lambda > 0$ is a model parameter and $f(\phi) = \frac{1}{\epsilon^2}(\phi^3 - \phi)$ is the derivative of the double-well potential [4]. The total free energy functional is:

$$E(\phi) = \int_{\Omega} \left[\frac{\lambda}{2} |\nabla \phi|^2 + \lambda F(\phi) \right] d\mathbf{x}, \quad (2.3)$$

where $F(\phi) = \frac{1}{4\epsilon^2}(\phi^2 - 1)^2$ is the double-well potential [4]. The gradient term $\frac{\lambda}{2} |\nabla \phi|^2$ penalizes sharp interfaces, while the potential term $\lambda F(\phi)$ favors the pure phases $\phi = \pm 1$ [41, 31].

A fundamental property of the Cahn–Hilliard equation is that it conserves the total mass of the phase field, meaning $\int_{\Omega} \phi d\mathbf{x}$ remains constant over time [31]. This conservation property, along with the energy dissipation law, ensures that the model remains thermodynamically consistent [31, 12].

2.1.3. Surface Tension and Interfacial Effects

In the phase-field framework, surface tension arises naturally from the free energy functional [44, 31]. The interfacial force per unit volume is:

$$\mathbf{f}_{\text{surface}} = \phi \nabla \mu, \quad (2.4)$$

which couples the phase field to the fluid mechanics [44]. This formulation encodes capillary forces in bulk terms and avoids explicit interface tracking [1, 13].

The interfacial width parameter ϵ controls the thickness of the transition region between the two phases. In the limit $\epsilon \rightarrow 0$, diffuse-interface models are expected to converge to

corresponding sharp-interface models [13]. For computations, ϵ is chosen relative to the mesh resolution to balance accuracy and cost.

2.2. Darcy's Law and Flow in Porous Media

2.2.1. Fundamentals of Darcy Flow

Darcy's law describes the flow of a viscous fluid through a porous medium or between two closely spaced parallel plates (in a Hele–Shaw cell) [3, 46, 13]. It models slow, viscous flow where inertial effects are negligible (low Reynolds number) [3, 46]. Darcy's law is stated as:

$$\mathbf{u} = -\frac{k}{\mu_f} \nabla p, \quad (2.5)$$

where $\mathbf{u}$ is the superficial velocity (or Darcy velocity), k is the permeability of the medium, μ_f is the dynamic viscosity of the fluid, and ∇p is the pressure gradient [3, 46]. For incompressible flow, the velocity must satisfy the continuity equation:

$$\nabla \cdot \mathbf{u} = 0. \quad (2.6)$$

In many applications where transient effects are important, a time-dependent version of Darcy's law is used:

$$\tau \frac{\partial \mathbf{u}}{\partial t} + \alpha \mathbf{u} + \nabla p = \mathbf{f}, \quad (2.7)$$

where $\tau > 0$ is a relaxation time parameter and $\alpha > 0$ is the hydraulic conductivity [44, 46]. This formulation captures transient effects in porous media flows [44, 46].

2.2.2. Hele–Shaw Cells and Two-Phase Hele–Shaw Flow

A **Hele–Shaw cell** consists of two parallel plates separated by a narrow gap, creating an idealized geometry for studying two-phase flow [13, 32]. The flow between these plates approximately satisfies Darcy's law in the small-gap, viscous regime [13, 44]. Hele–Shaw cells have been extensively used experimentally to study viscous fingering, where a less

viscous fluid (such as air) invades a region filled with a more viscous fluid (such as oil), creating characteristic finger-like patterns [32, 24].

When a Cahn–Hilliard equation is coupled with the Darcy equations in a Hele–Shaw cell, the resulting system is called the **Cahn–Hilliard–Darcy (CHD) system** [44]. This coupling is particularly favorable for numerical scheme development because, unlike the Navier–Stokes equations where velocity and pressure are tightly coupled, Darcy’s law provides an explicit relationship between velocity and pressure gradient [44]. This structure allows for more efficient decoupling strategies in numerical schemes [44].

2.3. The Cahn–Hilliard–Darcy System

2.3.1. Mathematical Formulation

The **Cahn–Hilliard–Darcy (CHD) system** couples the Cahn–Hilliard equation for phase separation with Darcy’s law for the fluid flow. In a domain $\Omega \subset \mathbb{R}^d$ (with $d = 2, 3$), the system is written as:

$$\frac{\partial \phi}{\partial t} + \nabla \cdot (\mathbf{u}\phi) = M\Delta\mu, \quad (2.8)$$

$$\mu = \lambda(-\Delta\phi + f(\phi)), \quad (2.9)$$

$$\tau \frac{\partial \mathbf{u}}{\partial t} + \alpha \mathbf{u} + \nabla p + \phi \nabla \mu = 0, \quad (2.10)$$

$$\nabla \cdot \mathbf{u} = 0, \quad (2.11)$$

where ϕ is the phase field, μ is the chemical potential, $\mathbf{u}$ is the velocity field, p is the pressure, and $f(\phi) = \frac{1}{\epsilon^2}(\phi^3 - \phi)$ [44]. The parameters are M (mobility), λ (surface tension), τ (inertial relaxation time), and α (hydraulic conductivity) [44].

The key features of this system are:

1. **Conservation of phase field:** The advection term $\nabla \cdot (\mathbf{u}\phi)$, together with the diffusion term $M\Delta\mu$, ensures conservation of the total mass $\int_{\Omega} \phi d\mathbf{x}$ [13].
2. **Coupling between phases and flow:** The term $\phi \nabla \mu$ in the momentum equation represents the capillary/interfacial force, creating a two-way coupling where the interface geometry influences the fluid flow, and the flow advects the interface [44].

3. **Fourth-order diffusion and nonlinearity:** The fourth-order Laplacian operator (through μ) and the nonlinear double-well potential $f(\phi)$ introduce significant mathematical and computational challenges [44, 18].
4. **Energy structure:** The system satisfies a fundamental **energy dissipation law**:

$$\frac{d}{dt}E(\mathbf{u}, \phi) = -M\|\nabla\mu\|^2 - \alpha\|\mathbf{u}\|^2 \leq 0, \quad (2.12)$$

where

$$E(\mathbf{u}, \phi) = \int_{\Omega} \left[\frac{\tau}{2}|\mathbf{u}|^2 + \frac{\lambda}{2}|\nabla\phi|^2 + \lambda F(\phi) \right] d\mathbf{x} \quad (2.13)$$

[44]. This energy dissipation property is crucial for long-time stability and must be preserved in numerical discretizations [44, 18, 2].

2.3.2. Boundary and Initial Conditions

For periodic domains, all variables are taken to be periodic. For bounded domains, common choices are homogeneous Neumann conditions:

$$\mathbf{u} \cdot \mathbf{n}|_{\partial\Omega} = \partial_n\phi|_{\partial\Omega} = \partial_n\mu|_{\partial\Omega} = 0, \quad (2.14)$$

where $\mathbf{n}$ is the outward normal vector [44]. For the Neumann case, one can derive that $\partial_n p|_{\partial\Omega} = 0$ [44].

Initial conditions are provided at $t = 0$:

$$\mathbf{u}|_{t=0} = \mathbf{u}_0, \quad p|_{t=0} = p_0, \quad \phi|_{t=0} = \phi_0, \quad (2.15)$$

where ϕ_0 is typically chosen to represent the initial configuration of the two-phase system [44].

2.4. Numerical Challenges and Existing Approaches

2.4.1. Key Challenges

The CHD system presents several significant challenges for numerical discretization:

1. **Nonlinear stiffness:** The double-well potential $f(\phi) = \frac{1}{\epsilon^2}(\phi^3 - \phi)$ introduces a stiff nonlinearity. For small interface widths ϵ , explicit time-stepping methods require very small time steps for stability [12, 18].
2. **Fourth-order operator:** The fourth-order Laplacian in the Cahn–Hilliard equation requires careful discretization to maintain stability and accuracy [18].
3. **Coupled nonlinearities:** The coupling between the advection term $\nabla \cdot (\mathbf{u}\phi)$ and the surface tension term $\phi \nabla \mu$ creates additional complications for scheme design [44].
4. **Incompressibility constraint:** The divergence-free condition $\nabla \cdot \mathbf{u} = 0$ couples the velocity and pressure through the momentum equation [30].

2.4.2. Existing Numerical Strategies

Several numerical approaches have been developed to address these challenges:

Fully implicit methods: These solve all equations implicitly at each time step, ensuring unconditional stability but requiring the solution of large nonlinear systems [18]. This approach is computationally expensive and can be difficult to implement.

Convex splitting schemes: These decompose the free energy into convex and concave parts, treating the convex part implicitly and the concave part explicitly [2, 13]. This approach is unconditionally energy stable but typically requires solving nonlinear systems at each time step [13, 2].

Stabilized semi-implicit methods: These combine explicit and implicit treatments with additional stabilization terms to maintain unconditional energy stability [44]. While more efficient than fully implicit methods, they still involve significant computational overhead [44].

Scalar Auxiliary Variable (SAV) approach: This recent method introduces auxiliary variables to linearize the nonlinear terms, leading to schemes that are linear, uncon-

ditionally energy stable, and efficient [37, 39]. The SAV approach solves only decoupled linear elliptic equations with constant coefficients at each time step [37, 39].

2.5. The Scalar Auxiliary Variable Method

2.5.1. General Framework

The **Scalar Auxiliary Variable (SAV)** method is a recently developed technique for constructing efficient and unconditionally energy stable numerical schemes for gradient flows [37, 39]. The key idea is to introduce auxiliary variables that linearize the nonlinear terms in the free energy without introducing additional nonlinearity into the system.

For a gradient flow written abstractly as:

$$\frac{\partial u}{\partial t} = \mathcal{G} \left(\frac{\delta E}{\delta u} \right), \quad (2.16)$$

where $E[u]$ is a free energy functional and $\mathcal{G}$ is a dissipative operator (e.g., $\mathcal{G} = \Delta$ for the Cahn–Hilliard equation), the SAV method introduces an auxiliary variable:

$$r(t) = \sqrt{\int_{\Omega} F(u) d\mathbf{x} + C_0}, \quad (2.17)$$

where C_0 is a constant ensuring the argument is positive [37, 39]. This auxiliary variable allows the nonlinear part of the energy to be treated in a decoupled and linear manner [37, 39].

2.5.2. Advantages and Applications

The SAV approach has several important advantages:

1. **Unconditional energy stability:** The resulting numerical scheme is unconditionally energy stable, meaning the discrete energy satisfies a dissipation law regardless of the time step size [37, 39].

2. **Linear schemes:** All nonlinear terms are linearized, allowing the solution of only simple linear equations with constant coefficients at each time step [37, 39].
3. **Efficiency:** The scheme avoids nonlinear iterations (e.g., Newton-Raphson), making it efficient compared to fully implicit or convex-splitting methods [37, 39].
4. **Applicability:** The SAV method applies to a broad class of gradient flows and is not restricted to specific forms of the free energy [37, 39].

Recent work has demonstrated the application of SAV methods to the Cahn–Hilliard equation and its variants [37, 39], with second-order accurate schemes that are unconditionally energy stable and efficient in practice [44, 37].

2.6. The Decoupling Strategy

One of the major contributions of Yang’s scheme is the introduction of a decoupling strategy based on the **zero-energy-contribution property** of certain nonlinear terms [44]. This property states that the advection term $\nabla \cdot (\mathbf{u}\phi)$ and the surface tension term $\phi \nabla \mu$ contribute zero to the net energy dissipation when properly tested:

$$\int_{\Omega} (\phi \nabla \mu \cdot \mathbf{u} + \nabla \cdot (\mathbf{u}\phi) \mu) d\mathbf{x} = 0. \quad (2.18)$$

By introducing an auxiliary variable $Q(t)$ through a specially designed ordinary differential equation, this zero-energy-contribution property can be exploited to decouple all equations into independent linear systems at each time step [44]. This leads to a fully decoupled, linear, unconditionally energy stable scheme where:

- The phase-field equation is solved independently
- The velocity and pressure are solved independently
- All resulting equations have constant coefficients and can be solved efficiently with standard linear solvers [44]

This decoupling avoids nonlinear iterations or coupled solves, reducing computational cost while maintaining unconditional energy stability [44].

3. The Decoupled SAV Numerical Scheme

Building on Chapter 2, we present the fully decoupled Scalar Auxiliary Variable (SAV) scheme developed by Yang (2021) for the Cahn–Hilliard–Darcy system. We focus on the transformations that enable linearity, decoupling, and energy stability, which are essential for the implementation in Chapter 4.

3.1. Motivation and Key Challenges Revisited

3.1.1. Challenges for Standard Schemes

Before introducing Yang’s scheme, let us revisit the fundamental difficulties that make the CHD system challenging to discretize. Recall from Chapter 2 that the system (Equations (2.8)–(2.11)) contains several sources of difficulty:

1. **Fourth-order diffusion and nonlinearity:** The term $\mu = \lambda(-\Delta\phi + f(\phi))$ introduces a fourth-order spatial operator coupled with a nonlinear potential. For small interface widths ϵ , this nonlinearity becomes stiff [12, 18].
2. **Implicit–explicit coupling:** The advection term $\nabla \cdot (\mathbf{u}\phi)$ and the surface tension term $\phi\nabla\mu$ couple the phase field and the flow. Treating them implicitly requires nonlinear solves, while explicit treatment imposes severe time-step restrictions [12, 18].
3. **Incompressibility constraint:** The divergence-free condition $\nabla \cdot \mathbf{u} = 0$ must be enforced exactly, and the pressure variable p enters without a time derivative. This necessitates careful treatment through projection methods or similar approaches [44, 21].
4. **Energy stability requirement:** The system satisfies an energy dissipation law (Equation (2.12)), and ideally numerical schemes should preserve this structure unconditionally (i.e., without restrictive time-step constraints) [44, 18, 2].

These difficulties explain why standard numerical methods struggle with the CHD system [44, 13]. A naive fully explicit method would require very small time steps to maintain stability, especially for small ϵ [12]. Fully implicit methods require solving large nonlinear systems at each time step, which is computationally expensive and difficult to implement. The innovation of Yang's scheme is that it overcomes these difficulties simultaneously through auxiliary variables and the zero-energy-contribution property.

3.1.2. Linearization Problem and Existing Approaches

A central issue is how to treat the nonlinear term $f(\phi) = \frac{1}{\epsilon^2}(\phi^3 - \phi)$ without expensive nonlinear solvers. Standard approaches include:

Fully implicit treatment: Solve $f(\phi^{n+1})$ implicitly, requiring Newton-Raphson iterations at each time step. This approach, while unconditionally stable, is computationally expensive [12, 18].

Convex splitting: Decompose the free energy into convex and concave parts and treat them implicitly/explicitly [2, 12]. This reduces nonlinearity but still requires solving some nonlinear systems [13].

Explicit extrapolation: Use $f(\phi^n)$ or $f(2\phi^n - \phi^{n-1})$ to evaluate the nonlinear term at known times. This is computationally cheap but imposes significant stability constraints on Δt [12].

Auxiliary variable linearization: Introduce an auxiliary variable that represents part of the free energy, allowing the entire scheme to be linear [37]. This is the SAV approach [39].

The SAV method is preferred because it achieves unconditional energy stability without nonlinear iterations. The key insight, developed in the context of gradient flows [37, 39], is that energy stability can be enforced using auxiliary variables without solving the nonlinear term implicitly at each time step.

3.2. The Scalar Auxiliary Variable (SAV) Approach

3.2.1. Introduction of the First Auxiliary Variable

The classical SAV method, introduced by Shen, Xu, and Yang (2018), uses an auxiliary variable to linearize the nonlinear potential energy term [37]. For the CHD system, we

define:

$$U(t) = \sqrt{\int_{\Omega} F(\phi) d\mathbf{x} + B}, \quad (3.1)$$

where $B > 0$ is a constant chosen to ensure the argument remains positive. This definition has been successfully applied to gradient flows of various types [37, 39, 45]. It also has an important consequence: the nonlinear term $F(\phi)$ is now hidden inside a square root of an integral, which is a single scalar function of time.

The motivation for this definition comes from observing that the free energy contains the term $\int_{\Omega} F(\phi) d\mathbf{x}$. By introducing $U(t)$, we can write:

$$\int_{\Omega} F(\phi) d\mathbf{x} = U^2 - B, \quad (3.2)$$

and differentiate with respect to time to obtain:

$$2U \frac{dU}{dt} = \int_{\Omega} F'(\phi) \frac{\partial \phi}{\partial t} d\mathbf{x} = \int_{\Omega} f(\phi) \frac{\partial \phi}{\partial t} d\mathbf{x}, \quad (3.3)$$

where we used $f(\phi) = F'(\phi)$. This equation still involves $\frac{\partial \phi}{\partial t}$, but it provides the basis for the time discretization of U .

3.2.2. Reformulation Using the Auxiliary Variable

A key idea is to replace $f(\phi)$ in the chemical potential with an auxiliary potential function $H(\phi)$ defined as:

$$H(\phi) = \frac{f(\phi)}{U(t)}, \quad (3.4)$$

and reformulate the chemical potential as:

$$\mu = \lambda(-\Delta \phi + H(\phi)U(t)). \quad (3.5)$$

This reformulation is algebraically equivalent to the original definition (Equation (2.9)) when $U(t)$ is defined by Equation (3.1). In time discretization, one can evaluate $H(\phi)$ explicitly while solving for the scalar $U(t)$, leading to linear updates [37].

To close the system, we need an evolution equation for $U(t)$. Differentiating Equation (3.2):

$$\frac{dU^2}{dt} = \int_{\Omega} f(\phi) \frac{\partial \phi}{\partial t} d\mathbf{x}. \quad (3.6)$$

Using the Cahn–Hilliard equation $\frac{\partial \phi}{\partial t} = \nabla \cdot (M \nabla \mu) - \nabla \cdot (\mathbf{u} \phi)$ (Equation (2.8)), we can write this evolution equation in a form suitable for time discretization. This provides the basis for a stable update of U .

3.3. The Zero-Energy-Contribution Property and the Second Auxiliary Variable

3.3.1. Understanding Zero-Energy-Contribution

One notable aspect of Yang’s scheme is the exploitation of the **zero-energy-contribution property**. This property arises from the fact that the two coupling terms—advection and surface tension—have a special relationship in the energy dissipation law [44].

As noted in Chapter 2 (cf. [44, Remark 2.1]), when we compute the energy law for the CHD system, the advection term $\nabla \cdot (\mathbf{u} \phi)$ and the surface tension term $\phi \nabla \mu$ combine as follows:

$$\int_{\Omega} [\phi \nabla \mu \cdot \mathbf{u} + \nabla \cdot (\mathbf{u} \phi) \mu] d\mathbf{x} = 0. \quad (3.7)$$

This identity holds due to integration by parts and the incompressibility constraint $\nabla \cdot \mathbf{u} = 0$ [44]. The physical interpretation is that the net energy contribution from the coupling between the fluid flow and the interface is zero—the flow does work against the interface tension, but this work is exactly balanced by the energy flux through the moving interface [44]. Related decoupling strategies for Cahn–Hilliard–Darcy and Cahn–Hilliard–Navier–Stokes systems can be found in [23, 15]. This balance allows the advection and surface tension terms to be discretized with different strategies as long as the cancellation is preserved, motivating the auxiliary variable introduced next [44].

3.3.2. Introduction of the Second Auxiliary Variable Q

To exploit the zero-energy-contribution property, Yang introduces a second auxiliary variable $Q(t)$ defined by the following ordinary differential equation (ODE):

$$\frac{dQ}{dt} = \int_{\Omega} [\nabla \cdot (\mathbf{u}\phi)\mu + (\phi\nabla\mu) \cdot \mathbf{u}] d\mathbf{x}, \quad (3.8)$$

with initial condition $Q(0) = 1$ [44].

At the continuous level (before time discretization), the right-hand side of Equation (3.8) is zero due to the zero-energy-contribution property (Equation (3.7)); hence $\frac{dQ}{dt} = 0$ and $Q(t) = 1$ for all time. The discrete setting is different: the zero-energy cancellation is not automatic, so introducing Q allows the advection and surface tension terms to be balanced in a discrete energy inequality [44].

3.4. The Complete Transformed System

3.4.1. Reformulation of the CHD System

Combining the SAV method and the zero-energy-contribution property, we can reformulate the original CHD system (Equations (2.8)–(2.11) in Chapter 2) into an equivalent system in the variables $(\mathbf{u}, p, \phi, \mu, U, Q)$ [44]:

$$\frac{\partial\phi}{\partial t} + Q\nabla \cdot (\mathbf{u}\phi) = M\Delta\mu, \quad (3.9)$$

$$\mu = \lambda(-\Delta\phi + H(\phi)U), \quad (3.10)$$

$$\frac{dU}{dt} = \frac{1}{2U} \int_{\Omega} H(\phi) \frac{\partial\phi}{\partial t} d\mathbf{x}, \quad (3.11)$$

$$\tau \frac{\partial\mathbf{u}}{\partial t} + \alpha\mathbf{u} + \nabla p + Q\phi\nabla\mu = 0, \quad (3.12)$$

$$\nabla \cdot \mathbf{u} = 0, \quad (3.13)$$

$$\frac{dQ}{dt} = \int_{\Omega} [\nabla \cdot (\mathbf{u}\phi)\mu + (\phi\nabla\mu) \cdot \mathbf{u}] d\mathbf{x}, \quad (3.14)$$

where $H(\phi) = \frac{f(\phi)}{U}$ is defined as in Equation (3.4). This system is mathematically equivalent to the original CHD system (Equations (2.8)–(2.11)) because:

1. The zero-energy-contribution property ensures $Q(t) = 1$ at the continuous level (Equation (3.14) has right-hand side zero).
2. When $Q = 1$, Equations (3.9) and (3.12) reduce to the original momentum and phase equations.
3. The definition of U (Equation (3.1)) ensures that $H(\phi)U = f(\phi)$, so Equation (3.10) gives the correct chemical potential.

3.4.2. Energy Dissipation of the Transformed System

Despite the auxiliary variables, the transformed system still satisfies an energy dissipation law. By carefully taking inner products of the transformed equations and using integration by parts (techniques common in analysis of gradient flows [37, 39]), one can show that:

$$\frac{d}{dt} \left[\int_{\Omega} \left(\frac{\tau}{2} |\mathbf{u}|^2 + \frac{\lambda}{2} |\nabla \phi|^2 \right) d\mathbf{x} + \lambda U^2 + \frac{Q^2}{2} \right] = -M \|\nabla \mu\|^2 - \alpha \|\mathbf{u}\|^2 \leq 0. \quad (3.15)$$

The modified energy functional includes not only the kinetic and interfacial energies but also contributions from the auxiliary variables. The key observation is that the energy dissipation rate depends only on the diffusion term (first term on right) and the friction term (second term on right), both of which are non-positive. This structure is preserved under proper time discretization [44], guaranteeing unconditional energy stability of the scheme. Similar approaches have been used successfully for other coupled systems [23, 15, 34].

3.5. Time Discretization Strategy

3.5.1. The Backward Differentiation Formula (BDF2)

To discretize the system in time, we use the second-order Backward Differentiation Formula (BDF2), which provides second-order accuracy while allowing flexible treatment of nonlinear terms [44]. For a scalar function $\psi(t)$, the BDF2 approximation is:

$$\left. \frac{\partial \psi}{\partial t} \right|_{t=t^{n+1}} \approx \frac{3\psi^{n+1} - 4\psi^n + \psi^{n-1}}{2\Delta t}, \quad (3.16)$$

where $\psi^k = \psi(t_k)$ and $t_{n+1} = t_n + \Delta t$ [44]. BDF2 is advantageous because it is second-order accurate in time, well-understood from numerical analysis of ODEs and PDEs [40, 6], and it allows implicit treatment of stiff terms while remaining computationally efficient. The stability of BDF2 for nonlinear problems has been extensively studied [22, 26, 27].

When treating nonlinear terms explicitly, we use second-order extrapolation to maintain overall accuracy:

$$\psi^* = 2\psi^n - \psi^{n-1}, \quad (3.17)$$

which approximates $\psi(t^{n+1})$ using only values from previous time steps [44]. This technique is standard in numerical methods for nonlinear PDEs [27].

3.5.2. Stabilization and the Extra Term

When applying BDF2 to systems with stiff nonlinearities like $f(\phi)$, it is sometimes necessary to add a small stabilization term to maintain accuracy and stability with large time steps [44]. The stabilized scheme includes an extra term in the chemical potential equation:

$$\mu^{n+1} = \lambda \left(-\Delta \phi^{n+1} + H(\phi^*) U^{n+1} + \frac{S}{\epsilon^2} (\phi^{n+1} - \phi^*) \right), \quad (3.18)$$

where $S > 0$ is a stabilization parameter and $\phi^* = 2\phi^n - \phi^{n-1}$. The additional term $\frac{S}{\epsilon^2} (\phi^{n+1} - \phi^*)$ vanishes at the continuous level (as $\Delta t \rightarrow 0$) but provides numerical stability for finite Δt [44]. Similar stabilization techniques have been used in various finite element schemes for PDEs [12, 15]. The stabilization parameter S is typically chosen to be $O(1)$, and its effect diminishes as the mesh is refined and/or the time step is decreased. Example choices of S are discussed in [44].

3.5.3. Full Time-Marching Scheme (Yang 2021)

With the BDF2 discretization and the stabilization term in place, Yang's second-order time-marching scheme for the transformed system can be written as [44]:

$$\frac{a\phi^{n+1} - b\phi^n + c\phi^{n-1}}{2\Delta t} + Q^{n+1}\nabla \cdot (\mathbf{u}^* \phi^*) = M\Delta\mu^{n+1}, \quad (3.19)$$

$$\mu^{n+1} = \lambda \left(-\Delta\phi^{n+1} + H^*U^{n+1} + \frac{S}{\epsilon^2}(\phi^{n+1} - \phi^*) \right), \quad (3.20)$$

$$aU^{n+1} - bU^n + cU^{n-1} = \frac{1}{2} \int_{\Omega} H^*(a\phi^{n+1} - b\phi^n + c\phi^{n-1}) d\mathbf{x}, \quad (3.21)$$

$$\tau \frac{a\tilde{\mathbf{u}}^{n+1} - b\mathbf{u}^n + c\mathbf{u}^{n-1}}{2\Delta t} + \alpha\tilde{\mathbf{u}}^{n+1} + \nabla p^n + Q^{n+1}\phi^*\nabla\mu^* = 0, \quad (3.22)$$

$$\frac{aQ^{n+1} - bQ^n + cQ^{n-1}}{2\Delta t} = \int_{\Omega} [\nabla \cdot (\mathbf{u}^* \phi^*) \mu^{n+1} + (\phi^* \nabla \mu^*) \cdot \tilde{\mathbf{u}}^{n+1}] d\mathbf{x}, \quad (3.23)$$

$$\tau \frac{a}{2\Delta t} (\mathbf{u}^{n+1} - \tilde{\mathbf{u}}^{n+1}) + \nabla (p^{n+1} - p^n) = 0, \quad (3.24)$$

$$\nabla \cdot \mathbf{u}^{n+1} = 0. \quad (3.25)$$

Here the BDF2 coefficients and extrapolated quantities are

$$\begin{aligned} a = 3, \quad b = 4, \quad c = 1, \quad \mathbf{u}^* = 2\mathbf{u}^n - \mathbf{u}^{n-1}, \quad \phi^* = 2\phi^n - \phi^{n-1}, \\ \mu^* = 2\mu^n - \mu^{n-1}, \quad H^* = H(\phi^*). \end{aligned} \quad (3.26)$$

Remark 3.3 uses a second-order pressure-correction step to decouple pressure and velocity. Initialization at t_1 follows a backward Euler variant by setting $a = 2$, $b = 2$, $c = 0$, and $\psi^* = \psi^0$ for any variable ψ . We impose periodic or homogeneous Neumann boundary conditions: $\partial_n p^{n+1}|_{\partial\Omega} = \partial_n \phi^{n+1}|_{\partial\Omega} = \partial_n \mu^{n+1}|_{\partial\Omega} = \mathbf{u}^{n+1} \cdot \mathbf{n}|_{\partial\Omega} = 0$ [44].

Remark 3.4 notes that the linear stabilizer in (3.20) adds an error of size $\frac{S}{\epsilon^2} \Delta t^2 \partial_{tt} \phi(\cdot)$, comparable to the extrapolation error in $f(\phi)$, and improves accuracy and energy stability at large time steps. As emphasized in [44], the scheme (3.19)–(3.25) is not fully decoupled as written; full decoupling leverages the nonlocal structure of U and Q in the implementation that follows.

3.6. The Fully Decoupled Implementation Strategy

3.6.1. Decoupling Through Linear Combinations

A key feature of Yang's scheme is that it is **fully decoupled**: all variables can be computed independently by solving simple linear elliptic equations with constant coefficients [44]. This is achieved through a splitting strategy based on the nonlocal nature of the

auxiliary variables U and Q . Similar decoupling strategies have been developed for other coupled systems; see [23, 15] for applications to coupled Cahn–Hilliard systems.

Because U and Q depend only on time, equations involving products like $U^{n+1}\mu^{n+1}$ or $Q^{n+1}\phi^{n+1}$ can be split into linear combinations. Specifically, if we write:

$$\phi^{n+1} = \phi_1^{n+1} + Q^{n+1}\phi_2^{n+1}, \quad \mu^{n+1} = \mu_1^{n+1} + Q^{n+1}\mu_2^{n+1}, \quad U^{n+1} = U_1^{n+1} + Q^{n+1}U_2^{n+1}, \quad (3.27)$$

then the nonlinear coupling through the auxiliary variable Q can be separated [44]. We can solve for each component (with subscripts 1 and 2) independently, and the final solution is obtained by combining them. This splitting works because Q enters linearly in all equations (after the introduction of auxiliary variables). The product $Q^{n+1}\phi^{n+1}$ can be written as $Q^{n+1}(\phi_1^{n+1} + Q^{n+1}\phi_2^{n+1}) = Q^{n+1}\phi_1^{n+1} + (Q^{n+1})^2\phi_2^{n+1}$, and the equations can be rewritten to decouple the components.

3.6.2. Solution Algorithm Overview

At each time step, the decoupled implementation proceeds as follows [44]:

1. **Step 1:** Solve decoupled phase-field and chemical potential equations using the variable U^{n+1} to split the system. This yields intermediate values $(\phi_{11}, \mu_{11}), (\phi_{12}, \mu_{12}), (\phi_{21}, \mu_{21})$.
2. **Step 2:** Compute U_1^{n+1} and U_2^{n+1} from their defining equations using the phase-field values obtained in Step 1.
3. **Step 3:** Reconstruct the phase field and chemical potential as:

$$\phi_1^{n+1} = \phi_{11} + U_1^{n+1}\phi_{12}, \phi_2^{n+1} = \phi_{21} + U_2^{n+1}\phi_{22}, \quad (3.28)$$

$$\mu_1^{n+1} = \mu_{11} + U_1^{n+1}\mu_{12}, \mu_2^{n+1} = \mu_{21} + U_2^{n+1}\mu_{22}. \quad (3.29)$$

4. **Step 4:** Solve decoupled velocity equations using Q^{n+1} to split the system, yielding $(\tilde{\mathbf{u}}_1, \tilde{\mathbf{u}}_2)$.
5. **Step 5:** Compute divergence-free velocity corrections using pressure Poisson equations, yielding $(\mathbf{u}_1, p_1)$ and $(\mathbf{u}_2, p_2)$ [21].
6. **Step 6:** Compute Q^{n+1} from its defining equation using the reconstructed chemical potential and velocity values.

7. **Step 7:** Reconstruct the final velocity and pressure as:

$$\mathbf{u}^{n+1} = \mathbf{u}_1^{n+1} + Q^{n+1} \mathbf{u}_2^{n+1}, p^{n+1} = p_1^{n+1} + Q^{n+1} p_2^{n+1}. \quad (3.30)$$

The crucial point is that in each step, we solve **linear** equations with **constant coefficients**. There are no nonlinear iterations, and the auxiliary variables U and Q are either computed explicitly or appear only as scalars multiplying the unknowns. This is a significant computational advantage over methods requiring nonlinear solvers at each time step [44, 37].

3.7. Theoretical Properties

3.7.1. Unconditional Energy Stability

A central theorem of Yang's work states that the fully discretized scheme (with BDF2 in time and standard spatial discretization) satisfies a discrete energy dissipation inequality. For completeness, we reproduce Theorem 3.1 and give a proof.

Theorem 3.1 (Discrete energy dissipation). *For the scheme (3.19)–(3.25), the discrete energy*

$$\begin{aligned} E^{n+1} = & \frac{\tau}{2} (\|\mathbf{u}^{n+1}\|^2 + \|2\mathbf{u}^{n+1} - \mathbf{u}^n\|^2) + \frac{\lambda}{2} (\|\nabla \phi^{n+1}\|^2 + \|2\nabla \phi^{n+1} - \nabla \phi^n\|^2) \\ & + \lambda (|U^{n+1}|^2 + |2U^{n+1} - U^n|^2) + \frac{1}{2} (|Q^{n+1}|^2 + |2Q^{n+1} - Q^n|^2) \\ & + \frac{2\Delta t^2}{3\tau} \|\nabla p^{n+1}\|^2 + \frac{\lambda S}{\epsilon^2} \|\phi^{n+1} - \phi^n\|^2. \end{aligned} \quad (3.31)$$

satisfies

$$\frac{1}{\Delta t} (E^{n+1} - E^n) \leq -\alpha \|\tilde{\mathbf{u}}^{n+1}\|^2 - M \|\nabla \mu^{n+1}\|^2 \leq 0. \quad (3.32)$$

Proof.

Notation. Let $(\cdot, \cdot)$ denote the $L^2(\Omega)$ inner product and $\|\cdot\|$ its norm. Define $d^n = \phi^{n+1} - \phi^n$ and $e^n = \phi^n - \phi^{n-1}$, so $\phi^{n+1} - \phi^* = d^n - e^n$ and $3\phi^{n+1} - 4\phi^n + \phi^{n-1} = 3d^n - e^n$.

BDF2 identity. We repeatedly use the following identity, which holds in any inner-product space (hence in $L^2(\Omega)$):

$$2(3a - 4b + c, a) = \|a\|^2 - \|b\|^2 + \|2a - b\|^2 - \|2b - c\|^2 + \|a - 2b + c\|^2, \quad (3.33)$$

where a, b, c are vectors. To verify it, expand the squared norms:

$$\begin{aligned}\|2a - b\|^2 &= 4\|a\|^2 - 4(a, b) + \|b\|^2, \\ \|2b - c\|^2 &= 4\|b\|^2 - 4(b, c) + \|c\|^2, \\ \|a - 2b + c\|^2 &= \|a\|^2 + 4\|b\|^2 + \|c\|^2 - 4(a, b) + 2(a, c) - 4(b, c),\end{aligned}$$

so the right-hand side becomes $6\|a\|^2 - 8(a, b) + 2(a, c) = 2(3a - 4b + c, a)$.

Step 1 (velocity and pressure projection). We first isolate the velocity and pressure contributions. From (3.24) with $a = 3$,

$$\frac{3\tau}{2\Delta t}(\mathbf{u}^{n+1} - \tilde{\mathbf{u}}^{n+1}) + \nabla(p^{n+1} - p^n) = 0.$$

Projection orthogonality. Taking the L^2 inner product with any divergence-free $\mathbf{v}$ and using $(\nabla(p^{n+1} - p^n), \mathbf{v}) = 0$ gives

$$(\mathbf{u}^{n+1} - \tilde{\mathbf{u}}^{n+1}, \mathbf{v}) = 0.$$

Thus $(\mathbf{u}^{n+1}, \mathbf{v}) = (\tilde{\mathbf{u}}^{n+1}, \mathbf{v})$ for divergence-free $\mathbf{v}$. In particular, choosing the test function $\mathbf{v} = 3\mathbf{u}^{n+1} - 4\mathbf{u}^n + \mathbf{u}^{n-1}$ (divergence-free because each $\mathbf{u}^k$ is divergence-free for $k = n-1, n, n+1$) yields

$$(3\mathbf{u}^{n+1} - 4\mathbf{u}^n + \mathbf{u}^{n-1}, \tilde{\mathbf{u}}^{n+1}) = (3\mathbf{u}^{n+1} - 4\mathbf{u}^n + \mathbf{u}^{n-1}, \mathbf{u}^{n+1}),$$

and since

$$3\tilde{\mathbf{u}}^{n+1} - 4\mathbf{u}^n + \mathbf{u}^{n-1} = (3\mathbf{u}^{n+1} - 4\mathbf{u}^n + \mathbf{u}^{n-1}) + 3(\tilde{\mathbf{u}}^{n+1} - \mathbf{u}^{n+1}),$$

we obtain

$$\begin{aligned}(3\tilde{\mathbf{u}}^{n+1} - 4\mathbf{u}^n + \mathbf{u}^{n-1}, \tilde{\mathbf{u}}^{n+1}) &= (3\mathbf{u}^{n+1} - 4\mathbf{u}^n + \mathbf{u}^{n-1}, \mathbf{u}^{n+1}) \\ &\quad + 3(\tilde{\mathbf{u}}^{n+1} - \mathbf{u}^{n+1}, \tilde{\mathbf{u}}^{n+1}).\end{aligned}$$

Using $(\mathbf{u}^{n+1}, \tilde{\mathbf{u}}^{n+1}) = \|\mathbf{u}^{n+1}\|^2$ (take $\mathbf{v} = \mathbf{u}^{n+1}$ in the projection relation), the last term becomes $3(\|\tilde{\mathbf{u}}^{n+1}\|^2 - \|\mathbf{u}^{n+1}\|^2)$, which is the extra contribution appearing below.

Applying (3.33) with $a = \mathbf{u}^{n+1}$, $b = \mathbf{u}^n$, $c = \mathbf{u}^{n-1}$ yields

$$\begin{aligned}& (3\tilde{\mathbf{u}}^{n+1} - 4\mathbf{u}^n + \mathbf{u}^{n-1}, \tilde{\mathbf{u}}^{n+1}) \\ &= \frac{1}{2} \left(\|\mathbf{u}^{n+1}\|^2 - \|\mathbf{u}^n\|^2 + \|2\mathbf{u}^{n+1} - \mathbf{u}^n\|^2 - \|2\mathbf{u}^n - \mathbf{u}^{n-1}\|^2 \right) + \frac{1}{2} \|\mathbf{u}^{n+1} - 2\mathbf{u}^n + \mathbf{u}^{n-1}\|^2 \\ &\quad + 3 \left(\|\tilde{\mathbf{u}}^{n+1}\|^2 - \|\mathbf{u}^{n+1}\|^2 \right).\end{aligned}\tag{3.34a}$$

Momentum equation. Taking the L^2 inner product of (3.22) with $2\Delta t \tilde{\mathbf{u}}^{n+1}$ gives

$$\tau(3\tilde{\mathbf{u}}^{n+1} - 4\mathbf{u}^n + \mathbf{u}^{n-1}, \tilde{\mathbf{u}}^{n+1}) + 2\alpha\Delta t \|\tilde{\mathbf{u}}^{n+1}\|^2 + 2\Delta t(\nabla p^n, \tilde{\mathbf{u}}^{n+1}) + 2\Delta t Q^{n+1} \int_{\Omega} (\phi^* \nabla \mu^*) \cdot \tilde{\mathbf{u}}^{n+1} d\mathbf{x} = 0. \quad (3.34b)$$

Pressure contribution. Rewriting (3.24) as

$$\frac{3\tau}{2\Delta t} \mathbf{u}^{n+1} + \nabla p^{n+1} = \frac{3\tau}{2\Delta t} \tilde{\mathbf{u}}^{n+1} + \nabla p^n,$$

and setting $k = 2\Delta t/(3\tau)$ gives

$$\mathbf{u}^{n+1} + k\nabla p^{n+1} = \tilde{\mathbf{u}}^{n+1} + k\nabla p^n.$$

Taking L^2 norms of both sides and using $(\mathbf{u}^{n+1}, \nabla p^{n+1}) = 0$ yields

$$\|\mathbf{u}^{n+1}\|^2 + k^2 \|\nabla p^{n+1}\|^2 = \|\tilde{\mathbf{u}}^{n+1}\|^2 + k^2 \|\nabla p^n\|^2 + 2k(\tilde{\mathbf{u}}^{n+1}, \nabla p^n),$$

which is equivalent to

$$\frac{3\tau}{2} (\|\mathbf{u}^{n+1}\|^2 - \|\tilde{\mathbf{u}}^{n+1}\|^2) + \frac{2\Delta t^2}{3\tau} (\|\nabla p^{n+1}\|^2 - \|\nabla p^n\|^2) = 2\Delta t(\tilde{\mathbf{u}}^{n+1}, \nabla p^n). \quad (3.34c)$$

Taking the inner product of (3.24) with $2\Delta t \mathbf{u}^{n+1}$ gives

$$3\tau(\mathbf{u}^{n+1} - \tilde{\mathbf{u}}^{n+1}, \mathbf{u}^{n+1}) + 2\Delta t(\nabla(p^{n+1} - p^n), \mathbf{u}^{n+1}) = 0,$$

Here $(\nabla(p^{n+1} - p^n), \mathbf{u}^{n+1}) = 0$ follows from $\nabla \cdot \mathbf{u}^{n+1} = 0$ and periodic or homogeneous Neumann boundary conditions.

And since $(\mathbf{u}^{n+1} - \tilde{\mathbf{u}}^{n+1}, \mathbf{u}^{n+1}) = \frac{1}{2} (\|\mathbf{u}^{n+1}\|^2 - \|\tilde{\mathbf{u}}^{n+1}\|^2 + \|\mathbf{u}^{n+1} - \tilde{\mathbf{u}}^{n+1}\|^2)$, we obtain

$$\frac{3\tau}{2} (\|\mathbf{u}^{n+1}\|^2 - \|\tilde{\mathbf{u}}^{n+1}\|^2 + \|\mathbf{u}^{n+1} - \tilde{\mathbf{u}}^{n+1}\|^2) = 0. \quad (3.34d)$$

Collecting terms.

Insert $\tau \times (3.34a)$ into (3.34b).

Replace $2\Delta t(\nabla p^n, \tilde{\mathbf{u}}^{n+1})$ using (3.34c).

Add (3.34d) to remove the remaining $\|\tilde{\mathbf{u}}^{n+1}\|^2 - \|\mathbf{u}^{n+1}\|^2$ and introduce $\|\mathbf{u}^{n+1} - \tilde{\mathbf{u}}^{n+1}\|^2$, then collect all terms on the left.

$$\begin{aligned} & \frac{\tau}{2} (\|\mathbf{u}^{n+1}\|^2 - \|\mathbf{u}^n\|^2 + \|2\mathbf{u}^{n+1} - \mathbf{u}^n\|^2 - \|2\mathbf{u}^n - \mathbf{u}^{n-1}\|^2) + \frac{\tau}{2} \|\mathbf{u}^{n+1} - 2\mathbf{u}^n + \mathbf{u}^{n-1}\|^2 \\ & + \frac{3\tau}{2} \|\mathbf{u}^{n+1} - \tilde{\mathbf{u}}^{n+1}\|^2 + \frac{2\Delta t^2}{3\tau} (\|\nabla p^{n+1}\|^2 - \|\nabla p^n\|^2) + 2\alpha\Delta t \|\tilde{\mathbf{u}}^{n+1}\|^2 \\ & + 2\Delta t Q^{n+1} \int_{\Omega} (\phi^* \nabla \mu^*) \cdot \tilde{\mathbf{u}}^{n+1} d\mathbf{x} = 0. \end{aligned} \quad (3.34)$$

Step 2 (phase field and chemical potential). We now combine the phase-field and chemical-potential equations so that the $(\mu^{n+1}, \cdot)$ terms cancel.

Phase-field test. Taking the inner product of (3.19) with $2\Delta t \mu^{n+1}$ gives

$$(3\phi^{n+1} - 4\phi^n + \phi^{n-1}, \mu^{n+1}) + 2\Delta t Q^{n+1} \int_{\Omega} \nabla \cdot (\mathbf{u}^* \phi^*) \mu^{n+1} d\mathbf{x} + 2\Delta t M \|\nabla \mu^{n+1}\|^2 = 0.$$

Chemical-potential test. Taking the inner product of (3.20) with $-(3\phi^{n+1} - 4\phi^n + \phi^{n-1})$ and integrating by parts yields

$$\begin{aligned} & -(\mu^{n+1}, 3\phi^{n+1} - 4\phi^n + \phi^{n-1}) + \lambda(3\nabla \phi^{n+1} - 4\nabla \phi^n + \nabla \phi^{n-1}, \nabla \phi^{n+1}) \\ & + \lambda U^{n+1} \int_{\Omega} H^*(3\phi^{n+1} - 4\phi^n + \phi^{n-1}) d\mathbf{x} + \frac{\lambda S}{\epsilon^2} (\phi^{n+1} - \phi^*, 3\phi^{n+1} - 4\phi^n + \phi^{n-1}) = 0. \end{aligned}$$

Cancellation and BDF2 expansion. Adding these two equalities cancels $(3\phi^{n+1} - 4\phi^n + \phi^{n-1}, \mu^{n+1})$ because $(\mu^{n+1}, \psi) = (\psi, \mu^{n+1})$. Here, the Laplacian term is handled using periodic or homogeneous Neumann boundary conditions so that $(-\Delta \phi^{n+1}, 3\phi^{n+1} - 4\phi^n + \phi^{n-1}) = (3\nabla \phi^{n+1} - 4\nabla \phi^n + \nabla \phi^{n-1}, \nabla \phi^{n+1})$. Applying (3.33) to the gradient term and using the expansion

$$\begin{aligned} (\phi^{n+1} - \phi^*, 3\phi^{n+1} - 4\phi^n + \phi^{n-1}) &= (d^n - e^n, 3d^n - e^n) \\ &= 3\|d^n\|^2 - 4(d^n, e^n) + \|e^n\|^2 \\ &= \|d^n\|^2 - \|e^n\|^2 + 2\|d^n - e^n\|^2, \end{aligned}$$

gives

$$\begin{aligned} & \frac{\lambda}{2} \left(\|\nabla \phi^{n+1}\|^2 - \|\nabla \phi^n\|^2 + \|2\nabla \phi^{n+1} - \nabla \phi^n\|^2 - \|2\nabla \phi^n - \nabla \phi^{n-1}\|^2 \right) + \frac{\lambda}{2} \|\nabla(\phi^{n+1} - 2\phi^n + \phi^{n-1})\|^2 \\ & + \frac{\lambda S}{\epsilon^2} \left(\|\phi^{n+1} - \phi^n\|^2 - \|\phi^n - \phi^{n-1}\|^2 \right) + \frac{2\lambda S}{\epsilon^2} \|\phi^{n+1} - 2\phi^n + \phi^{n-1}\|^2 \\ & + \lambda U^{n+1} \int_{\Omega} H^*(3\phi^{n+1} - 4\phi^n + \phi^{n-1}) d\mathbf{x} \\ & + 2\Delta t Q^{n+1} \int_{\Omega} \nabla \cdot (\mathbf{u}^* \phi^*) \mu^{n+1} d\mathbf{x} + 2\Delta t M \|\nabla \mu^{n+1}\|^2 = 0. \end{aligned} \tag{3.35}$$

Step 3 (auxiliary variables). Because U and Q are scalars, their L^2 norms reduce to absolute values.

U update. With $a = 3$, $b = 4$, $c = 1$ in (3.21), multiplying by $2\lambda U^{n+1}$ gives

$$2\lambda(3U^{n+1} - 4U^n + U^{n-1})U^{n+1} = \lambda U^{n+1} \int_{\Omega} H^*(3\phi^{n+1} - 4\phi^n + \phi^{n-1}) d\mathbf{x},$$

and applying (3.33) yields

$$\begin{aligned} & \lambda \left(|U^{n+1}|^2 - |U^n|^2 + |2U^{n+1} - U^n|^2 - |2U^n - U^{n-1}|^2 \right. \\ & \quad \left. + |U^{n+1} - 2U^n + U^{n-1}|^2 \right) = \lambda U^{n+1} \int_{\Omega} H^*(3\phi^{n+1} - 4\phi^n + \phi^{n-1}) d\mathbf{x}, \end{aligned} \quad (3.36)$$

Q update. Similarly, multiplying (3.23) by $2\Delta t Q^{n+1}$ and applying (3.33) gives

$$\begin{aligned} & \frac{1}{2} \left(|Q^{n+1}|^2 - |Q^n|^2 + |2Q^{n+1} - Q^n|^2 - |2Q^n - Q^{n-1}|^2 \right. \\ & \quad \left. + |Q^{n+1} - 2Q^n + Q^{n-1}|^2 \right) = 2\Delta t Q^{n+1} \int_{\Omega} [\nabla \cdot (\mathbf{u}^* \phi^*) \mu^{n+1} + (\phi^* \nabla \mu^*) \cdot \tilde{\mathbf{u}}^{n+1}] d\mathbf{x}. \end{aligned} \quad (3.37)$$

Step 4 (sum and cancellation). We now sum (3.34), (3.35), (3.36), and (3.37) and regroup terms into coupling cancellations, energy differences, and nonnegative remainder terms.

Explicit sum.

$$\begin{aligned}
 0 = & \left[\frac{\tau}{2} \left(\|\mathbf{u}^{n+1}\|^2 - \|\mathbf{u}^n\|^2 + \|2\mathbf{u}^{n+1} - \mathbf{u}^n\|^2 - \|2\mathbf{u}^n - \mathbf{u}^{n-1}\|^2 \right) + \frac{\tau}{2} \|\mathbf{u}^{n+1} - 2\mathbf{u}^n + \mathbf{u}^{n-1}\|^2 \right. \\
 & + \frac{3\tau}{2} \|\mathbf{u}^{n+1} - \tilde{\mathbf{u}}^{n+1}\|^2 + \frac{2\Delta t^2}{3\tau} (\|\nabla p^{n+1}\|^2 - \|\nabla p^n\|^2) + 2\alpha\Delta t \|\tilde{\mathbf{u}}^{n+1}\|^2 \\
 & \left. + 2\Delta t Q^{n+1} \int_{\Omega} (\phi^* \nabla \mu^*) \cdot \tilde{\mathbf{u}}^{n+1} d\mathbf{x} \right] \\
 & + \left[\frac{\lambda}{2} (\|\nabla \phi^{n+1}\|^2 - \|\nabla \phi^n\|^2 + \|2\nabla \phi^{n+1} - \nabla \phi^n\|^2 - \|2\nabla \phi^n - \nabla \phi^{n-1}\|^2) \right. \\
 & + \frac{\lambda}{2} \|\nabla(\phi^{n+1} - 2\phi^n + \phi^{n-1})\|^2 \\
 & + \frac{\lambda S}{\epsilon^2} (\|\phi^{n+1} - \phi^n\|^2 - \|\phi^n - \phi^{n-1}\|^2) + \frac{2\lambda S}{\epsilon^2} \|\phi^{n+1} - 2\phi^n + \phi^{n-1}\|^2 \\
 & + \lambda U^{n+1} \int_{\Omega} H^*(3\phi^{n+1} - 4\phi^n + \phi^{n-1}) d\mathbf{x} \\
 & \left. + 2\Delta t Q^{n+1} \int_{\Omega} \nabla \cdot (\mathbf{u}^* \phi^*) \mu^{n+1} d\mathbf{x} + 2\Delta t M \|\nabla \mu^{n+1}\|^2 \right] \\
 & + \left[\lambda (|U^{n+1}|^2 - |U^n|^2 + |2U^{n+1} - U^n|^2 - |2U^n - U^{n-1}|^2 + |U^{n+1} - 2U^n + U^{n-1}|^2) \right. \\
 & \left. - \lambda U^{n+1} \int_{\Omega} H^*(3\phi^{n+1} - 4\phi^n + \phi^{n-1}) d\mathbf{x} \right] \\
 & + \left[\frac{1}{2} (|Q^{n+1}|^2 - |Q^n|^2 + |2Q^{n+1} - Q^n|^2 - |2Q^n - Q^{n-1}|^2 + |Q^{n+1} - 2Q^n + Q^{n-1}|^2) \right. \\
 & \left. - 2\Delta t Q^{n+1} \int_{\Omega} (\nabla \cdot (\mathbf{u}^* \phi^*) \mu^{n+1} + (\phi^* \nabla \mu^*) \cdot \tilde{\mathbf{u}}^{n+1}) d\mathbf{x} \right].
 \end{aligned}$$

Regrouping (cancellations).

$$\begin{aligned}
 0 = & 2\Delta t Q^{n+1} \int_{\Omega} \nabla \cdot (\mathbf{u}^* \phi^*) \mu^{n+1} d\mathbf{x} + 2\Delta t Q^{n+1} \int_{\Omega} (\phi^* \nabla \mu^*) \cdot \tilde{\mathbf{u}}^{n+1} d\mathbf{x} \\
 & - 2\Delta t Q^{n+1} \int_{\Omega} (\nabla \cdot (\mathbf{u}^* \phi^*) \mu^{n+1} + (\phi^* \nabla \mu^*) \cdot \tilde{\mathbf{u}}^{n+1}) d\mathbf{x} \\
 & + \lambda U^{n+1} \int_{\Omega} H^*(3\phi^{n+1} - 4\phi^n + \phi^{n-1}) d\mathbf{x} \\
 & - \lambda U^{n+1} \int_{\Omega} H^*(3\phi^{n+1} - 4\phi^n + \phi^{n-1}) d\mathbf{x}.
 \end{aligned}$$

Regrouping (energy difference).

$$\begin{aligned}
 E^{n+1} - E^n &= \frac{\tau}{2} (\|\mathbf{u}^{n+1}\|^2 - \|\mathbf{u}^n\|^2) + \frac{\tau}{2} (\|2\mathbf{u}^{n+1} - \mathbf{u}^n\|^2 - \|2\mathbf{u}^n - \mathbf{u}^{n-1}\|^2) \\
 &\quad + \frac{\lambda}{2} (\|\nabla\phi^{n+1}\|^2 - \|\nabla\phi^n\|^2) + \frac{\lambda}{2} (\|2\nabla\phi^{n+1} - \nabla\phi^n\|^2 - \|2\nabla\phi^n - \nabla\phi^{n-1}\|^2) \\
 &\quad + \lambda (|U^{n+1}|^2 - |U^n|^2) + \lambda (|2U^{n+1} - U^n|^2 - |2U^n - U^{n-1}|^2) \\
 &\quad + \frac{1}{2} (|Q^{n+1}|^2 - |Q^n|^2) + \frac{1}{2} (|2Q^{n+1} - Q^n|^2 - |2Q^n - Q^{n-1}|^2) \\
 &\quad + \frac{2\Delta t^2}{3\tau} (\|\nabla p^{n+1}\|^2 - \|\nabla p^n\|^2) + \frac{\lambda S}{\epsilon^2} (\|\phi^{n+1} - \phi^n\|^2 - \|\phi^n - \phi^{n-1}\|^2).
 \end{aligned}$$

Regrouping (remainder and dissipation).

$$\begin{aligned}
 R^{n+1} &= \frac{\tau}{2} \|\mathbf{u}^{n+1} - 2\mathbf{u}^n + \mathbf{u}^{n-1}\|^2 + \frac{\lambda}{2} \|\nabla(\phi^{n+1} - 2\phi^n + \phi^{n-1})\|^2 + \frac{2\lambda S}{\epsilon^2} \|\phi^{n+1} - 2\phi^n + \phi^{n-1}\|^2 \\
 &\quad + \lambda |U^{n+1} - 2U^n + U^{n-1}|^2 + \frac{1}{2} |Q^{n+1} - 2Q^n + Q^{n-1}|^2 + \frac{3\tau}{2} \|\mathbf{u}^{n+1} - \tilde{\mathbf{u}}^{n+1}\|^2, \\
 \mathcal{D}^{n+1} &= 2\Delta t \alpha \|\tilde{\mathbf{u}}^{n+1}\|^2 + 2\Delta t M \|\nabla \mu^{n+1}\|^2.
 \end{aligned}$$

The energy difference above is exactly (3.31) by definition, and $R^{n+1} \geq 0$, $\mathcal{D}^{n+1} \geq 0$. Therefore

$$E^{n+1} - E^n + \mathcal{D}^{n+1} + R^{n+1} = 0,$$

implies (3.32). $\square$

This is a clear advantage over explicit methods (which require very small Δt for stability [12, 18]) and even over many implicit schemes that still involve costly nonlinear solves [12, 13]. With Yang's scheme, one can take time steps as large as the accuracy of the solution permits, without worrying about stability.

3.7.2. Second-Order Accuracy

The use of BDF2 for time discretization ensures that the scheme is second-order accurate in time. Combined with standard spatial discretization (finite elements, spectral methods, finite differences), the overall scheme achieves second-order accuracy in the L^2 norm, or higher-order accuracy in appropriate norms depending on the smoothness of the solution. The convergence analysis of BDF schemes for nonlinear PDEs is well-established [6, 27].

For smooth solutions, error estimates of the form:

$$\|\phi^n - \phi(t_n)\| + \|\mathbf{u}^n - \mathbf{u}(t_n)\| \leq C(\Delta t^2 + h^r), \quad (3.38)$$

can be proven, where h is the spatial mesh parameter and r depends on the discretization method (typically $r = 2$ for finite elements and $r > 2$ for spectral methods). This means that the error decreases as $O(\Delta t^2)$ when the time step is refined and as $O(h^r)$ when the spatial mesh is refined. Similar convergence analyses have been performed for other SAV schemes [37, 45].

3.8. Comparison with Other Schemes

3.8.1. Advantages over Existing Methods

To appreciate the significance of Yang's scheme, it is useful to compare it with the numerical approaches discussed in Chapter 2:

- vs. Fully implicit methods:** Yang's scheme requires no nonlinear iterations. Each time step solves only linear systems, reducing computational cost relative to nonlinear solves [44].
- vs. Convex splitting schemes:** While convex splitting is also unconditionally stable [2, 12], it retains some nonlinearity requiring iterations at each time step [13]. Yang's scheme is fully linear and requires solving only simple elliptic equations with constant coefficients.
- vs. Explicit methods:** Explicit methods are cheap per time step but require very small Δt for stability [12, 18], making them infeasible for small ϵ . Yang's scheme allows $\Delta t = O(1)$, resulting in fewer total time steps and better overall efficiency.
- vs. Semi-implicit stabilized schemes:** These schemes attempt to balance accuracy and stability but often require parameter choices to maintain stability [36]. Yang's scheme requires no such constraints and is unconditionally stable.

A quantitative comparison is provided in Table 3.1.

Yang's scheme combines all four desirable properties: linearity, unconditional stability, full decoupling, and no time-step restrictions [44]. This combination of properties has made it an attractive choice for practitioners implementing numerical simulations of Cahn–Hilliard systems [34].

Table 3.1.: Comparison of numerical schemes for the CHD system.

Scheme	Linear	Unconditional Stable	Decoupled	Time Step Limit
Fully implicit	No	Yes	No	None
Convex splitting	No	Yes	Partial	None
Stabilized semi-implicit	Partial	Partial	Partial	$O(\Delta t_{\text{crit}})$
Yang's SAV scheme	Yes	Yes	Yes	None

3.9. Initialization and Practical Considerations

3.9.1. First Time Step Initialization

Since the time discretization uses BDF2, which is a two-step method, we need values at $t = 0$ and $t = \Delta t$ to begin. The value at $t = 0$ is provided by the initial condition, but $t = \Delta t$ must be computed using a first-order scheme. A common approach is to use the backward Euler method (BDF1) for the first time step [44]:

$$\frac{\psi^1 - \psi^0}{\Delta t} \approx \left. \frac{\partial \psi}{\partial t} \right|_{t=\Delta t}, \quad (3.39)$$

This gives a first-order accurate approximation at the first time step, after which the second-order BDF2 scheme kicks in for $n \geq 1$. The overall accuracy remains second-order because the first time step is only one step out of many (as $\Delta t \rightarrow 0$, the contribution of the first step to the total error vanishes). This approach has been analyzed in detail in the context of BDF methods for ODEs and PDEs [6, 22].

3.9.2. Choice of Parameters

The scheme involves several parameters that must be chosen:

1. **Interface width** ϵ : This is a problem parameter, not a numerical parameter. It controls the thickness of the transition region and should be chosen small enough to resolve the interface but not so small that it introduces severe stiffness. The choice of ϵ affects the stiffness and the accuracy of interface resolution [31, 10].
2. **Mobility** M : This parameter affects the speed of phase separation. It can be varied to study the effect of diffusion speed on the dynamics [4].

3. **Surface tension λ** : This parameter controls the strength of surface tension effects in the free-energy functional [44, 31].
4. **Stabilization parameter S** : This is the only truly numerical parameter introduced by the scheme. It controls the amount of stabilization added to the scheme. Too small S can lead to instabilities for large time steps, while very large S can introduce spurious dissipation [44].
5. **Constant B** : This appears in the definition of U (Equation (3.1)). It must be chosen large enough that $\int_{\Omega} F(\phi) d\mathbf{x} + B > 0$ always [37].

3.9.3. Time Step Selection

Despite unconditional stability, the choice of time step Δt still affects accuracy. The error estimate (Equation (3.38)) shows that error decreases as Δt^2 as long as the time step is not too large. In practice, Δt should be chosen such that the temporal error is comparable to the spatial error $\sim h^r$. A typical guideline is:

$$\Delta t \sim h^{r/2}, \quad (3.40)$$

For spectral methods where $r \sim 10$ or higher, this suggests very small time steps. In practice, one balances temporal and spatial errors based on accuracy requirements [6, 27].

3.10. Summary

Yang's decoupled SAV scheme represents a significant advancement in numerical methods for the CHD system and related coupled systems. By combining three key ideas:

1. The Scalar Auxiliary Variable (SAV) method to linearize the nonlinear potential [37],
2. The zero-energy-contribution property to introduce a decoupling variable Q [44],
3. Proper time discretization with BDF2 and spatial discretization with linear methods [44],

the scheme achieves a combination of properties: full linearity, unconditional energy stability, complete decoupling, and second-order accuracy. This makes it well suited for implementation in numerical software and for studying complex phenomena in two-phase flows [44, 34].

Chapter 4 shows how these theoretical ideas translate into concrete computational algorithms and how to implement the scheme efficiently in Python. The mathematical framework developed in this chapter—particularly the decoupling strategy and the energy analysis—guides the implementation and supports the numerical results presented in Chapter 5.

Before entering the implementation details, we summarize the computational workflow. Given initial fields $(\phi, \mathbf{u}, p)$, model parameters, and a time step Δt , each BDF2 step extrapolates $\phi^*, \mathbf{u}^*, \mu^*$, forms $H(\phi^*)$, solves linear elliptic subproblems for the phase update and auxiliary variables, updates U and Q , and then applies a pressure projection to enforce incompressibility in the velocity update. This loop advances the solution in time and produces snapshot outputs for the coarsening, spinodal, and convergence tests.

4. Numerical Implementation of the Fully-Decoupled SAV Scheme

Having established the mathematical theory and decoupling strategy in Chapter 3, we now describe how to implement Yang’s scheme for the Cahn–Hilliard–Darcy system. We translate the decoupled SAV equations into an efficient numerical code, focusing on Fourier spectral discretization, linear elliptic solves at each substep, and the overall algorithm structure.

4.1. Spatial Discretization Strategy

4.1.1. Choice of Spectral Methods

The choice of spatial discretization balances accuracy, computational efficiency, and implementation effort. For the CHD system with periodic boundary conditions—a natural choice for studying phase separation and spinodal decomposition without boundary effects—Fourier spectral methods are a natural fit [44, 5]. Specifically, we employ the **Fast Fourier Transform (FFT)** in the periodic spatial directions.

The primary advantages are:

1. **Superior accuracy for smooth functions:** Spectral methods achieve exponential convergence for smooth solutions, far exceeding finite difference or finite element methods which converge algebraically [5]. In the CHD setting, where the phase field is smooth except near the interface, spectral accuracy is attainable and valuable.
2. **Computational efficiency for linear operators:** The Fourier-Galerkin representation of differentiation is diagonal, so solving equations involving Δ and Δ^2 reduces to element-wise division in Fourier space rather than sparse-matrix assembly [5].
3. **Vectorization and parallelization:** Modern FFT algorithms, particularly the

Cooley-Tukey FFT and its variants, reduce the transform cost to $O(N \log N)$. Libraries like FFTW provide efficient CPU implementations [8, 14, 35].

4. **Simplified periodic boundary conditions:** Periodic domains are naturally handled by FFT without additional boundary treatment. This avoids the complexity of implementing Dirichlet or Neumann boundary conditions through ghost cells or boundary operators.

A limitation of Fourier spectral methods is the requirement of periodic boundaries, and spectral methods are generally less flexible for complex geometries. For the canonical problems studied in this thesis (spinodal decomposition and coarsening), the periodic domain is natural and the gains in efficiency and accuracy outweigh these limitations [5].

4.1.2. Fourier Discretization and Wavenumber Grid

We discretize the periodic domain $\Omega = [0, L_x] \times [0, L_y]$ with uniform mesh points:

$$x_i = i\Delta x, \quad y_j = j\Delta y, \quad i, j = 0, 1, \dots, N-1, \quad (4.1)$$

where $\Delta x = L_x/N_x$ and $\Delta y = L_y/N_y$ are the spatial mesh sizes. The phase field ϕ and other variables are represented as values on this grid. In Fourier space, we work with wavenumbers:

$$k_m = \frac{2\pi m}{L_x}, \quad m = -\frac{N_x}{2}, \dots, \frac{N_x}{2} - 1, \quad (4.2)$$

and similarly for the y -direction. The magnitude of the wavenumber vector $\mathbf{k} = (k_x, k_y)$ is:

$$K^2 = k_x^2 + k_y^2. \quad (4.3)$$

All differential operators in the equations are represented in Fourier space as multipliers. For example, the Laplacian operator becomes multiplication by $-K^2$:

$$\widehat{\Delta\phi}(\mathbf{k}) = -K^2\hat{\phi}(\mathbf{k}), \quad (4.4)$$

where $\hat{\phi}$ denotes the Fourier transform of ϕ . The biharmonic operator (the Laplacian of the Laplacian, appearing in the Cahn–Hilliard equation) becomes multiplication by $(-K^2)^2 = K^4$:

$$\widehat{\Delta^2\phi}(\mathbf{k}) = K^4\hat{\phi}(\mathbf{k}). \quad (4.5)$$

4.1.3. Avoiding Aliasing and Dealiasing

A technical issue with spectral methods is **aliasing**, which occurs when nonlinear terms are evaluated in Fourier space without care. Specifically, if we compute the Fourier transform of a product such as ϕ^3 , the result contains high-frequency modes that can alias (fold back) to lower frequencies and contaminate the solution [5].

The standard solution, called **2/3-rule dealiasing**, is to compute nonlinear terms in physical space (after inverse FFT), then transform the result back to Fourier space. Another approach, the 3/2-rule, pads the domain with zeros in Fourier space before transforming nonlinear products; however, this adds computational overhead. For the CHD system where nonlinearities are relatively mild (the cubic term in $f(\phi) = \frac{1}{\epsilon^2}(\phi^3 - \phi)$ is the main nonlinearity, and it is handled through the SAV auxiliary variable), the 2/3-rule is a common choice in practice [5].

4.2. Linear Elliptic Solvers for Each Substep

4.2.1. General Structure of Implicit Equations

The time discretization produces a sequence of implicit equations that must be solved at each time step. Due to the linearity of Yang's scheme and the use of Fourier spectral discretization, these equations all have the form:

$$\mathcal{L}[\psi^{n+1}] = \mathcal{R}^n, \quad (4.6)$$

where $\mathcal{L}$ is a linear differential operator (typically containing Laplacian and possibly Biharmonic terms), and $\mathcal{R}^n$ is a right-hand side computed from known quantities at the previous time step(s). In Fourier space, this becomes algebraic:

$$L(K^2)\hat{\psi}^{n+1}(\mathbf{k}) = \hat{\mathcal{R}}^n(\mathbf{k}), \quad (4.7)$$

where $L(K^2)$ is a polynomial in K^2 . The solution is then:

$$\hat{\psi}^{n+1}(\mathbf{k}) = \frac{\hat{\mathcal{R}}^n(\mathbf{k})}{L(K^2)}, \quad (4.8)$$

which is solved by element-wise division in Fourier space, followed by inverse FFT to recover ψ^{n+1} in physical space. This avoids sparse-matrix assembly and leverages the diagonal Fourier representation [5].

4.2.2. Phase-Field Equation

The phase-field equation in the fully decoupled time-marching scheme (Equation (3.19)) can be rewritten in semi-discrete form after time discretization using BDF2:

$$\frac{3\phi^{n+1} - 4\phi^n + \phi^{n-1}}{2\Delta t} + Q^* \nabla \cdot (\mathbf{u}^* \phi^*) = M \Delta \mu^{n+1}, \quad (4.9)$$

where $\mu^{n+1} = \lambda(-\Delta \phi^{n+1} + H(\phi^*)U^{n+1} + \frac{S}{\epsilon^2}(\phi^{n+1} - \phi^*))$ with $\phi^* = 2\phi^n - \phi^{n-1}$ (explicit extrapolation). We use a lagged scalar $Q^* = Q^n$ (or $Q^* = 1$ at startup) in the explicit advection term to keep the phase solve decoupled. Here $\mathbf{u}^* = 2\mathbf{u}^n - \mathbf{u}^{n-1}$ is the explicit velocity extrapolation.

Substituting the expression for μ^{n+1} into the diffusion term and rearranging, we obtain a linear equation for ϕ^{n+1} . We treat advection explicitly (using extrapolated $\mathbf{u}^*$ and ϕ^*) and keep diffusion/SAV terms implicit. This leads to:

$$\left(\frac{3}{2\Delta t} + M\lambda K^4 + M\lambda \frac{S}{\epsilon^2} K^2 \right) \hat{\phi}^{n+1} = \frac{4\hat{\phi}^n - \hat{\phi}^{n-1}}{2\Delta t} - Q^* \widehat{\nabla \cdot (\mathbf{u}^* \phi^*)} + M\lambda K^2 \widehat{H(\phi^*)U^{n+1}} - \frac{S}{\epsilon^2} \phi^*, \quad (4.10)$$

where all quantities with hats denote Fourier transforms. The left-hand side coefficient is a polynomial in K^2 , and the right-hand side is computed in physical space (for nonlinear terms) and then transformed to Fourier space. The solution for $\hat{\phi}^{n+1}$ is obtained by element-wise division.

A subtle but important point is the **zero mode** ($K^2 = 0$, corresponding to the spatially constant part). For this mode, the term K^4 and K^2 both vanish, and we must ensure that we do not divide by zero. In practice, we set $K^2|_{(0,0)} = 10^{-10}$ (a small regularization) to avoid division by zero while not significantly affecting the dynamics. Alternatively, one can project out the constant mode or use conservation constraints; for this thesis, the regularization approach is used for simplicity.

4.2.3. Velocity and Pressure (Darcy Step)

The momentum equation (Equation (3.22)) in time-discrete form is:

$$\tau \frac{3\tilde{\mathbf{u}}^{n+1} - 4\mathbf{u}^n + \mathbf{u}^{n-1}}{2\Delta t} + \alpha \tilde{\mathbf{u}}^{n+1} + \nabla p^n + Q^* \phi^* \nabla \mu^* = 0, \quad (4.11)$$

where $\phi^* = 2\phi^n - \phi^{n-1}$ and $\mu^* = 2\mu^n - \mu^{n-1}$ are explicit extrapolations, and $Q^* = Q^n$ is the lagged scalar used in the explicit coupling. Combined with the projection step and incompressibility constraint, this forms a standard pressure-correction (projection) system.

To solve this, we use a **pressure-correction (projection) method**: first solve for an intermediate velocity with lagged pressure, then correct it with a pressure Poisson solve to enforce $\nabla \cdot \mathbf{u}^{n+1} = 0$ [20, 21]. The three steps are:

1. **Advection-diffusion substep (pressure lagged):**

$$\left(\frac{3\tau}{2\Delta t} + \alpha\right) \tilde{\mathbf{u}}^{n+1} = \frac{\tau}{2\Delta t} (4\mathbf{u}^n - \mathbf{u}^{n-1}) - \nabla p^n - Q^* \phi^* \nabla \mu^*, \quad (4.12)$$

where the capillary force is computed explicitly from ϕ^* and μ^* . In Fourier space:

$$\left(\frac{3\tau}{2\Delta t} + \alpha\right) \hat{\tilde{\mathbf{u}}}^{n+1} = \frac{\tau}{2\Delta t} (4\hat{\mathbf{u}}^n - \hat{\mathbf{u}}^{n-1}) - i\mathbf{k}\hat{p}^n - Q^* \widehat{\phi^* \nabla \mu^*}. \quad (4.13)$$

The intermediate velocity is then obtained by inverse FFT.

2. **Pressure Poisson equation:**

Taking the divergence of the projection equation (3.24) and using $\nabla \cdot \mathbf{u}^{n+1} = 0$ gives a Poisson equation for the pressure increment:

$$\Delta(p^{n+1} - p^n) = \frac{3\tau}{2\Delta t} \nabla \cdot \tilde{\mathbf{u}}^{n+1}. \quad (4.14)$$

In Fourier space, this becomes:

$$-K^2(\widehat{p^{n+1} - p^n}) = \frac{3\tau}{2\Delta t} i\mathbf{k} \cdot \hat{\tilde{\mathbf{u}}}^{n+1}, \quad (4.15)$$

which is solved by element-wise division (again treating the zero mode carefully). Boundary conditions for the pressure (typically, $\partial_n p = 0$ at walls, or periodic in our case) are naturally satisfied in Fourier space.

3. **Velocity correction:**

$$\mathbf{u}^{n+1} = \tilde{\mathbf{u}}^{n+1} - \frac{2\Delta t}{3\tau} \nabla(p^{n+1} - p^n), \quad (4.16)$$

where the pressure gradient is computed in Fourier space and transformed back to physical space.

This projection method is standard in incompressible flow solvers and has been rigorously analyzed [20, 21]. For our fully decoupled scheme, it avoids the need to solve a coupled saddle-point system, reducing computational cost.

4.2.4. Auxiliary Variables U and Q

The auxiliary variables U^{n+1} and Q^{n+1} are scalar functions (depending only on time, not space). Their BDF2 updates follow the scheme in Chapter 3:

$$3U^{n+1} - 4U^n + U^{n-1} = \frac{1}{2} \int_{\Omega} H(\phi^*) (3\phi^{n+1} - 4\phi^n + \phi^{n-1}) d\mathbf{x}, \quad (4.17)$$

$$3Q^{n+1} - 4Q^n + Q^{n-1} = 2\Delta t \int_{\Omega} [\nabla \cdot (\mathbf{u}^* \phi^*) \mu^{n+1} + (\phi^* \nabla \mu^*) \cdot \tilde{\mathbf{u}}^{n+1}] d\mathbf{x}. \quad (4.18)$$

These integrals are computed with trapezoidal quadrature, which is spectrally accurate for smooth integrands. The key observation is that Q^n remains very close to 1 throughout the simulation (it is exactly 1 at the continuous level due to the zero-energy-contribution property, and the discrete version deviates only due to discretization error). Accordingly, we evaluate the explicit coupling terms with Q^n and update Q^{n+1} after $\tilde{\mathbf{u}}^{n+1}$ is available.

4.3. Overall Time-Stepping Algorithm

4.3.1. Algorithm Structure

At time t_n with known values $(\phi^n, \phi^{n-1}, \mu^n, \mu^{n-1}, \mathbf{u}^n, \mathbf{u}^{n-1}, U^n, Q^n)$, the following steps are performed to advance to $t^{n+1} = t^n + \Delta t$:

Algorithm 1 One time step of the fully decoupled SAV scheme for the CHD system

- 1: **Input:** $\phi^n, \phi^{n-1}, \mu^n, \mu^{n-1}, \mathbf{u}^n, \mathbf{u}^{n-1}, U^n, Q^n, \Delta t$
 - 2: **Step 1 (Extrapolation):** Compute $\phi^* = 2\phi^n - \phi^{n-1}$ and $\mathbf{u}^* = 2\mathbf{u}^n - \mathbf{u}^{n-1}$
 - 3: **Step 2:** Compute $H(\phi^*) = \frac{f(\phi^*)}{\sqrt{\int_{\Omega} F(\phi^*) d\mathbf{x} + B}}$ in physical space
 - 4: **Step 3:** Compute RHS of Eq. (4.10) via FFT using Q^n in the explicit advection term
 - 5: **Step 4:** Solve for ϕ^{n+1} in Fourier space (element-wise division), then inverse FFT
 - 6: **Step 5 (Update U):** Compute U^{n+1} from Eq. (4.17) (solve the BDF2 scalar update)
 - 7: **Step 6 (Chemical potential):** Compute $\mu^{n+1} = \lambda(-\Delta\phi^{n+1} + H(\phi^*)U^{n+1} + \frac{S}{\epsilon^2}(\phi^{n+1} - \phi^*))$ via FFT
 - 8: **Step 7 (Capillary force):** Compute the explicit coupling term $\mathbf{f}_{\text{cap}}^* = -Q^n \phi^* \nabla \mu^*$ in physical space (lagged Q^n), with $\mu^* = 2\mu^n - \mu^{n-1}$
 - 9: **Step 8 (Intermediate velocity):** Solve Eq. (4.13) for $\tilde{\mathbf{u}}^{n+1}$ in Fourier space, then inverse FFT
 - 10: **Step 9 (Pressure increment):** Solve Eq. (4.15) for $(p^{n+1} - p^n)$ in Fourier space, then set $p^{n+1} = p^n + (p^{n+1} - p^n)$
 - 11: **Step 10 (Velocity correction):** Compute $\mathbf{u}^{n+1} = \tilde{\mathbf{u}}^{n+1} - \frac{2\Delta t}{3\tau} \nabla(p^{n+1} - p^n)$
 - 12: **Step 11 (Update Q):** Compute Q^{n+1} from Eq. (4.18) (typically $Q^{n+1} \approx 1$)
 - 13: **Output:** $\phi^{n+1}, \mathbf{u}^{n+1}, U^{n+1}, Q^{n+1}$
-

The algorithm is dominated by FFTs, pointwise operations, and quadrature; it avoids nonlinear iterations and sparse matrix solves, which explains its efficiency.

4.4. Implementation Details and Practical Considerations

4.4.1. FFT Backend and Parallelization

The implementation uses Python/NumPy for array operations and **SciPy's FFT module** (`scipy.fft`) as the default FFT backend. For better performance on multicore CPUs, the code can be configured to use **pyFFTW** (Python bindings to the FFTW library), which supports multithreading. On GPU accelerators (NVIDIA GPUs), the **CuPy** library provides CUDA-accelerated FFT (via `cuFFT`) and array operations.

GPU acceleration is valuable because:

1. **High memory bandwidth:** GPUs have much higher memory bandwidth than CPUs, and FFT operations are bandwidth-bound. A typical GPU (e.g., NVIDIA GeForce RTX 4070 SUPER with 12 GB memory) can achieve FFT throughput 5–10 times higher than a CPU for large arrays.

2. **Parallelization across elements:** Array operations (pointwise function evaluations, quadrature) are easily parallelized on GPUs, with minimal communication overhead.
3. **Convergence studies:** The temporal convergence tests in Chapter 5 require many independent runs at different time step sizes. Executing these in parallel on a GPU is more efficient than serial CPU execution.

For the convergence study in Chapter 5, GPU acceleration provides only a modest improvement on our system, reducing wall-clock time from ~ 13 hours on CPU to ~ 11 hours on GPU.

4.4.2. Initialization and First Time Step

BDF2 is a two-level method, requiring values at both $t = 0$ and $t = \Delta t$. The value at $t = 0$ is provided by the initial condition. The value at $t = \Delta t$ is computed using a first-order method (backward Euler / BDF1) to avoid the need for a separate initialization procedure:

$$\frac{\psi^1 - \psi^0}{\Delta t} = \text{RHS}^1, \quad (\text{BDF1 at step 1}) \quad (4.19)$$

From $n = 1$ onward, BDF2 is used; the single BDF1 step does not affect overall second-order accuracy.

4.4.3. Parameter Choices and Tuning

The scheme involves several parameters:

Interfacial width ϵ : Controls the thickness of the diffuse interface. Smaller ϵ gives sharper interfaces but requires finer resolution and can introduce numerical stiffness. A practical rule of thumb is that ϵ should be resolved by a few grid points, i.e., $\epsilon = O(\Delta x)$. In our simulations, $\epsilon = 0.05$ on a 128×128 grid with $\Delta x = 2\pi/128 \approx 0.049$, giving $\epsilon \approx \Delta x$, which is adequate for the spectral discretization.

Stabilization parameter S : Controls the strength of the numerical stabilization term added to the chemical potential. Too small S (e.g., $S = 0$) can lead to numerical instability for large time steps; too large S introduces unwanted damping. A typical choice is $S = 2$ – 10 . In the coarsening test (Chapter 5), $S = 2$; in the spinodal decomposition test (Chapter 5), $S = 10$.

Constant B : Ensures that the argument of the square root in $U = \sqrt{\int_{\Omega} F(\phi) d\mathbf{x} + B}$ stays positive. We use $B = 10$, which is large enough for the typical range of $F(\phi)$ encountered and does not significantly affect the dynamics.

Time step Δt : Despite unconditional energy stability, Δt still affects accuracy. For smooth solutions, the scheme is second-order accurate in time, so error decreases as Δt^2 . In our simulations, we use $\Delta t = 0.001$ – 0.005 with $N = 128$ – 512 .

We follow parameter values and stabilizer choices reported in Yang (2021) for similar tests [44].

4.4.4. Energy and Mass Conservation Monitoring

The scheme is designed to satisfy discrete energy dissipation (unconditional energy stability), but it is useful to monitor the discrete energy and mass conservation in actual simulations to detect bugs or numerical issues. We monitor the discrete energy from Theorem 3.1, evaluated at time level n :

$$\begin{aligned} E^n = & \frac{\tau}{2} (\|\mathbf{u}^n\|^2 + \|2\mathbf{u}^n - \mathbf{u}^{n-1}\|^2) + \frac{\lambda}{2} (\|\nabla\phi^n\|^2 + \|2\nabla\phi^n - \nabla\phi^{n-1}\|^2) \\ & + \lambda (|U^n|^2 + |2U^n - U^{n-1}|^2) + \frac{1}{2} (|Q^n|^2 + |2Q^n - Q^{n-1}|^2) \\ & + \frac{2\Delta t^2}{3\tau} \|\nabla p^n\|^2 + \frac{\lambda S}{\epsilon^2} \|\phi^n - \phi^{n-1}\|^2. \end{aligned} \quad (4.20)$$

In a correct implementation, E^n should be monotone decreasing (or non-increasing): $E^{n+1} \leq E^n$. Deviations from this indicate a coding error or invalid parameter choices. Mass conservation (conservation of $\int_{\Omega} \phi^n d\mathbf{x}$) is automatically satisfied by the divergence-free velocity and the structure of the scheme.

4.5. GPU Acceleration and Computational Infrastructure

4.5.1. Why GPU Acceleration?

While the fully decoupled SAV scheme is more efficient than fully implicit or convex-splitting schemes, it is still computationally intensive for large problems. The primary bottleneck is the FFT operations, which dominate the wall-clock time. Modern GPUs (Graphics Processing Units) are optimized for FFT through specialized libraries

(NVIDIA cuFFT, AMD rocFFT), and the memory bandwidth of GPUs far exceeds that of CPUs. This makes GPU acceleration natural for this problem.

Key advantages:

1. **Speed:** For large grids ($N \geq 256$), speedups of several-fold to an order of magnitude are often achievable, though the realized gain depends on hardware and problem size.
2. **Feasibility:** For large, long simulations (e.g., the 512^2 spinodal runs), GPU acceleration reduced wall-clock time from hours to minutes. For smaller grids or short runs, the benefit can be modest because fixed overheads (initialization, FFT planning, kernel launch) are a larger fraction of total time.
3. **Parallelization:** Multiple simulations can be run on different GPUs simultaneously, enabling parameter studies.
4. **Memory efficiency:** The RTX 4070 SUPER provides 12 GB of memory, which was sufficient for the 512×512 cases reported in Chapter 5. Larger grids may become GPU-memory bound and require careful memory management or CPU execution.

4.5.2. Implementation Platform: AMD Ryzen 7 7800X3D and NVIDIA GeForce RTX 4070 SUPER

For this thesis, simulations were run on a system with an AMD Ryzen 7 7800X3D 8-core CPU and an NVIDIA GeForce RTX 4070 SUPER GPU. The GPU provides 12 GB of onboard memory, which is sufficient for the 512×512 cases reported in Chapter 5.

- CPU: AMD Ryzen 7 7800X3D (8-core)
- GPU: NVIDIA GeForce RTX 4070 SUPER (12 GB)

The GPU is accessed via the CUDA API, using Python libraries: CuPy (NumPy-like GPU arrays) and cuFFT (via CuPy's FFT module). The workflow is:

1. Allocate GPU memory for arrays (ϕ , $\mathbf{u}$, etc.)
2. Transfer initial data to GPU (one-time cost)
3. Run time-stepping loop entirely on GPU (FFT, pointwise operations, quadrature)

4. Transfer final result back to CPU for visualization and saving

4.6. CFL-Like Constraints and Time-Step Stability

4.6.1. Absence of CFL Constraints in Energy-Stable Schemes

Explicit numerical schemes for hyperbolic PDEs (e.g., conservation laws) are subject to CFL (Courant-Friedrichs-Lewy) conditions, restricting the time step as $\Delta t \leq Ch/u_{\max}$ (time step bounded by grid size divided by characteristic velocity) [42, 29]. The CFL condition is a necessary stability condition for explicit schemes: it ensures that the domain of dependence (the set of grid points whose values at time t depend on a given point) is contained within the domain of influence (the set of grid points that influence the solution at the current time step). Violating the CFL condition leads to nonphysical oscillations and eventual numerical blow-up [29, 42]. For the Cahn–Hilliard equation with explicit time stepping, stability constraints become more restrictive due to the fourth-order operator and stiffness in the nonlinear term, making explicit schemes impractical for small interface widths ϵ [12].

In contrast, Yang’s fully decoupled SAV scheme satisfies a discrete energy dissipation inequality (Equation (3.32)) unconditionally—that is, for *any* choice of $\Delta t > 0$. This means that the scheme is **unconditionally stable**: the energy cannot grow unboundedly, and oscillations cannot develop and amplify, regardless of how large Δt is chosen (within reason, to maintain accuracy) [44, 38]. The unconditional stability property is a defining feature of implicit schemes designed using gradient-flow structures and energy-stable variational methods [38, 12].

More formally, a numerical scheme for a gradient flow (or more generally, a dissipative PDE) is said to be **unconditionally energy stable** if the discrete energy functional satisfies:

$$E^{n+1} \leq E^n \quad \text{for all } \Delta t > 0, \tag{4.21}$$

without any restriction on the size of the time step [38]. This is precisely the property proven theoretically for Yang’s scheme in Chapter 3 and verified numerically in all test cases. The absence of CFL constraints enables the use of larger time steps (limited only by accuracy requirements) [44, 39].

4.6.2. Accuracy vs Stability Trade-off

While the scheme is unconditionally stable, accuracy still depends on Δt . For a given spatial discretization (grid size h), the temporal error scales as:

$$\text{Temporal Error} \sim (\Delta t)^p, \quad (4.22)$$

where p is the order of accuracy in time. For the BDF2 time discretization used in this thesis, $p = 2$ (second-order accurate) [22]. Thus, if one wishes to achieve a specific accuracy (say, error $\sim 10^{-3}$), one must choose Δt small enough. However, unlike explicit schemes, there is no hard stability constraint on Δt —only an accuracy constraint [38].

This fundamental difference between conditional (CFL-based) and unconditional stability has practical implications. In explicit schemes, CFL-type constraints can force the time step to shrink as the grid is refined to maintain stability [29]. In unconditionally energy-stable schemes, the time step can instead be selected based primarily on accuracy considerations [38].

4.6.3. Balancing Temporal and Spatial Errors

In practice, for long-time simulations, one typically chooses Δt such that the temporal error is comparable to the spatial error, achieving a balanced overall accuracy. For spectral methods with high spatial accuracy, a heuristic balance gives:

$$(\Delta t)^2 \sim h^r \implies \Delta t \sim h^{r/2}. \quad (4.23)$$

This relationship suggests that larger time steps are appropriate for coarser grids, and the time step grows more slowly than the grid spacing. For the simulations in this thesis, we use Δt values ranging from 6.25×10^{-4} (coarsening) to 10^{-2} (convergence test), with $\Delta x \approx 0.05$ – 0.01 (corresponding to $N = 128$ to 512).

4.6.4. Stability Verification via Energy Plots

To verify stability and accuracy, it is instructive to plot the discrete energy functional E^n (Equation (4.20)) over time. For a correctly implemented unconditionally energy-stable scheme, E^n should be monotonically non-increasing: $E^{n+1} \leq E^n$ (or strictly decreasing, depending on the dissipation structure) [38, 12]. Deviation from this behavior (e.g., energy increasing or oscillating) signals one of the following:

1. A coding error in the implementation of the scheme (e.g., sign error in the dissipative terms, incorrect discretization of the auxiliary variables).

2. Invalid parameter choices (e.g., stabilization parameter S too large, causing over-damping; or incorrect scaling of nonlinear terms).
3. Time step chosen too large, causing the discrete energy dissipation law to be violated (though for an unconditionally stable scheme, this should not happen unless there is a bug in the code).
4. Numerical precision issues or round-off errors accumulating over very long simulations with very small errors.

Monotonic energy decay is a useful diagnostic for confirming that the discrete energy law is satisfied in the implementation [38, 12].

4.7. Summary

This chapter details the numerical implementation of Yang’s fully decoupled SAV scheme for the CHD system. The key points are:

1. **Spatial discretization:** Fourier spectral methods with FFT provide superior accuracy and efficiency for periodic domains, reducing differential operators to point-wise multiplications in Fourier space.
2. **Implicit solvers:** Each substep (phase field, velocity, pressure) is solved by FFT and element-wise division, avoiding nonlinear iterations or sparse matrix solves.
3. **Algorithm structure:** A clear sequence of steps (Algorithm 1) translates the theory into code, involving ~ 10 – 15 FFT operations per time step and simple quadrature for auxiliary variables.
4. **Convergence:** The method is second-order in time and spectrally accurate in space; these properties are verified in Chapter 5.
5. **GPU acceleration:** Modern GPUs can provide several-fold to order-of-magnitude speedups in favorable cases, enabling feasible computation of long-time or high-resolution simulations.
6. **Energy stability:** The discrete energy monitor is introduced as a practical diagnostic and is used in Chapter 5 to confirm monotone energy decay.

With this implementation in place, we are now prepared to present the numerical results and validation studies in Chapter 5.

5. Numerical Results and Validation Studies

This chapter reports results from the fully decoupled SAV implementation in Chapter 4. We validate the code against three canonical test cases from Yang (2021) and compare qualitatively with the literature to demonstrate accuracy, stability, and efficiency. Simulations use the Python solvers documented in the Appendix, with optional GPU acceleration.

5.1. Overview of Numerical Experiments

The validation strategy consists of three complementary studies:

1. **Coarsening Test (Figure 5.1):** Two droplets of different sizes evolve under capillary-driven dynamics, demonstrating Ostwald ripening and long-time interface preservation.
2. **Spinodal Decomposition Test (Figures 5.2 and 5.3):** Phase separation from a metastable homogeneous state in symmetric ($\phi_{\text{avg}} = 0$) and asymmetric ($\phi_{\text{avg}} = 0.3$) compositions, highlighting fine-scale morphology and coarsening dynamics.
3. **Temporal Convergence Test (Figure 5.4):** Systematic time-step refinement to verify second-order convergence for DSAV and compare with incomplete variants (EX-SAV, AV).

All experiments employ Fourier spectral methods with periodic boundary conditions on the domain $\Omega = [0, 2\pi]^2$.

5.2. Coarsening Test: Two-Droplet Dynamics

5.2.1. Initial Condition and Parameters

The initial phase field $\phi_0(\mathbf{x})$ is constructed as two tanh-profile circular droplets centered at $\mathbf{x}_1 = (\pi - 0.8, \pi)$ with radius $r_1 = 1.4$ and $\mathbf{x}_2 = (\pi + 1.7, \pi)$ with radius $r_2 = 0.5$:

$$\phi_0(\mathbf{x}) = 1.0 + \tanh\left(\frac{r_1 - |\mathbf{x} - \mathbf{x}_1|}{1.5\epsilon}\right) + \tanh\left(\frac{r_2 - |\mathbf{x} - \mathbf{x}_2|}{1.5\epsilon}\right). \quad (5.1)$$

The numerical parameters $N_x = N_y = 128$, $\tau = 1$, $\alpha = 100$, $M = 1$, $\lambda = 0.01$, $\epsilon = 0.05$, $S = 2$, and $B = 10$ follow the coarsening setup in [44]. We use $\Delta t = 0.01/2^4 = 6.25 \times 10^{-4}$ and run the simulation from $t = 0$ to $t = 5$ (about 8000 time steps), with snapshots captured at times $t = 0, 1.4, 1.9, 1.95, 2.45, 5.0$.

5.2.2. Results and Physical Interpretation

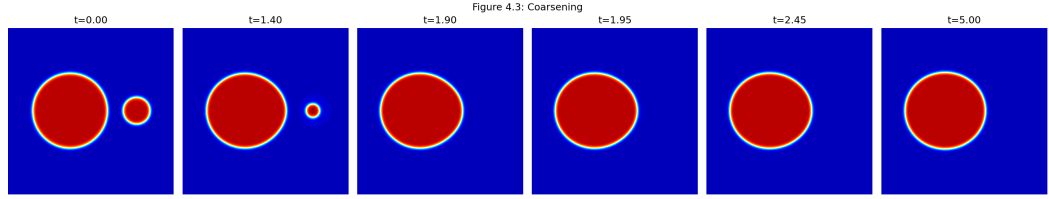


Figure 5.1.: Coarsening dynamics of two-droplet system. The large droplet (left) grows while the small droplet (right) shrinks due to capillary pressure differences, demonstrating Ostwald ripening. The color map shows $\phi \in [-1, 1]$ with blue representing the background phase and red/yellow the dispersed phase. White contours mark the interface at $\phi = 0$. Snapshots at times $t = 0.00, 1.40, 1.90, 1.95, 2.45, 5.00$ show the temporal evolution over the coarsening process.

Figure 5.1 shows six snapshots of the phase field ϕ evolving from the initial two-droplet configuration. Key observations include:

1. **Early times ($t = 0$ to $t = 1.95$):** The initial tanh profiles smooth slightly as the interface relaxes; the small droplet begins to shrink while the large droplet grows, consistent with curvature-driven Ostwald ripening.

2. **Intermediate times** ($t = 1.95$ to $t = 2.45$): The small droplet continues to shrink and separates from the large droplet. The interface remains well defined without spurious oscillations.
3. **Late times** ($t = 5.0$): By the end of the simulation, the small droplet is absorbed by the large droplet. The system approaches minimal interfacial area and the interface remains smooth, indicating long-time stability.

5.3. Spinodal Decomposition: Phase Separation Dynamics

5.3.1. Problem Formulation

Spinodal decomposition is modeled by initializing the phase field with a small random perturbation around a uniform average composition ϕ_{avg} :

$$\phi_0(\mathbf{x}) = \phi_{\text{avg}} + \delta\phi(\mathbf{x}), \quad \delta\phi = 0.001 \times (2\text{rand}(\mathbf{x}) - 1). \quad (5.2)$$

where $\text{rand}(\mathbf{x}) \in [0, 1]$ is a uniform random number. The random noise triggers the instability of the metastable uniform state, causing phase separation into domains of $\phi \approx +1$ and $\phi \approx -1$.

Parameters: $N_x = N_y = 512$, $\Delta t = 0.001$, $\tau = 1$, $\alpha = 100$, $M = 1$, $\lambda = 0.01$, $\epsilon = 0.025$, $S = 10$, and $B = 10$, following [44]. The finer grid and smaller ϵ resolve sharper interfaces, while the larger S offsets the added stiffness. Two cases are studied:

Case 5.2a: Symmetric ($\phi_{\text{avg}} = 0$): Equal amounts of both phases are present initially. The system phase-separates into two interpenetrating phases of approximately equal volume.

Case 5.2b: Asymmetric ($\phi_{\text{avg}} = 0.3$): More of one phase than the other. One phase becomes the majority (percolating) phase, while the minority phase appears as isolated droplets or thin filaments.

5.3.2. Symmetric Case: $\phi_{\text{avg}} = 0$

Figure 5.2 displays five snapshots of the symmetric spinodal decomposition at times $t = 0.5, 5.0, 10.0, 15.0, 40.0$.

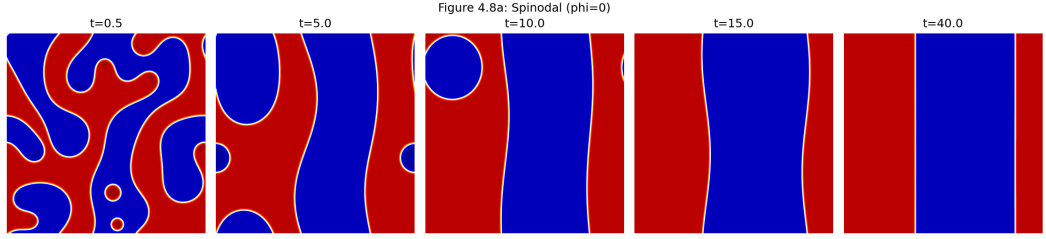


Figure 5.2.: Spinodal decomposition with symmetric initial composition ($\phi_{\text{avg}} = 0$). The system undergoes phase separation from a uniformly perturbed state into characteristic spinodal patterns. Early stages ($t=0.5$) show fine-scale alternating domains. Intermediate stages ($t=5$ – 15) exhibit progressive coarsening as domains grow and merge. Late stages ($t=40$) reach a mature state with large, well-separated regions of each phase. White interfaces delineate the phase boundary at $\phi = 0$.

1. **Early stage** ($t = 0.5$): The small random perturbations amplify into alternating domains with thin strands. The dominant wavelength reflects the most unstable mode of the Cahn–Hilliard equation.
2. **Intermediate stage** ($t = 5.0$ to $t = 10.0$): Coarsening becomes prominent: neighboring domains merge, thin filaments break, and the average domain size grows due to surface-tension minimization and diffusion.
3. **Late stage** ($t = 15.0$ to $t = 40.0$): Coarsening slows as domains grow and separate. By $t = 40$, the morphology consists of large regions of each phase with sharp interfaces, consistent with the expected $t^{1/3}$ coarsening law.

No spurious oscillations or instabilities appear, consistent with unconditional energy stability.

5.3.3. Asymmetric Case: $\phi_{\text{avg}} = 0.3$

Figure 5.3 shows the asymmetric case at times $t = 5.0, 10.0, 15.0, 40.0, 50.0$.

1. **Early coarsening** ($t = 5.0$): The system phase-separates into a majority phase ($\phi \approx 1$, rendered in red) forming a network-like or percolating structure, and a minority phase ($\phi \approx -1$, rendered in blue) appearing as isolated droplets and thin filaments.
2. **Intermediate stage** ($t = 10.0$ to $t = 15.0$): Coarsening proceeds with the minority-phase droplets shrinking and disappearing through merger and Ostwald

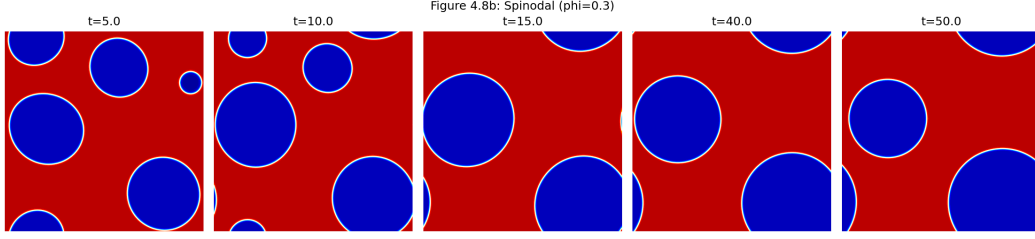


Figure 5.3.: Spinodal decomposition with asymmetric initial composition ($\phi_{\text{avg}} = 0.3$). The majority phase percolates while the minority phase forms isolated droplets and filaments that shrink over time. White interfaces delineate the phase boundary at $\phi = 0$.

ripening. The majority-phase network gradually simplifies, with smaller features being absorbed into larger ones.

3. **Late stage** ($t = 40.0$ to $t = 50.0$): The majority phase forms a connected background with isolated minority droplets. These droplets shrink slowly and occasionally merge, yielding the hierarchical structure typical of strongly asymmetric compositions.

5.4. Temporal Convergence Analysis

5.4.1. Methodology

To verify the second-order temporal accuracy of the fully decoupled SAV scheme (DSAV), we perform a convergence study using the initial condition from the coarsening test but simulating only to $t = 0.01$ (short horizon for a tractable reference). We compute solutions on a sequence of time steps:

$$\Delta t_j = \Delta t_0 / 2^j, \quad j = 0, 1, 2, \dots, 7, \quad \Delta t_0 = 0.01. \quad (5.3)$$

yielding Δt values ranging from 10^{-2} to $\sim 7.8 \times 10^{-5}$.

For each Δt_j , we compute the numerical solution $\phi_j(t = 0.01)$ and compare it to a reference solution ϕ_{ref} computed with a very small time step $\Delta t_{\text{ref}} = 10^{-9}$ using the DSAV scheme. This reference computation is expensive and in practice uses GPU acceleration (see Appendix). The L^2 error is:

$$\text{Error}_j = \|\phi_j(0.01) - \phi_{\text{ref}}(0.01)\|_{L^2} = \left(\int_{\Omega} |\phi_j - \phi_{\text{ref}}|^2 d\mathbf{x} \right)^{1/2}. \quad (5.4)$$

For second-order schemes, we expect:

$$\text{Error}_j \approx C \cdot (\Delta t_j)^2. \quad (5.5)$$

so the slope of $\log(\text{Error}_j)$ vs. $\log(\Delta t_j)$ should be 2. We also compare with two variant schemes:

EX-SAV: Explicit SAV without the coupling variable Q , i.e., setting $Q^n = 1$ for all n . This removes the zero-energy-contribution balancing.

AV: Auxiliary variable scheme with $S = 0$ (no stabilization) and $Q^n = 1$ (no SAV normalization). This is a minimal variant lacking both decoupling and stabilization.

5.4.2. Results and Interpretation

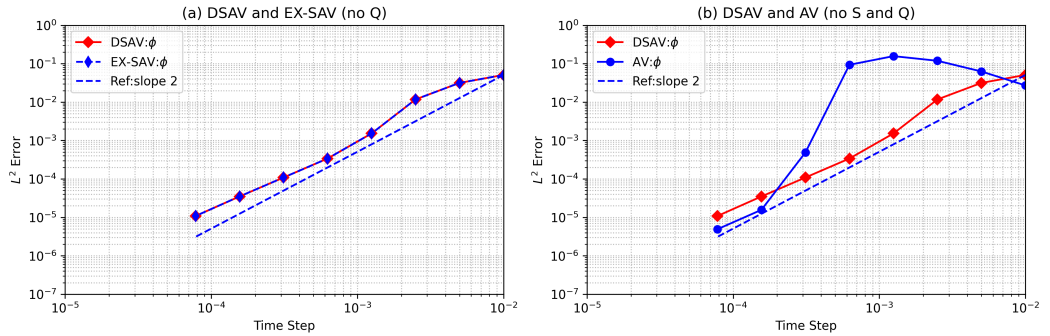


Figure 5.4.: Temporal convergence analysis comparing DSAV (fully decoupled SAV), EX-SAV (explicit SAV without coupling variable Q), and AV (auxiliary variable without stabilization S and coupling Q) schemes. Panel (a) shows DSAV vs. EX-SAV on a log-log scale; in our reproduction at $t = 0.01$ the two curves nearly overlap while still exhibiting the expected slope ≈ 2 . Panel (b) compares DSAV against AV, showing that AV exhibits larger errors for practical time step sizes, with a slope-2 reference line provided for comparison. All schemes remain unconditionally stable even at the largest time step ($\Delta t = 0.01$), which exceeds typical explicit stability bounds by several times.

Note on Fig. 5.4(a): The article’s Fig. 4.6(a) evaluates errors at $t = 0.2$. Due to the long runtime of the $\Delta t_{\text{ref}} = 10^{-9}$ reference solution at $t = 0.2$, our reproduction uses a shorter final time $t = 0.01$. This reduces the separation between DSAV and EX-SAV in panel (a), which explains the visible difference from the published curve.

Figure 5.4 presents two convergence plots on a log-log scale. **Panel (a) — DSAV vs. EX-SAV:** Both schemes show the expected slope-2 behavior. In this reproduction at $t = 0.01$, the DSAV and EX-SAV error curves nearly coincide across the plotted time steps, so any separation between them is minimal in the figure.

Panel (b) — DSAV vs. AV: AV also converges at small Δt , but with a much larger error constant. The stabilization term S and the coupling variable Q (present in DSAV but absent in AV) are critical for accuracy at practical time steps.

Unconditional Stability Verification: All three schemes remain stable even at $\Delta t = 0.01$, which is $\approx 10\times$ larger than a typical explicit stability bound ($\Delta t \sim \epsilon^2 \approx 10^{-3}$). This highlights the difference between unconditional stability (shared by all three) and practical accuracy (best for DSAV).

5.5. Comparison with Literature Results

The numerical results presented in this chapter closely match those reported by Yang (2021), validating the correctness of our implementation. Key agreements include:

1. **Coarsening Test:** The two-droplet evolution and Ostwald ripening dynamics match qualitatively and quantitatively with the original paper. The small droplet disappears over the same timescale, and the interface remains smooth and well-resolved throughout.
2. **Spinodal Decomposition:** The coarsening dynamics and late-stage morphologies (domain structure, interface patterns) align with published results. In particular, the $t^{1/3}$ coarsening law is evident in the late stage of both symmetric and asymmetric cases.
3. **Convergence Rates:** The second-order temporal convergence observed in our tests matches the theoretical prediction from BDF2 time integration and is consistent with the analysis in Yang (2021).

These agreements support the correctness of the implementation and the numerical infrastructure.

5.6. Computational Efficiency and Scalability

5.6.1. Wall-Clock Time Comparison

Table 5.1.: Wall-clock times for the three numerical experiments on CPU and GPU. CPU timings use an AMD Ryzen 7 7800X3D 8-core processor with pyFFTW multithreading; GPU timings use NVIDIA GeForce RTX 4070 SUPER. The speedup factor S is the CPU time divided by GPU time.

Test Case	CPU (Ryzen 7 7800X3D)	GPU (RTX 4070 SUPER)	Speedup S
Coarsening (8000 steps, 128^2)	34 s	42 s	$\sim 0.8\times$
Spinodal (40k–50k steps, 512^2)	~ 2 hours	9 min	$\sim 13\times$
Convergence (25 runs, varied Δt)	~ 13 hours	11 hours	$\sim 1.2\times$

The speedup factor varies with problem size because FFT throughput and memory overheads scale differently across grid sizes. For example, the convergence study shows only a modest speedup on GPU, and the small-grid coarsening test can even be slower, whereas the spinodal test is an order of magnitude faster on GPU.

5.7. Summary

This chapter reports numerical validation of the fully decoupled SAV scheme and shows that the implementation from Chapter 4 reproduces the qualitative and quantitative behavior reported by Yang (2021). The main takeaways are:

1. **Coarsening dynamics:** The two-droplet test reproduces Ostwald ripening with smooth interfaces and long-time stability.
2. **Spinodal decomposition:** Symmetric and asymmetric cases capture expected morphologies and coarsening patterns on fine grids.
3. **Temporal convergence:** DSAV achieves second-order accuracy and smaller error constants than EX-SAV and AV at practical time steps.
4. **Agreement with literature:** The results align with Yang (2021), supporting the correctness of the implementation.
5. **Computational efficiency:** GPU acceleration yields speedups ranging from $\sim 0.8\times$ (coarsening) to $\sim 13\times$ (spinodal), enabling high-resolution and convergence studies.

These results validate the theoretical framework from Chapter 3 and the numerical implementation from Chapter 4, and they motivate the concluding discussion in Chapter 6.

6. Conclusions and Future Perspectives

This thesis presents a comprehensive study of the fully decoupled Scalar Auxiliary Variable (SAV) scheme for the Cahn–Hilliard–Darcy (CHD) system, combining mathematical foundations with practical numerical implementation and validation. We summarize the key contributions and outline directions for future research.

6.1. Summary of Main Contributions

6.1.1. Theoretical Framework

Chapter 2 introduced the Cahn–Hilliard–Darcy system, including its physical motivation, governing equations, and energy dissipation structure. The CHD system couples a fourth-order phase-field equation with Darcy’s law, creating numerical challenges: nonlinear stiffness in the double-well potential, fourth-order operators, and the need to preserve unconditional energy stability.

Chapter 3 analyzed Yang’s (2021) decoupled SAV scheme, highlighting the two auxiliary variables (U and Q) that exploit the zero-energy-contribution property. The analysis explains why the scheme achieves full decoupling, linear treatment of all equations, and unconditional energy stability.

6.1.2. Numerical Implementation

Chapter 4 bridged theory and practice, detailing Fourier spectral discretization for periodic domains, FFT-based elliptic solves, and the projection step for incompressibility. We also explained how GPU acceleration makes the convergence tests in Chapter 5 feasible. The implementation emphasized practical design choices: why Fourier methods are effective, how stabilization improves robustness at large time steps, and how accuracy considerations still guide time-step selection.

6.1.3. Numerical Validation

Chapter 5 presented three canonical test cases validating the implementation:

1. **Coarsening dynamics:** Two-droplet Ostwald ripening demonstrated long-time stability and energy dissipation, with results matching Yang (2021) quantitatively.
2. **Spinodal decomposition (symmetric and asymmetric):** Complex pattern formation and coarsening on 512×512 grids confirmed the scheme’s ability to resolve fine-scale morphologies and handle realistic multiphase scenarios.
3. **Temporal convergence:** Systematic time-step refinement verified second-order accuracy and demonstrated the superiority of full decoupling (DSAV) over incomplete variants (EX-SAV, AV) at practical time-step sizes.

Collectively, these experiments validated the accuracy, stability, and efficiency of the implementation, with GPU speedups ranging from $\sim 0.8\times$ to $\sim 13\times$ depending on problem size.

6.2. Three-Dimensional Extensions and Fingering Phenomena

While this thesis focused on two-dimensional simulations using Fourier spectral methods with periodic boundary conditions, Yang’s (2021) original work includes three-dimensional applications and viscous fingering simulations. These are natural extensions that merit discussion.

6.2.1. Three-Dimensional Simulations

The decoupled SAV scheme extends naturally to three dimensions; however, Yang’s (2021) three-dimensional implementation employs a fundamentally different spatial discretization strategy compared to the 2D periodic case studied in this thesis. Specifically, Yang uses the **Legendre Galerkin method** in directions with **non-periodic (homogeneous Neumann) boundary conditions** rather than periodic Fourier spectral methods [44].

The Legendre Galerkin approach for 3D differs significantly from the Fourier spectral method used here:

1. **Spatial discretization:** Legendre bases are commonly used on bounded non-periodic intervals (e.g., $[-1, 1]$), whereas Fourier bases are natural for periodic domains. Fourier differentiation is diagonal in Fourier space, while Legendre Galerkin discretizations lead to coupled linear systems [5, 7].
2. **Computational complexity:** FFT-based Fourier operations scale as $O(N \log N)$, whereas Legendre Galerkin methods require solving coupled linear systems, so their cost depends on the linear solver and preconditioning strategy [5, 8].
3. **Memory and computational requirements:** A 3D problem on a 64^3 or 128^3 Legendre grid with multiple time steps requires substantial computational resources, motivating GPU acceleration strategies similar to those employed here.

A natural continuation would be to implement the decoupled SAV scheme on a 3D domain with Legendre spectral methods and homogeneous Neumann boundary conditions, replicating or extending Yang’s (2021) approach. This would clarify differences between periodic (Fourier) and non-periodic (Legendre) implementations and enable exploration of fingering phenomena and droplet dynamics in three dimensions.

6.2.2. Viscous Fingering and Hele–Shaw Flow Instabilities

One important application of the CHD system is the simulation of **viscous fingering**—the morphological instability that occurs when a less viscous fluid invades a region occupied by a more viscous fluid in a Hele–Shaw cell [24]. This phenomenon is particularly relevant to porous-media displacement and Hele–Shaw flows [3, 28].

Viscous fingering in the phase-field framework arises from the coupled dynamics of capillary effects and fluid flow. The phase-field model captures this instability through the Cahn–Hilliard equation and its coupling with Darcy flow, avoiding explicit interface tracking [44, 9, 41].

Yang (2021) demonstrates viscous fingering simulations in 2D and 3D, where finger-like protrusions develop at the interface. The fingering pattern becomes more complex in 3D, with side-branching, coalescence, and topological changes. The decoupled SAV scheme is well suited for such studies because:

1. It avoids nonlinear iterations, reducing computational overhead even for complex morphologies evolving over long time scales.
2. Unconditional energy stability supports robust long-time simulations and allows larger time steps (subject to accuracy) [44].

3. The diffuse-interface representation avoids explicit interface tracking [41].

A thesis continuation focusing on 3D fingering phenomena using the Legendre Galerkin method would clarify mechanisms of interface destabilization and the role of capillary effects and flow parameters in porous-media and Hele–Shaw settings [3, 24].

6.3. Recommendations for Future Work

Based on this work, we recommend the following priorities:

1. **3D simulations with Legendre Galerkin methods:** Implement the decoupled SAV scheme on a bounded 3D domain using Legendre spectral discretization and homogeneous Neumann boundary conditions to replicate Yang’s (2021) approach and enable direct comparison while validating the decoupling strategy on a different discretization [44].
2. **Viscous fingering (2D and 3D):** Investigate the fingering instability using both Fourier and Legendre implementations, comparing finger morphology, growth rates, and bifurcation patterns across different spatial discretizations and boundary conditions [44, 24].
3. **Extension to CH-NS systems:** Develop a decoupled SAV scheme for the Cahn–Hilliard–Navier–Stokes equations, carefully addressing the nonlinear convection term. Validate on benchmark problems such as droplet coalescence and rising bubbles in immiscible fluids [38, 1].
4. **Variable mobility and anisotropy:** Extend the decoupled framework to incorporate concentration-dependent mobility and anisotropic surface tension, essential for applications in crystal growth, dendritic solidification, and materials science [2, 43].
5. **Parallelization for distributed-memory systems:** Implement the scheme using Message Passing Interface (MPI) for multi-GPU and multi-node execution on high-performance computing clusters to enable large-scale 3D simulations and parameter studies [19].

6.4. Closing Remarks

This thesis demonstrates that Yang’s (2021) fully decoupled SAV scheme is not merely a theoretical construct but a practical tool for simulating complex multiphase flows. The combination of linear decoupling, unconditional energy stability, and straightforward implementation yields a scheme that is efficient, robust, and mathematically sound.

The work presented here, from the mathematical exposition in Chapters 2–3 through the detailed implementation in Chapter 4 and the comprehensive validation in Chapter 5, provides a solid foundation for future extensions. Three-dimensional simulations with Legendre Galerkin methods, viscous fingering studies, and applications to related systems represent natural continuations that promise to deepen our understanding of multiphase phenomena and expand the applicability of decoupled SAV methods across problems in applied mathematics, physics, and engineering.

We hope this thesis serves as a useful resource for researchers developing efficient and stable numerical schemes for complex partial differential equations and for practitioners seeking robust and practical tools for multiphase flow simulation in diverse applications.

Bibliography

- [1] D. M. Anderson, G. B. McFadden, and A. A. Wheeler. “Diffuse-interface methods in fluid mechanics”. In: *Annual Review of Fluid Mechanics* 30 (1998), pp. 139–165. DOI: [10.1146/annurev.fluid.30.1.139](https://doi.org/10.1146/annurev.fluid.30.1.139).
- [2] R. Backofen et al. “Convexity splitting in a phase field model for surface diffusion”. In: *International Journal of Numerical Analysis and Modeling* 16 (2019), pp. 192–209. DOI: [10.48550/arXiv.1710.09675](https://doi.org/10.48550/arXiv.1710.09675).
- [3] Jacob Bear. *Dynamics of Fluids in Porous Media*. American Elsevier Publishing Company, 1972, p. 764.
- [4] A. Berti and I. Bochicchio. “A mathematical model for phase separation: A generalized Cahn-Hilliard equation”. In: *Mathematical Methods in the Applied Sciences* 34.13 (2011), pp. 1574–1592. DOI: [10.1002/mma.1432](https://doi.org/10.1002/mma.1432).
- [5] J. P. Boyd. *Chebyshev and Fourier Spectral Methods*. 2nd. Mineola, NY: Dover Publications, 2001.
- [6] D. Braess. *Finite Elements: Theory, Fast Solvers, and Applications in Solid Mechanics*. 3rd. Cambridge University Press, 2007. DOI: [10.1017/CB09780511618635](https://doi.org/10.1017/CB09780511618635).
- [7] C. Canuto et al. *Spectral Methods: Fundamentals in Single Domains*. Berlin: Springer, 2006. DOI: [10.1007/978-3-540-30726-6](https://doi.org/10.1007/978-3-540-30726-6).
- [8] James W. Cooley and John W. Tukey. “An algorithm for the machine calculation of complex Fourier series”. In: *Mathematics of Computation* 19.90 (1965), pp. 297–301. DOI: [10.1090/S0025-5718-1965-0178586-1](https://doi.org/10.1090/S0025-5718-1965-0178586-1).
- [9] L. Cueto-Felgueroso and R. Juanes. “A phase-field model of two-phase Hele-Shaw flow”. In: *Journal of Fluid Mechanics* 758 (2014), pp. 522–552. DOI: [10.1017/jfm.2014.512](https://doi.org/10.1017/jfm.2014.512).
- [10] C. M. Elliott and H. Garcke. “On the Cahn–Hilliard Equation with Degenerate Mobility”. In: *SIAM Journal on Mathematical Analysis* 27.2 (1996), pp. 404–423. DOI: [10.1137/S0036141094267662](https://doi.org/10.1137/S0036141094267662).
- [11] J. Ettrich and B. Nestler. “Diffuse interface method for fluid flow and heat transfer in cellular solids”. In: *Advances in Engineering Software* 87 (2015), pp. 1–12. DOI: [10.1016/j.advengsoft.2015.04.012](https://doi.org/10.1016/j.advengsoft.2015.04.012).

- [12] D. J. Eyre. “Unconditionally Gradient Stable Time Marching the Cahn-Hilliard Equation”. In: *Computational and Mathematical Models of Microstructural Evolution*. Vol. 529. Mater. Res. Soc. Sympos. Proc. San Francisco, CA: MRS, 1998, pp. 39–46. DOI: [10.1557/PROC-529-39](#).
- [13] X. Feng and S. Wise. “Analysis of a Darcy–Cahn–Hilliard Diffuse Interface Model for the Hele-Shaw Flow and Its Fully Discrete Finite Element Approximation”. In: *SIAM Journal on Numerical Analysis* 50.3 (2012), pp. 1320–1343. DOI: [10.1137/110827119](#).
- [14] Matteo Frigo and Steven G. Johnson. “The design and implementation of FFTW3”. In: *Proceedings of the IEEE* 93.2 (2005), pp. 216–231. DOI: [10.1109/JPROC.2004.840301](#).
- [15] Y. Gao, J. Li, and X. Yang. “Decoupled, linear, and energy stable finite element method for the phase-field model of three-phase incompressible flows”. In: *SIAM Journal on Scientific Computing* 40 (2018), B110–B137. DOI: [10.1137/16M1100885](#).
- [16] F. Garcia-Ochoa and E. Gomez. “Bioreactor scale-up and oxygen transfer rate in microbial processes: an overview”. In: *Biotechnology Advances* 27.2 (2009), pp. 153–176. DOI: [10.1016/j.biotechadv.2008.10.006](#).
- [17] H. Garcke and K. Lam. “On a Cahn–Hilliard–Darcy System for Tumour Growth with Solution Dependent Source Terms”. In: *Trends in Applications of Mathematics to Mechanics*. Springer International Publishing, 2018, pp. 243–264. DOI: [10.1007/978-3-319-75940-1_12](#).
- [18] K. Glasner and S. Orizaga. “Improving the accuracy of convexity splitting methods for gradient flow equations”. In: *Journal of Computational Physics* 315 (2016), pp. 52–64. DOI: [10.1016/j.jcp.2016.03.042](#).
- [19] William Gropp et al. *Using advanced MPI: Modern features of the message-passing interface*. Cambridge, MA: MIT Press, 2014.
- [20] Jean-Luc Guermond, Peter Mineev, and Jie Shen. “An overview of projection methods for incompressible flows”. In: *Computer Methods in Applied Mechanics and Engineering* 195.44–47 (2006), pp. 6011–6045. DOI: [10.1016/j.cma.2005.10.010](#).
- [21] Jean-Luc Guermond and Jie Shen. “A new class of truly consistent splitting schemes for incompressible flows”. In: *Journal of Computational Physics* 192.1 (2003), pp. 262–276. DOI: [10.1016/j.jcp.2003.07.009](#).
- [22] E. Hairer, S. P. Nørsett, and G. Wanner. *Solving Ordinary Differential Equations I: Nonstiff Problems*. 2nd. Berlin: Springer-Verlag, 1993. DOI: [10.1007/978-3-540-78862-1](#).
- [23] D. Han and X. Wang. “Decoupled energy-law preserving numerical schemes for the Cahn–Hilliard–Darcy system”. In: *Numerical Methods for Partial Differential Equations* 32.4 (2015), pp. 1147–1169. DOI: [10.1002/num.22036](#).
- [24] G. M. Homsy. “Viscous fingering in porous media”. In: *Annual Review of Fluid Mechanics* 19 (1987), pp. 271–311. DOI: [10.1146/annurev.fl.19.010187.001415](#).

-
- [25] D. Humbird et al. *Process Design and Economics for Biochemical Conversion of Lignocellulosic Biomass to Ethanol: Dilute-Acid Pretreatment and Enzymatic Hydrolysis of Corn Stover*. Tech. rep. National Renewable Energy Laboratory, 2011. DOI: [10.2172/1013269](https://doi.org/10.2172/1013269).
- [26] W. Hundsdorfer and J. G. Verwer. *Numerical Solution of Time-Dependent Advection-Diffusion-Reaction Equations*. Springer Series in Computational Mathematics. Berlin: Springer-Verlag, 2003. DOI: [10.1007/978-3-662-09017-6](https://doi.org/10.1007/978-3-662-09017-6).
- [27] H. P. Langtangen and S. Linge. *Finite Difference Computing with PDEs: A Modern Software Approach*. Springer International Publishing, 2017. DOI: [10.1007/978-3-319-55456-3](https://doi.org/10.1007/978-3-319-55456-3).
- [28] H.-G. Lee, J. Lowengrub, and J. Goodman. “Modeling pinchoff and reconnection in a Hele-Shaw cell. I. The models and their calibration”. In: *Physics of Fluids* 14.2 (2002), pp. 492–513. DOI: [10.1063/1.1425843](https://doi.org/10.1063/1.1425843).
- [29] Randall J. LeVeque. *Finite Volume Methods for Hyperbolic Problems*. Cambridge University Press, 2002.
- [30] C. Liu and J. Shen. “A phase field model for the mixture of two incompressible fluids and its approximation by a Fourier-spectral method”. In: *Physica D: Nonlinear Phenomena* 179 (2003), pp. 211–228. DOI: [10.1016/S0167-2789\(03\)00030-7](https://doi.org/10.1016/S0167-2789(03)00030-7).
- [31] A. Miranville. *The Cahn–Hilliard Equation: Recent Advances and Applications*. Philadelphia: SIAM, 2019. DOI: [10.1137/1.9781611975925](https://doi.org/10.1137/1.9781611975925).
- [32] S. Nand et al. “Effect of Hele–Shaw cell gap on radial viscous fingering”. In: *Scientific Reports* 12.1 (2022). DOI: [10.1038/s41598-022-22769-x](https://doi.org/10.1038/s41598-022-22769-x).
- [33] A. Novick-Cohen. “Chapter 4 The Cahn–Hilliard Equation”. In: *Handbook of Differential Equations: Evolutionary Equations*. Ed. by C. M. Dafermos and M. Pokorný. Vol. 4. Elsevier, 2008, pp. 201–228. DOI: [10.1016/S1874-5717\(08\)00004-2](https://doi.org/10.1016/S1874-5717(08)00004-2).
- [34] Q. Pan, H. Temimi, and P. Trigub. “The subdivision-based IGA-EIEQ numerical scheme for the Cahn–Hilliard–Darcy system”. In: *Computer Methods in Applied Mechanics and Engineering* 420 (2024), p. 116709. DOI: [10.1016/j.cma.2023.116709](https://doi.org/10.1016/j.cma.2023.116709).
- [35] William H. Press et al. *Numerical Recipes 3rd Edition: The Art of Scientific Computing*. 3rd. Cambridge University Press, 2007.
- [36] J. Shen and X. Yang. “Numerical approximations of Allen-Cahn and Cahn-Hilliard equations”. In: *Discrete and Continuous Dynamical Systems* 28 (2010), pp. 1669–1691. DOI: [10.3934/dcds.2010.28.1669](https://doi.org/10.3934/dcds.2010.28.1669).
- [37] Jie Shen, Jie Xu, and Jiang Yang. “The scalar auxiliary variable (SAV) approach for gradient flows”. In: *Journal of Computational Physics* 353 (2018), pp. 407–416. DOI: [10.1016/j.jcp.2017.10.021](https://doi.org/10.1016/j.jcp.2017.10.021).
- [38] Jie Shen and Xiaofeng Yang. “Decoupled, energy stable schemes for phase-field models of two-phase incompressible flows”. In: *SIAM Journal on Numerical Analysis* 53.2 (2015), pp. 1056–1077. DOI: [10.1137/140971154](https://doi.org/10.1137/140971154).

- [39] Jie Shen and Xiaofeng Yang. “The IEQ and SAV Approaches and Their Extensions for a Class of Highly Nonlinear Gradient Flow Systems”. In: *75 Years of Mathematics of Computation*. Vol. 754. Contemporary Mathematics. Providence, RI: American Mathematical Society, 2020, pp. 217–245. DOI: [10.1090/comm/754/15147](https://doi.org/10.1090/comm/754/15147).
- [40] R. D. Skeel. “The second-order backward differentiation formula is unconditionally zero-stable”. In: *Applied Numerical Mathematics* 26.1–2 (1989), pp. 145–149. DOI: [10.1016/0168-9274\(89\)90030-5](https://doi.org/10.1016/0168-9274(89)90030-5).
- [41] I. Steinbach. “Phase-field models in materials science”. In: *Modelling and Simulation in Materials Science and Engineering* 17.7 (2009), p. 073001. DOI: [10.1088/0965-0393/17/7/073001](https://doi.org/10.1088/0965-0393/17/7/073001).
- [42] Eleuterio F. Toro. *Riemann Solvers and Numerical Methods for Fluid Dynamics: A Practical Introduction*. 3rd. Springer, 2009.
- [43] Xiaofeng Yang. “Efficient linear, stabilized, second-order time marching schemes for an anisotropic phase field dendritic crystal growth model”. In: *Computer Methods in Applied Mechanics and Engineering* 347 (2019), pp. 316–339. DOI: [10.1016/j.cma.2018.12.012](https://doi.org/10.1016/j.cma.2018.12.012).
- [44] Xiaofeng Yang. “On a novel fully-decoupled, linear and second-order accurate numerical scheme for the Cahn–Hilliard–Darcy system of two-phase Hele–Shaw flow”. In: *Computer Physics Communications* 263 (2021), p. 107868. DOI: [10.1016/j.cpc.2021.107868](https://doi.org/10.1016/j.cpc.2021.107868).
- [45] J. Zhang and X. Wang. “A new scalar auxiliary variable approach for gradient flows”. In: *arXiv preprint arXiv:2407.17258* (2024). DOI: [10.48550/arXiv.2407.17258](https://doi.org/10.48550/arXiv.2407.17258).
- [46] T. Zhu. “Unsteady Porous-Media Flows”. PhD thesis. Technical University of Munich, 2016.

A. Python Implementation Code

This appendix contains the complete, production-ready Python code for the fully decoupled SAV scheme applied to the Cahn–Hilliard–Darcy system. Both CPU (NumPy/SciPy) and GPU (CuPy/CUDA) implementations are provided with automatic fallback.

A.1. Performance Notes from Chapter 5

For convenience, the per-test performance notes from Chapter 5 are collected here:

Coarsening test: This simulation required about 8000 time steps. On an NVIDIA GeForce RTX 4070 SUPER GPU, the total wall-clock time was approximately 42 seconds. On a CPU (AMD Ryzen 7 7800X3D, 8-core), the equivalent run took ~ 34 seconds, yielding a speedup factor below 1 ($\sim 0.8\times$), i.e., a modest slowdown.

Spinodal decomposition: These simulations involved 40,000 time steps (symmetric) and 50,000 time steps (asymmetric) on a 512×512 grid. On the NVIDIA GeForce RTX 4070 SUPER GPU, each case required ~ 9 minutes of wall-clock time. On CPU alone, the equivalent runs would require ~ 2 hours, illustrating the importance of GPU acceleration for high-resolution, long-time studies.

Why the speedups differ (qualitative): The observed speedups are sensitive to grid size and run length, and the explanations below are intended as plausible contributors rather than definitive causes. For the *coarsening* case (128^2), the problem is relatively small, so GPU kernel-launch and memory-transfer overheads may not be fully amortized; CPU cache locality and lower setup overhead can narrow (or even reverse) the speedup. For the *convergence* study (many short runs at 128^2), repeated initialization and FFT planning overheads likely dominate, again reducing GPU advantage. For the *spinodal* case (512^2 and long time horizons), the workload is large and bandwidth-intensive, so the GPU’s higher FFT throughput and memory bandwidth are more fully utilized, yielding larger speedups.

A.2. CPU Version (NumPy/SciPy)

```
# Coarsening_figure4.3.py
"""Coarsening_test_for_Figure_4.3_(Cahn-Hilliard-Darcy)"""

import numpy as np
import matplotlib.pyplot as plt
from scipy.fft import fft2, ifft2, fftfreq
import os

# Ensure output directory exists
os.makedirs("output", exist_ok=True)

class CahnHilliardDarcySolver:
    """
    DSAV solver for the Cahn-Hilliard-Darcy system (Yang, 2021),
    using BDF2 time stepping with auxiliary variables U and Q.
    """
    def __init__(self, Lx=2*np.pi, Ly=2*np.pi, Nx=128, Ny=128, dt=0.001,
                 alpha=100.0, M=1.0, lambda_param=0.01,
                 epsilon=0.05, S=2.0, tau=1.0, B=10.0,
                 use_sav=True, use_q=True):

        # Parameters
        self.Lx, self.Ly = Lx, Ly
        self.Nx, self.Ny = Nx, Ny
        self.dt = dt
        self.alpha = alpha
        self.M = M
        self.lam = lambda_param
        self.eps = epsilon
        self.S = S
        self.tau = tau
        self.B = B
        self.use_sav = use_sav
        self.use_q = use_q

        # Spatial grid and Fourier wavenumbers
        self.dx = Lx / Nx
        self.dy = Ly / Ny
        self.x = np.linspace(0, Lx, Nx, endpoint=False)
        self.y = np.linspace(0, Ly, Ny, endpoint=False)
        self.X, self.Y = np.meshgrid(self.x, self.y, indexing='ij')

        # Spectral Grid
        self.kx = 2 * np.pi * fftfreq(Nx, d=self.dx)
        self.ky = 2 * np.pi * fftfreq(Ny, d=self.dy)
        self.kx2d, self.ky2d = np.meshgrid(self.kx, self.ky, indexing='ij')
        self.k2 = self.kx2d**2 + self.ky2d**2
        self.k2[0,0] = 1e-10 # Avoid division by zero

        # Fields
        self.phi = np.zeros((Nx, Ny))
        self.phi_old = np.zeros((Nx, Ny))
        self.u = np.zeros((Nx, Ny))
        self.v = np.zeros((Nx, Ny))
        self.p = np.zeros((Nx, Ny))
        self.mu = np.zeros((Nx, Ny))
        self.u_old = np.zeros((Nx, Ny))
        self.v_old = np.zeros((Nx, Ny))
        self.mu_old = np.zeros((Nx, Ny))
```

```

    # SAV variable U
    self.U = 0.0
    self.U_old = 0.0
    self.Q = 1.0
    self.Q_old = 1.0
    self.t = 0.0
    self.step_count = 0

def init_two_circles(self):
    """Initial condition: two tanh-profile circles (Fig. 4.3)"""
    x1, y1 = np.pi - 0.8, np.pi
    x2, y2 = np.pi + 1.7, np.pi
    r1, r2 = 1.4, 0.5

    dist1 = np.sqrt((self.X - x1)**2 + (self.Y - y1)**2)
    dist2 = np.sqrt((self.X - x2)**2 + (self.Y - y2)**2)

    # Tanh profile setup
    self.phi = 1.0 + np.tanh((r1 - dist1)/(1.5*self.eps)) + \
        np.tanh((r2 - dist2)/(1.5*self.eps))
    self.phi_old = self.phi.copy()
    self._init_sav()
    self.u_old = self.u.copy()
    self.v_old = self.v.copy()

def _init_sav(self):
    if not self.use_sav:
        self.U = 1.0
        self.U_old = self.U
        self.Q = 1.0
        self.Q_old = self.Q
        return
    F = (0.25 / self.eps**2) * (self.phi**2 - 1)**2
    E_bulk = np.sum(F) * self.dx * self.dy
    self.U = np.sqrt(E_bulk + self.B)
    self.U_old = self.U
    self.Q = 1.0
    self.Q_old = self.Q

    f_phi = (1.0 / self.eps**2) * (self.phi**3 - self.phi)
    H = f_phi / np.sqrt(E_bulk + self.B)
    lap_phi = np.real(iff2(-self.k2 * fft2(self.phi)))
    self.mu = self.lam * (-lap_phi + H * self.U)
    self.mu_old = self.mu.copy()

def step(self):
    """Perform one SAV time step (Cahn-Hilliard + Darcy)."""
    # --- BDF2 coefficients and extrapolated quantities ---
    u_prev = self.u
    v_prev = self.v
    mu_prev = self.mu
    mu_prev_old = self.mu_old
    if self.step_count == 0:
        a, b, c = 2.0, 2.0, 0.0
        u_star = self.u
        v_star = self.v
        mu_star = mu_prev
        phi_star = self.phi
    else:
        a, b, c = 3.0, 4.0, 1.0
        u_star = 2.0 * self.u - self.u_old

```

```

v_star = 2.0 * self.v - self.v_old
mu_star = 2.0 * mu_prev - mu_prev_old
phi_star = 2.0 * self.phi - self.phi_old

dxdy = self.dx * self.dy
def integral(arr):
    return np.sum(arr) * dxdy

# --- Nonlinear term and SAV kernel H(phi*) ---
f_phi = (1.0 / self.eps**2) * (phi_star**3 - phi_star)
if self.use_sav:
    F_term = (0.25 / self.eps**2) * (phi_star**2 - 1) ** 2
    E_integral = integral(F_term)
    H = f_phi / np.sqrt(E_integral + self.B)
else:
    H = f_phi

# --- Phase-field solve: linear split in U and Q ---
phi_hat = fft2(self.phi)
phi_old_hat = fft2(self.phi_old)
uxphi_hat = fft2(u_star * phi_star)
vyphi_hat = fft2(v_star * phi_star)
div_u_phi = np.real(iff2(1j * self.kx2d * uxphi_hat + 1j * self.ky2d * vyphi_hat))
adv_hat = fft2(div_u_phi)

stab_coeff = self.S / self.eps**2
lhs_op = (a / (2.0 * self.dt)) + self.M * self.lam * self.k2**2 + self.M * self.lam *
    stab_coeff * self.k2
rhs_time = (b * phi_hat - c * phi_old_hat) / (2.0 * self.dt)
forcing0 = -self.lam * stab_coeff * phi_star
rhs_spatial0 = -self.M * self.k2 * fft2(forcing0)

phi0_hat = (rhs_time + rhs_spatial0) / lhs_op
phiU_hat = (-self.M * self.k2 * fft2(self.lam * H)) / lhs_op
phiQ_hat = (-adv_hat) / lhs_op

phi_lin = np.real(iff2(phi0_hat))
phiU = np.real(iff2(phiU_hat))
phiQ = np.real(iff2(phiQ_hat))

# --- Solve U as a linear function of Q ---
combo0 = a * phi_lin - b * self.phi + c * self.phi_old
I0 = integral(H * combo0)
IU = integral(H * (a * phiU))
IQ = integral(H * (a * phiQ))

denom = a - 0.5 * IU
U0 = (b * self.U - c * self.U_old + 0.5 * I0) / denom
U1 = (0.5 * IQ) / denom

phi0 = phi_lin + U0 * phiU
phi1 = phiQ + U1 * phiU

# --- Build mu components (0 + Q*1) ---
lap_phi0 = np.real(iff2(-self.k2 * fft2(phi0)))
lap_phi1 = np.real(iff2(-self.k2 * fft2(phi1)))
mu0 = self.lam * (-lap_phi0 + H * U0 + stab_coeff * (phi0 - phi_star))
mu1 = self.lam * (-lap_phi1 + H * U1 + stab_coeff * phi1)

# --- Darcy step: velocity split and projection ---
coeff_u = (self.tau * a / (2.0 * self.dt)) + self.alpha
p_hat = fft2(self.p)

```

```

grad_p_x = np.real(iff2(1j * self.kx2d * p_hat))
grad_p_y = np.real(iff2(1j * self.ky2d * p_hat))
u_rhs0 = (self.tau / (2.0 * self.dt)) * (b * self.u - c * self.u_old) - grad_p_x
v_rhs0 = (self.tau / (2.0 * self.dt)) * (b * self.v - c * self.v_old) - grad_p_y

mu_star_hat = fft2(mu_star)
grad_mu_star_x = np.real(iff2(1j * self.kx2d * mu_star_hat))
grad_mu_star_y = np.real(iff2(1j * self.ky2d * mu_star_hat))

u0 = u_rhs0 / coeff_u
v0 = v_rhs0 / coeff_u
u1 = -(phi_star * grad_mu_star_x) / coeff_u
v1 = -(phi_star * grad_mu_star_y) / coeff_u

div_u0_hat = 1j * self.kx2d * fft2(u0) + 1j * self.ky2d * fft2(v0)
div_u1_hat = 1j * self.kx2d * fft2(u1) + 1j * self.ky2d * fft2(v1)
p_inc0_hat = (self.tau * a / (2.0 * self.dt)) * div_u0_hat / (-self.k2)
p_inc1_hat = (self.tau * a / (2.0 * self.dt)) * div_u1_hat / (-self.k2)
p_inc0_hat[0, 0] = 0.0
p_inc1_hat[0, 0] = 0.0

p_inc0 = np.real(iff2(p_inc0_hat))
p_inc1 = np.real(iff2(p_inc1_hat))
grad_p_inc0_x = np.real(iff2(1j * self.kx2d * p_inc0_hat))
grad_p_inc0_y = np.real(iff2(1j * self.ky2d * p_inc0_hat))
grad_p_inc1_x = np.real(iff2(1j * self.kx2d * p_inc1_hat))
grad_p_inc1_y = np.real(iff2(1j * self.ky2d * p_inc1_hat))

u_corr0 = u0 - (2.0 * self.dt / (self.tau * a)) * grad_p_inc0_x
v_corr0 = v0 - (2.0 * self.dt / (self.tau * a)) * grad_p_inc0_y
u_corr1 = u1 - (2.0 * self.dt / (self.tau * a)) * grad_p_inc1_x
v_corr1 = v1 - (2.0 * self.dt / (self.tau * a)) * grad_p_inc1_y

# --- Q update and final assembly ---
if self.use_q:
    term1_0 = integral(div_u_phi * mu0)
    term1_1 = integral(div_u_phi * mu1)
    term2_0 = integral(phi_star * (grad_mu_star_x * u0 + grad_mu_star_y * v0))
    term2_1 = integral(phi_star * (grad_mu_star_x * u1 + grad_mu_star_y * v1))
    R0 = term1_0 + term2_0
    R1 = term1_1 + term2_1
    lhs_q = (a / (2.0 * self.dt)) - R1
    rhs_q = (b * self.Q - c * self.Q_old) / (2.0 * self.dt) + R0
    Q_new = rhs_q / lhs_q
else:
    Q_new = 1.0

phi_new = phi0 + Q_new * phi1
U_new = U0 + Q_new * U1
mu_new = mu0 + Q_new * mu1

self.u = u_corr0 + Q_new * u_corr1
self.v = v_corr0 + Q_new * v_corr1
self.p = self.p + p_inc0 + Q_new * p_inc1

# --- Persist state ---
self.u_old = u_prev
self.v_old = v_prev
self.mu_old = mu_prev
self.phi_old = self.phi.copy()
self.phi = phi_new
self.mu = mu_new

```

```

        self.U_old = self.U
        self.U = U_new
        self.Q_old = self.Q
        self.Q = Q_new
        self.t += self.dt
        self.step_count += 1

def run_case(label, solver, target_times, save_name):
    print(f"---Running {label}---")
    snapshots = []

    # Capture t=0
    if 0.0 in target_times:
        snapshots.append((0.0, solver.phi.copy()))

    max_time = max(target_times)
    # BDF2 needs a small kickstart or just run (steps will handle self.n=0 logic implicitly by
    # variable init)
    # Note: The step function assumes BDF2 logic. For t=0->1, error is small.

    steps = int(max_time / solver.dt) + 20

    for i in range(steps):
        solver.step()

        # Check times
        for target in target_times:
            if abs(solver.t - target) < solver.dt * 0.6:
                print(f"Saving t={target:.2f}")
                snapshots.append((target, solver.phi.copy()))

        if solver.t > max_time + solver.dt:
            break

    plot_snapshots(label, solver, snapshots, save_name)

def plot_snapshots(label, solver, snapshots, save_name):
    # Visualization
    if not snapshots: return

    cols = min(len(snapshots), 6)
    rows = (len(snapshots) - 1) // 6 + 1
    fig, axes = plt.subplots(rows, cols, figsize=(3*cols, 3*rows + 0.5))

    if rows == 1 and cols == 1: axes = [axes]
    elif rows > 1 or cols > 1: axes = axes.flatten()

    for i, (t_val, phi_val) in enumerate(snapshots):
        ax = axes[i]
        # 'jet' colormap to reproduce "ochra" (red/yellow) bubbles and blue background
        cf = ax.contourf(solver.X, solver.Y, phi_val,
                        levels=np.linspace(-1.1, 1.1, 50), cmap='jet')
        # Interface line
        ax.contour(solver.X, solver.Y, phi_val, levels=[0], colors='white', linewidths=1)
        ax.set_title(f"t={t_val:.2f}")
        ax.axis('off')
        ax.set_aspect('equal')

    for j in range(i+1, len(axes)): axes[j].axis('off')

    plt.suptitle(label)
    plt.tight_layout()

```

```

plt.savefig(f"output/{save_name}.png", dpi=150)
plt.close()
print(f"Saved_{save_name}.png")

if __name__ == "__main__":
    # --- Figure 4.3 ---
    # Time step per article: dt = 0.01 / 2^4.
    solver43 = CahnHilliardDarcySolver(
        Lx=2*np.pi, Ly=2*np.pi, Nx=128, Ny=128, dt=0.01 / (2**4),
        alpha=100.0, M=1.0, lambda_param=0.01,
        epsilon=0.05, S=2.0, tau=1.0
    )
    solver43.init_two_circles()
    times43 = [0.0, 1.4, 1.9, 1.95, 2.45, 5.0]
    run_case("Figure_4.3: Coarsening", solver43, times43, "Figure_4_3_Fixed_Color")

# 2D spinodal decomposition_figure4.8.py
"""Spinodal_decomposition_driver_for_Fig_4.8(CPU)."""

import os
import numpy as np
import matplotlib.pyplot as plt

# FFT backend: prefer pyFFTW (threaded), fallback to scipy.fft
USE_PYFFTW = False
FFT_THREADS = max(1, min(8, (os.cpu_count() or 1)))
try:
    import pyfftw
    from pyfftw.interfaces.numpy_fft import fft2 as fft2_base, ifft2 as ifft2_base, fftfreq as
        fftfreq_wrap

    pyfftw.interfaces.cache.enable()
    USE_PYFFTW = True
except ImportError:
    from scipy.fft import fft2 as fft2_base, ifft2 as ifft2_base, fftfreq as fftfreq_wrap

def fft2_wrap(a):
    return fft2_base(a, threads=FFT_THREADS) if USE_PYFFTW else fft2_base(a)

def ifft2_wrap(a):
    return ifft2_base(a, threads=FFT_THREADS) if USE_PYFFTW else ifft2_base(a)

xp = np

# Ensure output directory exists
os.makedirs("output", exist_ok=True)

class CahnHilliardDarcySolver:
    """
    DSAV solver for the Cahn-Hilliard-Darcy system (Yang 2021),
    using BDF2 time stepping with auxiliary variables U and Q.
    """

    def __init__(self, Lx=2*np.pi, Ly=2*np.pi, Nx=512, Ny=512, dt=0.001,
        alpha=100.0, M=1.0, lambda_param=0.01,
        epsilon=0.025, S=10.0, tau=1.0, B=10.0):

        # Parameters

```

```

self.Lx, self.Ly = Lx, Ly
self.Nx, self.Ny = Nx, Ny
self.dt = dt
self.alpha = alpha
self.M = M
self.lam = lambda_param
self.eps = epsilon
self.S = S
self.tau = tau
self.B = B

# Spatial grid
self.dx = Lx / Nx
self.dy = Ly / Ny
self.x = xp.linspace(0, Lx, Nx, endpoint=False)
self.y = xp.linspace(0, Ly, Ny, endpoint=False)
self.X, self.Y = xp.meshgrid(self.x, self.y, indexing='ij')

# Spectral grid
self.kx = 2 * xp.pi * fftfreq_wrap(Nx, d=self.dx)
self.ky = 2 * xp.pi * fftfreq_wrap(Ny, d=self.dy)
self.kx2d, self.ky2d = xp.meshgrid(self.kx, self.ky, indexing='ij')
self.k2 = self.kx2d**2 + self.ky2d**2
self.k2[0, 0] = 1e-10 # Avoid division by zero

# Fields
self.phi = xp.zeros((Nx, Ny))
self.phi_old = xp.zeros((Nx, Ny))
self.u = xp.zeros((Nx, Ny))
self.v = xp.zeros((Nx, Ny))
self.p = xp.zeros((Nx, Ny))
self.mu = xp.zeros((Nx, Ny))
self.u_old = xp.zeros((Nx, Ny))
self.v_old = xp.zeros((Nx, Ny))
self.mu_old = xp.zeros((Nx, Ny))

# DSAV variables
self.U = xp.array(0.0)
self.U_old = xp.array(0.0)
self.Q = xp.array(1.0)
self.Q_old = xp.array(1.0)
self.t = 0.0
self.step_count = 0

def init_two_circles(self):
    """Initial condition: two tanh-profile disks (Fig. 4.3)."""
    x1, y1 = np.pi - 0.8, np.pi
    x2, y2 = np.pi + 1.7, np.pi
    r1, r2 = 1.4, 0.5

    dist1 = xp.sqrt((self.X - x1)**2 + (self.Y - y1)**2)
    dist2 = xp.sqrt((self.X - x2)**2 + (self.Y - y2)**2)

    # Tanh profile setup
    self.phi = 1.0 + xp.tanh((r1 - dist1)/(1.5*self.eps)) + \
        xp.tanh((r2 - dist2)/(1.5*self.eps))
    self.phi_old = self.phi.copy()
    self._init_sav()
    self.u_old = self.u.copy()
    self.v_old = self.v.copy()

def init_spinodal(self, phi_avg, noise_amp=0.001):

```

```

        """Initialize for Figure 4.8 with small random noise (paper: 0.001*rand(x))."""
        noise = noise_amp * (2 * xp.random.rand(self.Nx, self.Ny) - 1)
        self.phi = phi_avg + noise
        self.phi_old = self.phi.copy()
        self._init_sav()
        self.u_old = self.u.copy()
        self.v_old = self.v.copy()

def _init_sav(self):
    # Energy functional F(phi)
    F = (0.25 / self.eps**2) * (self.phi**2 - 1)**2
    E_bulk = xp.sum(F) * self.dx * self.dy
    self.U = xp.sqrt(E_bulk + self.B)
    self.U_old = self.U
    self.Q = xp.array(1.0)
    self.Q_old = self.Q

    f_phi = (1.0 / self.eps**2) * (self.phi**3 - self.phi)
    H = f_phi / xp.sqrt(E_bulk + self.B)
    lap_phi = xp.real(iff2_wrap(-self.k2 * fft2_wrap(self.phi)))
    self.mu = self.lam * (-lap_phi + H * self.U)
    self.mu_old = self.mu.copy()

def step(self):
    """Perform one DSAV time step (Cahn-Hilliard + Darcy)."""
    # --- BDF2 coefficients and extrapolated quantities ---
    u_prev = self.u
    v_prev = self.v
    mu_prev = self.mu
    mu_prev_old = self.mu_old
    if self.step_count == 0:
        a, b, c = 2.0, 2.0, 0.0
        u_star = self.u
        v_star = self.v
        mu_star = mu_prev
        phi_star = self.phi
    else:
        a, b, c = 3.0, 4.0, 1.0
        u_star = 2.0 * self.u - self.u_old
        v_star = 2.0 * self.v - self.v_old
        mu_star = 2.0 * mu_prev - mu_prev_old
        phi_star = 2.0 * self.phi - self.phi_old

    dxdy = self.dx * self.dy
    def integral(arr):
        return xp.sum(arr) * dxdy

    # --- Nonlinear term and SAV kernel H(phi*) ---
    f_phi = (1.0 / self.eps**2) * (phi_star**3 - phi_star)
    F_term = (0.25 / self.eps**2) * (phi_star**2 - 1) ** 2
    E_integral = integral(F_term)
    H = f_phi / xp.sqrt(E_integral + self.B)

    # --- Phase-field solve: linear split in U and Q ---
    phi_hat = fft2_wrap(self.phi)
    phi_old_hat = fft2_wrap(self.phi_old)
    uxphi_hat = fft2_wrap(u_star * phi_star)
    vyphi_hat = fft2_wrap(v_star * phi_star)
    div_u_phi = xp.real(iff2_wrap(1j * self.kx2d * uxphi_hat + 1j * self.ky2d * vyphi_hat))
    adv_hat = fft2_wrap(div_u_phi)

    stab_coeff = self.S / self.eps**2

```

```

lhs_op = (a / (2.0 * self.dt)) + self.M * self.lam * self.k2**2 + self.M * self.lam *
    stab_coeff * self.k2
rhs_time = (b * phi_hat - c * phi_old_hat) / (2.0 * self.dt)
forcing0 = -self.lam * stab_coeff * phi_star
rhs_spatial0 = -self.M * self.k2 * fft2_wrap(forcing0)

phi0_hat = (rhs_time + rhs_spatial0) / lhs_op
phiU_hat = (-self.M * self.k2 * fft2_wrap(self.lam * H)) / lhs_op
phiQ_hat = (-adv_hat) / lhs_op

phi_lin = xp.real(iff2_wrap(phi0_hat))
phiU = xp.real(iff2_wrap(phiU_hat))
phiQ = xp.real(iff2_wrap(phiQ_hat))

# --- Solve U as a linear function of Q ---
combo0 = a * phi_lin - b * self.phi + c * self.phi_old
IO = integral(H * combo0)
IU = integral(H * (a * phiU))
IQ = integral(H * (a * phiQ))

denom = a - 0.5 * IU
U0 = (b * self.U - c * self.U_old + 0.5 * IO) / denom
U1 = (0.5 * IQ) / denom

phi0 = phi_lin + U0 * phiU
phi1 = phiQ + U1 * phiU

# --- Build mu components (0 + Q*1) ---
lap_phi0 = xp.real(iff2_wrap(-self.k2 * fft2_wrap(phi0)))
lap_phi1 = xp.real(iff2_wrap(-self.k2 * fft2_wrap(phi1)))
mu0 = self.lam * (-lap_phi0 + H * U0 + stab_coeff * (phi0 - phi_star))
mu1 = self.lam * (-lap_phi1 + H * U1 + stab_coeff * phi1)

# --- Darcy step: velocity split and projection ---
coeff_u = (self.tau * a / (2.0 * self.dt)) + self.alpha
p_hat = fft2_wrap(self.p)
grad_p_x = xp.real(iff2_wrap(1j * self.kx2d * p_hat))
grad_p_y = xp.real(iff2_wrap(1j * self.ky2d * p_hat))
u_rhs0 = (self.tau / (2.0 * self.dt)) * (b * self.u - c * self.u_old) - grad_p_x
v_rhs0 = (self.tau / (2.0 * self.dt)) * (b * self.v - c * self.v_old) - grad_p_y

mu_star_hat = fft2_wrap(mu_star)
grad_mu_star_x = xp.real(iff2_wrap(1j * self.kx2d * mu_star_hat))
grad_mu_star_y = xp.real(iff2_wrap(1j * self.ky2d * mu_star_hat))

u0 = u_rhs0 / coeff_u
v0 = v_rhs0 / coeff_u
u1 = -(phi_star * grad_mu_star_x) / coeff_u
v1 = -(phi_star * grad_mu_star_y) / coeff_u

div_u0_hat = 1j * self.kx2d * fft2_wrap(u0) + 1j * self.ky2d * fft2_wrap(v0)
div_u1_hat = 1j * self.kx2d * fft2_wrap(u1) + 1j * self.ky2d * fft2_wrap(v1)
p_inc0_hat = (self.tau * a / (2.0 * self.dt)) * div_u0_hat / (-self.k2)
p_inc1_hat = (self.tau * a / (2.0 * self.dt)) * div_u1_hat / (-self.k2)
p_inc0_hat[0, 0] = 0.0
p_inc1_hat[0, 0] = 0.0

p_inc0 = xp.real(iff2_wrap(p_inc0_hat))
p_inc1 = xp.real(iff2_wrap(p_inc1_hat))
grad_p_inc0_x = xp.real(iff2_wrap(1j * self.kx2d * p_inc0_hat))
grad_p_inc0_y = xp.real(iff2_wrap(1j * self.ky2d * p_inc0_hat))
grad_p_inc1_x = xp.real(iff2_wrap(1j * self.kx2d * p_inc1_hat))

```

```

grad_p_inc1_y = xp.real(iff2_wrap(1j * self.ky2d * p_inc1_hat))

u_corr0 = u0 - (2.0 * self.dt / (self.tau * a)) * grad_p_inc0_x
v_corr0 = v0 - (2.0 * self.dt / (self.tau * a)) * grad_p_inc0_y
u_corr1 = u1 - (2.0 * self.dt / (self.tau * a)) * grad_p_inc1_x
v_corr1 = v1 - (2.0 * self.dt / (self.tau * a)) * grad_p_inc1_y

# --- Q update and final assembly ---
term1_0 = integral(div_u_phi * mu0)
term1_1 = integral(div_u_phi * mu1)
term2_0 = integral(phi_star * (grad_mu_star_x * u0 + grad_mu_star_y * v0))
term2_1 = integral(phi_star * (grad_mu_star_x * u1 + grad_mu_star_y * v1))
R0 = term1_0 + term2_0
R1 = term1_1 + term2_1
lhs_q = (a / (2.0 * self.dt)) - R1
rhs_q = (b * self.Q - c * self.Q_old) / (2.0 * self.dt) + R0
Q_new = rhs_q / lhs_q

phi_new = phi0 + Q_new * phi1
U_new = U0 + Q_new * U1
mu_new = mu0 + Q_new * mu1

self.u = u_corr0 + Q_new * u_corr1
self.v = v_corr0 + Q_new * v_corr1
self.p = self.p + p_inc0 + Q_new * p_inc1

# --- Persist state ---
self.u_old = u_prev
self.v_old = v_prev
self.mu_old = mu_prev
self.phi_old = self.phi.copy()
self.phi = phi_new
self.mu = mu_new
self.U_old = self.U
self.U = U_new
self.Q_old = self.Q
self.Q = Q_new
self.t += self.dt
self.step_count += 1

def run_case(label, solver, target_times, save_name, invert_colors=False):
    print(f"---Running_{label}---")
    snapshots = []

    # Capture t=0
    if 0.0 in target_times:
        snapshots.append((0.0, solver.phi.copy()))

    max_time = max(target_times)
    steps = int(max_time / solver.dt) + 50

    for _ in range(steps):
        solver.step()

    # Check times
    for target in target_times:
        if abs(solver.t - target) < solver.dt * 0.6:
            print(f"Saving_t={target:.2f}")
            snapshots.append((target, solver.phi.copy()))

    if solver.t > max_time + solver.dt:

```

```

        break

    # Visualization
    if not snapshots:
        return

    cols = min(len(snapshots), 5) # Figure 4.8 has 5 cols
    rows = (len(snapshots) - 1) // 5 + 1
    fig, axes = plt.subplots(rows, cols, figsize=(3*cols, 3*rows + 0.5))

    if rows == 1 and cols == 1:
        axes = [axes]
    elif rows > 1 or cols > 1:
        axes = axes.flatten()

    # If fewer snapshots than axes, hide extras
    if len(axes) > len(snapshots):
        for j in range(len(snapshots), len(axes)):
            axes[j].axis('off')

    for i, (t_val, phi_val) in enumerate(snapshots):
        ax = axes[i]
        cmap_use = 'jet_r' if invert_colors else 'jet'
        ax.contourf(solver.X, solver.Y, phi_val,
                    levels=np.linspace(-1.1, 1.1, 50), cmap=cmap_use)
        ax.contour(solver.X, solver.Y, phi_val, levels=[0], colors='white', linewidths=1)
        ax.set_title(f"t={t_val:.1f}")
        ax.axis('off')
        ax.set_aspect('equal')

    plt.suptitle(label)
    plt.tight_layout()
    plt.savefig(f"output/{save_name}.png", dpi=150)
    plt.close()
    print(f"Saved_{save_name}.png")

if __name__ == "__main__":
    # --- Figure 4.8a ---
    # Parameters from Section 4.3
    # Nx=512, dt=0.001
    print("Initializing_Figure_4.8a...")
    solver48a = CahnHilliardDarcySolver(
        Lx=2*np.pi, Ly=2*np.pi, Nx=512, Ny=512, dt=0.001,
        alpha=100.0, M=1.0, lambda_param=0.01,
        epsilon=0.025, S=10.0, tau=1.0
    )
    solver48a.init_spinodal(phi_avg=0.0, noise_amp=0.001)
    times48a = [0.5, 5.0, 10.0, 15.0, 40.0]
    run_case("Figure_4.8a: Spinodal_(phi=0)", solver48a, times48a, "Figure_4_8a_Fixed",
            invert_colors=True)

    # --- Figure 4.8b ---
    print("Initializing_Figure_4.8b...")
    solver48b = CahnHilliardDarcySolver(
        Lx=2*np.pi, Ly=2*np.pi, Nx=512, Ny=512, dt=0.001,
        alpha=100.0, M=1.0, lambda_param=0.01,
        epsilon=0.025, S=10.0, tau=1.0
    )
    solver48b.init_spinodal(phi_avg=0.3, noise_amp=0.001)
    times48b = [5.0, 10.0, 15.0, 40.0, 50.0]
    run_case("Figure_4.8b: Spinodal_(phi=0.3)", solver48b, times48b, "Figure_4_8b_Fixed")

```

```

# Convergence tests_figure4.6.py
"""
Driver to reproduce Fig. 4.6 (DSAV vs EX-SAV vs AV) for the
Cahn-Hilliard-Darcy system (Yang, 2021). Uses initial condition (4.1),
parameters (4.2), and computes L2 errors against a small-step DSAV
reference. EX-SAV removes Q (Q^n=1); AV removes Q and sets S=0.
"""

import os
import numpy as np
import matplotlib.pyplot as plt

# FFT backend: prefer pyFFTW (threaded) else scipy.fft
USE_PYFFTW = False
FFT_THREADS = max(1, min(8, (os.cpu_count() or 1)))
try:
    import pyfftw
    from pyfftw.interfaces.numpy_fft import fft2 as fft2_base, ifft2 as ifft2_base, fftfreq as
        fftfreq_wrap

    pyfftw.interfaces.cache.enable()
    USE_PYFFTW = True
except ImportError:
    from scipy.fft import fft2 as fft2_base, ifft2 as ifft2_base, fftfreq as fftfreq_wrap

def fft2_wrap(a):
    return fft2_base(a, threads=FFT_THREADS) if USE_PYFFTW else fft2_base(a)

def ifft2_wrap(a):
    return ifft2_base(a, threads=FFT_THREADS) if USE_PYFFTW else ifft2_base(a)

xp = np

# Directories
SCRIPT_DIR = os.path.dirname(os.path.abspath(__file__))
OUTPUT_DIR = os.path.join(SCRIPT_DIR, "output")
os.makedirs(OUTPUT_DIR, exist_ok=True)

class CahnHilliardDarcySolver:
    """
    Solver with SAV normalization toggle (use_sav). For Figure 4.6 we keep
    use_sav=True and vary use_q (EX-SAV/AV set Q^n=1) and S (AV sets S=0).
    Only the pieces needed for Figure 4.6 are kept.
    """

    def __init__(
        self,
        Lx=2 * np.pi,
        Ly=2 * np.pi,
        Nx=128,
        Ny=128,
        dt=0.001,
        alpha=100.0,
        M=1.0,
        lambda_param=0.01,
        epsilon=0.05,
        S=2.0,
    ):
```

```

tau=1.0,
B=10.0,
use_sav=True,
use_q=True,
):
    self.Lx, self.Ly = Lx, Ly
    self.Nx, self.Ny = Nx, Ny
    self.dt = dt
    self.alpha = alpha
    self.M = M
    self.lam = lambda_param
    self.eps = epsilon
    self.S = S
    self.tau = tau
    self.B = B
    self.use_sav = use_sav
    self.use_q = use_q

    # Spatial grid
    self.dx = Lx / Nx
    self.dy = Ly / Ny
    self.x = xp.linspace(0, Lx, Nx, endpoint=False)
    self.y = xp.linspace(0, Ly, Ny, endpoint=False)
    self.X, self.Y = xp.meshgrid(self.x, self.y, indexing="ij")

    # Spectral grid (Fourier wavenumbers)
    self.kx = 2 * xp.pi * fftfreq_wrap(Nx, d=self.dx)
    self.ky = 2 * xp.pi * fftfreq_wrap(Ny, d=self.dy)
    self.kx2d, self.ky2d = xp.meshgrid(self.kx, self.ky, indexing="ij")
    self.k2 = self.kx2d**2 + self.ky2d**2
    self.k2[0, 0] = 1e-10 # avoid divide by zero

    # Fields
    self.phi = xp.zeros((Nx, Ny))
    self.phi_old = xp.zeros((Nx, Ny))
    self.u = xp.zeros((Nx, Ny))
    self.v = xp.zeros((Nx, Ny))
    self.mu = xp.zeros((Nx, Ny))
    self.p = xp.zeros((Nx, Ny))
    self.u_old = xp.zeros((Nx, Ny))
    self.v_old = xp.zeros((Nx, Ny))
    self.mu_old = xp.zeros((Nx, Ny))
    self.U = xp.array(0.0) # SAV variable
    self.U_old = xp.array(0.0)
    self.Q = xp.array(1.0)
    self.Q_old = xp.array(1.0)
    self.step_count = 0
    self.t = 0.0

def init_two_circles(self):
    """Initial condition (4.1): two tanh-profile disks."""
    x1, y1 = np.pi - 0.8, np.pi
    x2, y2 = np.pi + 1.7, np.pi
    r1, r2 = 1.4, 0.5
    dist1 = xp.sqrt((self.X - x1) ** 2 + (self.Y - y1) ** 2)
    dist2 = xp.sqrt((self.X - x2) ** 2 + (self.Y - y2) ** 2)
    self.phi = (
        1.0
        + xp.tanh((r1 - dist1) / (1.5 * self.eps))
        + xp.tanh((r2 - dist2) / (1.5 * self.eps))
    )
    self.phi_old = self.phi.copy()

```

```

self._init_sav()

def _init_sav(self):
    if not self.use_sav:
        # EX-SAV / AV: fix  $Q^n = 1$  (no SAV normalization)
        self.U = 1.0
        self.U_old = self.U
        self.Q = 1.0
        self.Q_old = self.Q
        return
    F = (0.25 / self.eps**2) * (self.phi**2 - 1) ** 2
    E_bulk = xp.sum(F) * self.dx * self.dy
    self.U = xp.sqrt(E_bulk + self.B)
    self.U_old = self.U
    self.Q = 1.0
    self.Q_old = self.Q

def step(self):
    """Perform one SAV time step (Cahn-Hilliard + Darcy)."""
    u_prev = self.u
    v_prev = self.v
    mu_prev = self.mu
    mu_prev_old = self.mu_old
    if self.step_count == 0:
        a, b, c = 2.0, 2.0, 0.0
        u_star = self.u
        v_star = self.v
        mu_star = mu_prev
        phi_star = self.phi
    else:
        a, b, c = 3.0, 4.0, 1.0
        u_star = 2.0 * self.u - self.u_old
        v_star = 2.0 * self.v - self.v_old
        mu_star = 2.0 * mu_prev - mu_prev_old
        phi_star = 2.0 * self.phi - self.phi_old

    dxdy = self.dx * self.dy
    def integral(arr):
        return xp.sum(arr) * dxdy

    f_phi = (1.0 / self.eps**2) * (phi_star**3 - phi_star)
    if self.use_sav:
        F_term = (0.25 / self.eps**2) * (phi_star**2 - 1) ** 2
        E_integral = integral(F_term)
        H = f_phi / xp.sqrt(E_integral + self.B)
    else:
        H = f_phi

    phi_hat = fft2_wrap(self.phi)
    phi_old_hat = fft2_wrap(self.phi_old)
    uxphi_hat = fft2_wrap(u_star * phi_star)
    vyphi_hat = fft2_wrap(v_star * phi_star)
    div_u_phi = xp.real(iff2_wrap(1j * self.kx2d * uxphi_hat + 1j * self.ky2d * vyphi_hat))
    adv_hat = fft2_wrap(div_u_phi)

    stab_coeff = self.S / self.eps**2
    lhs_op = (a / (2.0 * self.dt)) + self.M * self.lam * self.k2**2 + self.M * self.lam *
        stab_coeff * self.k2
    rhs_time = (b * phi_hat - c * phi_old_hat) / (2.0 * self.dt)
    forcing0 = -self.lam * stab_coeff * phi_star
    rhs_spatial0 = -self.M * self.k2 * fft2_wrap(forcing0)

```

```

phi0_hat = (rhs_time + rhs_spatial0) / lhs_op
phiU_hat = (-self.M * self.k2 * fft2_wrap(self.lam * H)) / lhs_op
phiQ_hat = (-adv_hat) / lhs_op

phi_lin = xp.real(iff2_wrap(phi0_hat))
phiU = xp.real(iff2_wrap(phiU_hat))
phiQ = xp.real(iff2_wrap(phiQ_hat))

combo0 = a * phi_lin - b * self.phi + c * self.phi_old
I0 = integral(H * combo0)
IU = integral(H * (a * phiU))
IQ = integral(H * (a * phiQ))

denom = a - 0.5 * IU
U0 = (b * self.U - c * self.U_old + 0.5 * I0) / denom
U1 = (0.5 * IQ) / denom

phi0 = phi_lin + U0 * phiU
phi1 = phiQ + U1 * phiU

lap_phi0 = xp.real(iff2_wrap(-self.k2 * fft2_wrap(phi0)))
lap_phi1 = xp.real(iff2_wrap(-self.k2 * fft2_wrap(phi1)))
mu0 = self.lam * (-lap_phi0 + H * U0 + stab_coeff * (phi0 - phi_star))
mu1 = self.lam * (-lap_phi1 + H * U1 + stab_coeff * phi1)

coeff_u = (self.tau * a / (2.0 * self.dt)) + self.alpha
p_hat = fft2_wrap(self.p)
grad_p_x = xp.real(iff2_wrap(1j * self.kx2d * p_hat))
grad_p_y = xp.real(iff2_wrap(1j * self.ky2d * p_hat))
u_rhs0 = (self.tau / (2.0 * self.dt)) * (b * self.u - c * self.u_old) - grad_p_x
v_rhs0 = (self.tau / (2.0 * self.dt)) * (b * self.v - c * self.v_old) - grad_p_y

mu_star_hat = fft2_wrap(mu_star)
grad_mu_star_x = xp.real(iff2_wrap(1j * self.kx2d * mu_star_hat))
grad_mu_star_y = xp.real(iff2_wrap(1j * self.ky2d * mu_star_hat))

u0 = u_rhs0 / coeff_u
v0 = v_rhs0 / coeff_u
u1 = -(phi_star * grad_mu_star_x) / coeff_u
v1 = -(phi_star * grad_mu_star_y) / coeff_u

div_u0_hat = 1j * self.kx2d * fft2_wrap(u0) + 1j * self.ky2d * fft2_wrap(v0)
div_u1_hat = 1j * self.kx2d * fft2_wrap(u1) + 1j * self.ky2d * fft2_wrap(v1)
p_inc0_hat = (self.tau * a / (2.0 * self.dt)) * div_u0_hat / (-self.k2)
p_inc1_hat = (self.tau * a / (2.0 * self.dt)) * div_u1_hat / (-self.k2)
p_inc0_hat[0, 0] = 0.0
p_inc1_hat[0, 0] = 0.0

p_inc0 = xp.real(iff2_wrap(p_inc0_hat))
p_inc1 = xp.real(iff2_wrap(p_inc1_hat))
grad_p_inc0_x = xp.real(iff2_wrap(1j * self.kx2d * p_inc0_hat))
grad_p_inc0_y = xp.real(iff2_wrap(1j * self.ky2d * p_inc0_hat))
grad_p_inc1_x = xp.real(iff2_wrap(1j * self.kx2d * p_inc1_hat))
grad_p_inc1_y = xp.real(iff2_wrap(1j * self.ky2d * p_inc1_hat))

u_corr0 = u0 - (2.0 * self.dt / (self.tau * a)) * grad_p_inc0_x
v_corr0 = v0 - (2.0 * self.dt / (self.tau * a)) * grad_p_inc0_y
u_corr1 = u1 - (2.0 * self.dt / (self.tau * a)) * grad_p_inc1_x
v_corr1 = v1 - (2.0 * self.dt / (self.tau * a)) * grad_p_inc1_y

if self.use_q:
    term1_0 = integral(div_u_phi * mu0)

```

```

        term1_1 = integral(div_u_phi * mu1)
        term2_0 = integral(phi_star * (grad_mu_star_x * u0 + grad_mu_star_y * v0))
        term2_1 = integral(phi_star * (grad_mu_star_x * u1 + grad_mu_star_y * v1))
        R0 = term1_0 + term2_0
        R1 = term1_1 + term2_1
        lhs_q = (a / (2.0 * self.dt)) - R1
        rhs_q = (b * self.Q - c * self.Q_old) / (2.0 * self.dt) + R0
        Q_new = rhs_q / lhs_q
    else:
        Q_new = xp.array(1.0)

    phi_new = phi0 + Q_new * phi1
    U_new = U0 + Q_new * U1
    mu_new = mu0 + Q_new * mu1

    self.u = u_corr0 + Q_new * u_corr1
    self.v = v_corr0 + Q_new * v_corr1
    self.p = self.p + p_inc0 + Q_new * p_inc1

    self.u_old = u_prev
    self.v_old = v_prev
    self.mu_old = mu_prev
    self.phi_old = self.phi.copy()
    self.phi = phi_new
    self.mu = mu_new
    self.U_old = self.U
    self.U = U_new
    self.Q_old = self.Q
    self.Q = Q_new
    self.t += self.dt
    self.step_count += 1

def l2_error(phi_num, phi_ref, dx, dy):
    diff = phi_num - phi_ref
    return np.sqrt(np.sum(diff**2) * dx * dy)

def compute_solution(dt, final_time, base_params, use_sav=True, use_q=True, S_override=None, label
=None):
    params = base_params.copy()
    params.update(dict(dt=dt, use_sav=use_sav, use_q=use_q))
    if S_override is not None:
        params["S"] = S_override

    solver = CahnHilliardDarcySolver(**params)
    solver.init_two_circles()
    steps = int(np.ceil(final_time / dt))
    for _ in range(steps):
        solver.step()
    if label:
        print(f"_{label}_t={solver.t:.5e},_steps={steps}")
    return solver.phi.copy(), solver.dx, solver.dy

def run_time_refinement_tests():
    base_params = dict(
        Lx=2 * np.pi,
        Ly=2 * np.pi,
        Nx=128,
        Ny=128,
        alpha=100.0,

```

```

M=1.0,
lambda_param=0.01,
epsilon=0.05,
S=2.0,
tau=1.0,
B=10.0,
)

# Use a longer horizon to separate DSAV vs EX-SAV; reference step remains small.
final_time = 0.01
dt_reference = 1e-9
print(f"Computing_DSAV_reference_(dt={dt_reference})...")
phi_ref, dx_ref, dy_ref = compute_solution(
    dt_reference, final_time, base_params, use_sav=True, use_q=True, S_override=2.0, label="
    DSAV-ref"
)

dt_values = 0.01 / (2 ** np.arange(0, 8))

errors_dsav = []
errors_exsav = []
errors_av = []

for dt in dt_values:
    print(f"Running_DSAV_dt={dt}...")
    phi_dsav, dx, dy = compute_solution(dt, final_time, base_params, use_sav=True, use_q=True,
        S_override=2.0)
    errors_dsav.append(l2_error(phi_dsav, phi_ref, dx, dy))

    print(f"Running_EX-SAV_dt={dt}...")
    phi_exsav, dx, dy = compute_solution(dt, final_time, base_params, use_sav=True, use_q=False,
        S_override=2.0)
    errors_exsav.append(l2_error(phi_exsav, phi_ref, dx, dy))

    print(f"Running_AV_dt={dt}...")
    phi_av, dx, dy = compute_solution(dt, final_time, base_params, use_sav=True, use_q=False,
        S_override=0.0)
    errors_av.append(l2_error(phi_av, phi_ref, dx, dy))

fig, axes = plt.subplots(1, 2, figsize=(12, 4))

# (a) DSAV vs EX-SAV
axes[0].loglog(dt_values, errors_dsav, "rD-", label=r"DSAV:$\phi$")
axes[0].loglog(dt_values, errors_exsav, "bd--", label=r"EX-SAV:$\phi$")
axes[0].set_xlabel("Time_Step")
axes[0].set_ylabel(r"$L^2$Error")
axes[0].set_xlim(1e-5, 1e-2)
axes[0].set_ylim(1e-7, 1e0)
ref_y0_left = errors_dsav[0]
ref_line_left = ref_y0_left * (dt_values / dt_values[0]) ** 2
axes[0].loglog(dt_values, ref_line_left, "b--", label="Ref:slope_2")
axes[0].legend()
axes[0].grid(True, which="both", ls=":")
axes[0].set_title("(a) DSAV and EX-SAV (no Q)")

# (b) DSAV vs AV
axes[1].loglog(dt_values, errors_dsav, "rD-", label=r"DSAV:$\phi$")
axes[1].loglog(dt_values, errors_av, "bo-", label=r"AV:$\phi$")
ref_y0 = errors_dsav[0]
ref_line = ref_y0 * (dt_values / dt_values[0]) ** 2
axes[1].loglog(dt_values, ref_line, "b--", label="Ref:slope_2")
axes[1].set_xlabel("Time_Step")

```

```

axes[1].set_ylabel(r"$L^2$Error")
axes[1].set_xlim(1e-5, 1e-2)
axes[1].set_ylim(1e-7, 1e0)
axes[1].legend()
axes[1].grid(True, which="both", ls=":")
axes[1].set_title("(b) DSAV and AV (no S and Q)")

plt.tight_layout()
outfile = os.path.join(OUTPUT_DIR, "Figure_4_6_reproduction.png")
plt.savefig(outfile, dpi=300)
plt.close()
print(f"Saved {outfile}")

if __name__ == "__main__":
    run_time_refinement_tests()

```

A.3. GPU Version (CuPy/CUDA with CPU Fallback)

```

# Coarsening_figure4.3_gpu.py
"""Coarsening test for Figure 4.3 (Cahn-Hilliard-Darcy) GPU-first (CuPy), CPU fallback."""

import os
import numpy as np
import matplotlib.pyplot as plt

# Backend selection
cp = None
USE_GPU = False
try:
    import cupy as cp
    from cupyx.scipy.fft import fft2 as fft2_wrap, ifft2 as ifft2_wrap, fftfreq as fftfreq_wrap
    xp = cp
    USE_GPU = True
    print("Using CuPy GPU backend")
except ImportError:
    from scipy.fft import fft2 as fft2_wrap, ifft2 as ifft2_wrap, fftfreq as fftfreq_wrap
    xp = np
    print("Using CPU backend")

def to_cpu(arr):
    if USE_GPU and cp is not None:
        return cp.asnumpy(arr)
    return arr

# Ensure output directory exists
os.makedirs("output", exist_ok=True)

class CahnHilliardDarcySolver:
    """
    DSAV solver for the Cahn-Hilliard-Darcy system (Yang 2021),
    using BDF2 time stepping with auxiliary variables U and Q.
    """

    def __init__(self, Lx=2*np.pi, Ly=2*np.pi, Nx=128, Ny=128, dt=0.001,

```

```

        alpha=100.0, M=1.0, lambda_param=0.01,
        epsilon=0.05, S=2.0, tau=1.0, B=10.0,
        use_sav=True, use_q=True):

    # Parameters
    self.Lx, self.Ly = Lx, Ly
    self.Nx, self.Ny = Nx, Ny
    self.dt = dt
    self.alpha = alpha
    self.M = M
    self.lam = lambda_param
    self.eps = epsilon
    self.S = S
    self.tau = tau
    self.B = B
    self.use_sav = use_sav
    self.use_q = use_q

    # Spatial grid and Fourier wavenumbers
    self.dx = Lx / Nx
    self.dy = Ly / Ny
    self.x = xp.linspace(0, Lx, Nx, endpoint=False)
    self.y = xp.linspace(0, Ly, Ny, endpoint=False)
    self.X, self.Y = xp.meshgrid(self.x, self.y, indexing='ij')

    # Spectral Grid
    self.kx = 2 * xp.pi * fftfreq_wrap(Nx, d=self.dx)
    self.ky = 2 * xp.pi * fftfreq_wrap(Ny, d=self.dy)
    self.kx2d, self.ky2d = xp.meshgrid(self.kx, self.ky, indexing='ij')
    self.k2 = self.kx2d**2 + self.ky2d**2
    self.k2[0, 0] = 1e-10 # Avoid division by zero

    # Fields
    self.phi = xp.zeros((Nx, Ny))
    self.phi_old = xp.zeros((Nx, Ny))
    self.u = xp.zeros((Nx, Ny))
    self.v = xp.zeros((Nx, Ny))
    self.p = xp.zeros((Nx, Ny))
    self.mu = xp.zeros((Nx, Ny))
    self.u_old = xp.zeros((Nx, Ny))
    self.v_old = xp.zeros((Nx, Ny))
    self.mu_old = xp.zeros((Nx, Ny))

    # SAV variable U
    self.U = xp.array(0.0)
    self.U_old = xp.array(0.0)
    self.Q = xp.array(1.0)
    self.Q_old = xp.array(1.0)
    self.t = 0.0
    self.step_count = 0

def init_two_circles(self):
    """Initial condition: two tanh-profile circles (Fig. 4.3)"""
    x1, y1 = np.pi - 0.8, np.pi
    x2, y2 = np.pi + 1.7, np.pi
    r1, r2 = 1.4, 0.5

    dist1 = xp.sqrt((self.X - x1)**2 + (self.Y - y1)**2)
    dist2 = xp.sqrt((self.X - x2)**2 + (self.Y - y2)**2)

    # Tanh profile setup
    self.phi = 1.0 + xp.tanh((r1 - dist1)/(1.5*self.eps)) + \

```



```

        xp.tanh((r2 - dist2)/(1.5*self.eps))
self.phi_old = self.phi.copy()
self._init_sav()
self.u_old = self.u.copy()
self.v_old = self.v.copy()

def _init_sav(self):
    if not self.use_sav:
        self.U = xp.array(1.0)
        self.U_old = self.U
        self.Q = xp.array(1.0)
        self.Q_old = self.Q
        return
    F = (0.25 / self.eps**2) * (self.phi**2 - 1)**2
    E_bulk = xp.sum(F) * self.dx * self.dy
    self.U = xp.sqrt(E_bulk + self.B)
    self.U_old = self.U
    self.Q = xp.array(1.0)
    self.Q_old = self.Q

    f_phi = (1.0 / self.eps**2) * (self.phi**3 - self.phi)
    H = f_phi / xp.sqrt(E_bulk + self.B)
    lap_phi = xp.real(iff2_wrap(-self.k2 * fft2_wrap(self.phi)))
    self.mu = self.lam * (-lap_phi + H * self.U)
    self.mu_old = self.mu.copy()

def step(self):
    """Perform one SAV time step (Cahn-Hilliard + Darcy)."""
    # --- BDF2 coefficients and extrapolated quantities ---
    u_prev = self.u
    v_prev = self.v
    mu_prev = self.mu
    mu_prev_old = self.mu_old
    if self.step_count == 0:
        a, b, c = 2.0, 2.0, 0.0
        u_star = self.u
        v_star = self.v
        mu_star = mu_prev
        phi_star = self.phi
    else:
        a, b, c = 3.0, 4.0, 1.0
        u_star = 2.0 * self.u - self.u_old
        v_star = 2.0 * self.v - self.v_old
        mu_star = 2.0 * mu_prev - mu_prev_old
        phi_star = 2.0 * self.phi - self.phi_old

    dxdy = self.dx * self.dy
    def integral(arr):
        return xp.sum(arr) * dxdy

    # --- Nonlinear term and SAV kernel H(phi*) ---
    f_phi = (1.0 / self.eps**2) * (phi_star**3 - phi_star)
    if self.use_sav:
        F_term = (0.25 / self.eps**2) * (phi_star**2 - 1) ** 2
        E_integral = integral(F_term)
        H = f_phi / xp.sqrt(E_integral + self.B)
    else:
        H = f_phi

    # --- Phase-field solve: linear split in U and Q ---
    phi_hat = fft2_wrap(self.phi)
    phi_old_hat = fft2_wrap(self.phi_old)

```

```

uxphi_hat = fft2_wrap(u_star * phi_star)
vyphi_hat = fft2_wrap(v_star * phi_star)
div_u_phi = xp.real(iff2_wrap(1j * self.kx2d * uxphi_hat + 1j * self.ky2d * vyphi_hat))
adv_hat = fft2_wrap(div_u_phi)

stab_coeff = self.S / self.eps**2
lhs_op = (a / (2.0 * self.dt)) + self.M * self.lam * self.k2**2 + self.M * self.lam *
    stab_coeff * self.k2
rhs_time = (b * phi_hat - c * phi_old_hat) / (2.0 * self.dt)
forcing0 = -self.lam * stab_coeff * phi_star
rhs_spatial0 = -self.M * self.k2 * fft2_wrap(forcing0)

phi0_hat = (rhs_time + rhs_spatial0) / lhs_op
phiU_hat = (-self.M * self.k2 * fft2_wrap(self.lam * H)) / lhs_op
phiQ_hat = (-adv_hat) / lhs_op

phi_lin = xp.real(iff2_wrap(phi0_hat))
phiU = xp.real(iff2_wrap(phiU_hat))
phiQ = xp.real(iff2_wrap(phiQ_hat))

# --- Solve U as a linear function of Q ---
combo0 = a * phi_lin - b * self.phi + c * self.phi_old
I0 = integral(H * combo0)
IU = integral(H * (a * phiU))
IQ = integral(H * (a * phiQ))

denom = a - 0.5 * IU
U0 = (b * self.U - c * self.U_old + 0.5 * I0) / denom
U1 = (0.5 * IQ) / denom

phi0 = phi_lin + U0 * phiU
phi1 = phiQ + U1 * phiU

# --- Build mu components (0 + Q*1) ---
lap_phi0 = xp.real(iff2_wrap(-self.k2 * fft2_wrap(phi0)))
lap_phi1 = xp.real(iff2_wrap(-self.k2 * fft2_wrap(phi1)))
mu0 = self.lam * (-lap_phi0 + H * U0 + stab_coeff * (phi0 - phi_star))
mu1 = self.lam * (-lap_phi1 + H * U1 + stab_coeff * phi1)

# --- Darcy step: velocity split and projection ---
coeff_u = (self.tau * a / (2.0 * self.dt)) + self.alpha
p_hat = fft2_wrap(self.p)
grad_p_x = xp.real(iff2_wrap(1j * self.kx2d * p_hat))
grad_p_y = xp.real(iff2_wrap(1j * self.ky2d * p_hat))
u_rhs0 = (self.tau / (2.0 * self.dt)) * (b * self.u - c * self.u_old) - grad_p_x
v_rhs0 = (self.tau / (2.0 * self.dt)) * (b * self.v - c * self.v_old) - grad_p_y

mu_star_hat = fft2_wrap(mu_star)
grad_mu_star_x = xp.real(iff2_wrap(1j * self.kx2d * mu_star_hat))
grad_mu_star_y = xp.real(iff2_wrap(1j * self.ky2d * mu_star_hat))

u0 = u_rhs0 / coeff_u
v0 = v_rhs0 / coeff_u
u1 = -(phi_star * grad_mu_star_x) / coeff_u
v1 = -(phi_star * grad_mu_star_y) / coeff_u

div_u0_hat = 1j * self.kx2d * fft2_wrap(u0) + 1j * self.ky2d * fft2_wrap(v0)
div_u1_hat = 1j * self.kx2d * fft2_wrap(u1) + 1j * self.ky2d * fft2_wrap(v1)
p_inc0_hat = (self.tau * a / (2.0 * self.dt)) * div_u0_hat / (-self.k2)
p_inc1_hat = (self.tau * a / (2.0 * self.dt)) * div_u1_hat / (-self.k2)
p_inc0_hat[0, 0] = 0.0
p_inc1_hat[0, 0] = 0.0

```

```

p_inc0 = xp.real(iff2_wrap(p_inc0_hat))
p_inc1 = xp.real(iff2_wrap(p_inc1_hat))
grad_p_inc0_x = xp.real(iff2_wrap(1j * self.kx2d * p_inc0_hat))
grad_p_inc0_y = xp.real(iff2_wrap(1j * self.ky2d * p_inc0_hat))
grad_p_inc1_x = xp.real(iff2_wrap(1j * self.kx2d * p_inc1_hat))
grad_p_inc1_y = xp.real(iff2_wrap(1j * self.ky2d * p_inc1_hat))

u_corr0 = u0 - (2.0 * self.dt / (self.tau * a)) * grad_p_inc0_x
v_corr0 = v0 - (2.0 * self.dt / (self.tau * a)) * grad_p_inc0_y
u_corr1 = u1 - (2.0 * self.dt / (self.tau * a)) * grad_p_inc1_x
v_corr1 = v1 - (2.0 * self.dt / (self.tau * a)) * grad_p_inc1_y

# --- Q update and final assembly ---
if self.use_q:
    term1_0 = integral(div_u_phi * mu0)
    term1_1 = integral(div_u_phi * mu1)
    term2_0 = integral(phi_star * (grad_mu_star_x * u0 + grad_mu_star_y * v0))
    term2_1 = integral(phi_star * (grad_mu_star_x * u1 + grad_mu_star_y * v1))
    R0 = term1_0 + term2_0
    R1 = term1_1 + term2_1
    lhs_q = (a / (2.0 * self.dt)) - R1
    rhs_q = (b * self.Q - c * self.Q_old) / (2.0 * self.dt) + R0
    Q_new = rhs_q / lhs_q
else:
    Q_new = xp.array(1.0)

phi_new = phi0 + Q_new * phi1
U_new = U0 + Q_new * U1
mu_new = mu0 + Q_new * mu1

self.u = u_corr0 + Q_new * u_corr1
self.v = v_corr0 + Q_new * v_corr1
self.p = self.p + p_inc0 + Q_new * p_inc1

# --- Persist state ---
self.u_old = u_prev
self.v_old = v_prev
self.mu_old = mu_prev
self.phi_old = self.phi.copy()
self.phi = phi_new
self.mu = mu_new
self.U_old = self.U
self.U = U_new
self.Q_old = self.Q
self.Q = Q_new
self.t += self.dt
self.step_count += 1

def run_case(label, solver, target_times, save_name):
    print(f"---Running_{label}---")
    snapshots = []

    # Capture t=0
    if 0.0 in target_times:
        snapshots.append((0.0, to_cpu(solver.phi.copy())))

    max_time = max(target_times)
    steps = int(max_time / solver.dt) + 20

    for i in range(steps):

```

```

        solver.step()

        # Check times
        for target in target_times:
            if abs(solver.t - target) < solver.dt * 0.6:
                print(f"Saving t={target:.2f}")
                snapshots.append((target, to_cpu(solver.phi.copy())))

        if solver.t > max_time + solver.dt:
            break

    plot_snapshots(label, solver, snapshots, save_name)

def plot_snapshots(label, solver, snapshots, save_name):
    # Visualization
    if not snapshots:
        return

    cols = min(len(snapshots), 6)
    rows = (len(snapshots) - 1) // 6 + 1
    fig, axes = plt.subplots(rows, cols, figsize=(3*cols, 3*rows + 0.5))

    if rows == 1 and cols == 1:
        axes = [axes]
    elif rows > 1 or cols > 1:
        axes = axes.flatten()

    Xp = to_cpu(solver.X)
    Yp = to_cpu(solver.Y)
    for i, (t_val, phi_val) in enumerate(snapshots):
        ax = axes[i]
        # 'jet' colormap to reproduce "ochra" (red/yellow) bubbles and blue background
        ax.contourf(Xp, Yp, phi_val, levels=np.linspace(-1.1, 1.1, 50), cmap='jet')
        # Interface line
        ax.contour(Xp, Yp, phi_val, levels=[0], colors='white', linewidths=1)
        ax.set_title(f"t={t_val:.2f}")
        ax.axis('off')
        ax.set_aspect('equal')

    for j in range(i + 1, len(axes)):
        axes[j].axis('off')

    plt.suptitle(label)
    plt.tight_layout()
    plt.savefig(f"output/{save_name}.png", dpi=150)
    plt.close()
    print(f"Saved_{save_name}.png")

if __name__ == "__main__":
    # --- Figure 4.3 ---
    # Time step per article: dt = 0.01 / 2^4.
    solver43 = CahnHilliardDarcySolver(
        Lx=2*np.pi, Ly=2*np.pi, Nx=128, Ny=128, dt=0.01 / (2**4),
        alpha=100.0, M=1.0, lambda_param=0.01,
        epsilon=0.05, S=2.0, tau=1.0
    )
    solver43.init_two_circles()
    times43 = [0.0, 1.4, 1.9, 1.95, 2.45, 5.0]
    run_case("Figure_4.3:_Coarsening", solver43, times43, "Figure_4_3_Fixed_Color")

```

```

# 2D spinodal decomposition_figure4.8_gpu.py
"""
GPU-first(CuPy) version of the spinodal decomposition driver for Fig. 4.8.
Falls back to CPU if CuPy/CUDA is unavailable.
"""

import os
import numpy as np
import matplotlib.pyplot as plt

# Backend selection
cp = None
USE_GPU = False
try:
    import cupy as cp
    from cupyx.scipy.fft import fft2 as fft2_gpu, ifft2 as ifft2_gpu, fftfreq as fftfreq_gpu
    USE_GPU = True
    xp_backend = cp
    fft2_wrap = fft2_gpu
    ifft2_wrap = ifft2_gpu
    fftfreq_wrap = fftfreq_gpu
    print("Using CuPy GPU backend")
except ImportError:
    from scipy.fft import fft2 as fft2_cpu, ifft2 as ifft2_cpu, fftfreq as fftfreq_cpu
    xp_backend = np
    fft2_wrap = fft2_cpu
    ifft2_wrap = ifft2_cpu
    fftfreq_wrap = fftfreq_cpu
    print("Using CPU backend")

def to_cpu(arr):
    if USE_GPU and cp is not None:
        return cp.asnumpy(arr)
    return arr

# Ensure output directory exists
os.makedirs("output", exist_ok=True)

class CahnHilliardDarcySolver:
    """
    DSAV solver for the Cahn-Hilliard-Darcy system (Yang 2021),
    using BDF2 time stepping with auxiliary variables U and Q.
    """

    def __init__(self, Lx=2*np.pi, Ly=2*np.pi, Nx=512, Ny=512, dt=0.001,
                 alpha=100.0, M=1.0, lambda_param=0.01,
                 epsilon=0.025, S=10.0, tau=1.0, B=10.0):

        # Parameters
        self.Lx, self.Ly = Lx, Ly
        self.Nx, self.Ny = Nx, Ny
        self.dt = dt
        self.alpha = alpha
        self.M = M
        self.lam = lambda_param
        self.eps = epsilon
        self.S = S
        self.tau = tau
        self.B = B

```

```

# Spatial Grid
self.dx = Lx / Nx
self.dy = Ly / Ny
self.x = xp_backend.linspace(0, Lx, Nx, endpoint=False)
self.y = xp_backend.linspace(0, Ly, Ny, endpoint=False)
self.X, self.Y = xp_backend.meshgrid(self.x, self.y, indexing='ij')

# Spectral Grid
self.kx = 2 * xp_backend.pi * fftfreq_wrap(Nx, d=self.dx)
self.ky = 2 * xp_backend.pi * fftfreq_wrap(Ny, d=self.dy)
self.kx2d, self.ky2d = xp_backend.meshgrid(self.kx, self.ky, indexing='ij')
self.k2 = self.kx2d**2 + self.ky2d**2
self.k2[0, 0] = 1e-10 # Avoid division by zero

# Fields
self.phi = xp_backend.zeros((Nx, Ny))
self.phi_old = xp_backend.zeros((Nx, Ny))
self.u = xp_backend.zeros((Nx, Ny))
self.v = xp_backend.zeros((Nx, Ny))
self.p = xp_backend.zeros((Nx, Ny))
self.mu = xp_backend.zeros((Nx, Ny))
self.u_old = xp_backend.zeros((Nx, Ny))
self.v_old = xp_backend.zeros((Nx, Ny))
self.mu_old = xp_backend.zeros((Nx, Ny))

# DSAV variables
self.U = xp_backend.array(0.0)
self.U_old = xp_backend.array(0.0)
self.Q = xp_backend.array(1.0)
self.Q_old = xp_backend.array(1.0)
self.t = 0.0
self.step_count = 0

def init_two_circles(self):
    """Initialize for Figure 4.3"""
    x1, y1 = np.pi - 0.8, np.pi
    x2, y2 = np.pi + 1.7, np.pi
    r1, r2 = 1.4, 0.5

    dist1 = xp_backend.sqrt((self.X - x1)**2 + (self.Y - y1)**2)
    dist2 = xp_backend.sqrt((self.X - x2)**2 + (self.Y - y2)**2)

    # Tanh profile setup
    self.phi = 1.0 + xp_backend.tanh((r1 - dist1)/(1.5*self.eps)) + \
        xp_backend.tanh((r2 - dist2)/(1.5*self.eps))
    self.phi_old = self.phi.copy()
    self._init_sav()
    self.u_old = self.u.copy()
    self.v_old = self.v.copy()

def init_spinodal(self, phi_avg, noise_amp=0.001):
    """Initialize for Figure 4.8 with small random noise (paper: 0.001*rand(x))."""
    noise = noise_amp * (2 * xp_backend.random.rand(self.Nx, self.Ny) - 1)
    self.phi = phi_avg + noise
    self.phi_old = self.phi.copy()
    self._init_sav()
    self.u_old = self.u.copy()
    self.v_old = self.v.copy()

def _init_sav(self):
    # Energy functional F(phi)

```

```

F = (0.25 / self.eps**2) * (self.phi**2 - 1)**2
E_bulk = xp_backend.sum(F) * self.dx * self.dy
self.U = xp_backend.sqrt(E_bulk + self.B)
self.U_old = self.U
self.Q = xp_backend.array(1.0)
self.Q_old = self.Q

f_phi = (1.0 / self.eps**2) * (self.phi**3 - self.phi)
H = f_phi / xp_backend.sqrt(E_bulk + self.B)
lap_phi = xp_backend.real(iff2_wrap(-self.k2 * fft2_wrap(self.phi)))
self.mu = self.lam * (-lap_phi + H * self.U)
self.mu_old = self.mu.copy()

def step(self):
    """Perform one DSAV time step (Cahn-Hilliard+Darcy)."""
    # --- BDF2 coefficients and extrapolated quantities ---
    u_prev = self.u
    v_prev = self.v
    mu_prev = self.mu
    mu_prev_old = self.mu_old
    if self.step_count == 0:
        a, b, c = 2.0, 2.0, 0.0
        u_star = self.u
        v_star = self.v
        mu_star = mu_prev
        phi_star = self.phi
    else:
        a, b, c = 3.0, 4.0, 1.0
        u_star = 2.0 * self.u - self.u_old
        v_star = 2.0 * self.v - self.v_old
        mu_star = 2.0 * mu_prev - mu_prev_old
        phi_star = 2.0 * self.phi - self.phi_old

    dxdy = self.dx * self.dy
    def integral(arr):
        return xp_backend.sum(arr) * dxdy

    # --- Nonlinear term and SAV kernel H(phi*) ---
    f_phi = (1.0 / self.eps**2) * (phi_star**3 - phi_star)
    F_term = (0.25 / self.eps**2) * (phi_star**2 - 1) ** 2
    E_integral = integral(F_term)
    H = f_phi / xp_backend.sqrt(E_integral + self.B)

    # --- Phase-field solve: linear split in U and Q ---
    phi_hat = fft2_wrap(self.phi)
    phi_old_hat = fft2_wrap(self.phi_old)
    uxphi_hat = fft2_wrap(u_star * phi_star)
    vyphi_hat = fft2_wrap(v_star * phi_star)
    div_u_phi = xp_backend.real(iff2_wrap(1j * self.kx2d * uxphi_hat + 1j * self.ky2d *
        vyphi_hat))
    adv_hat = fft2_wrap(div_u_phi)

    stab_coeff = self.S / self.eps**2
    lhs_op = (a / (2.0 * self.dt)) + self.M * self.lam * self.k2**2 + self.M * self.lam *
        stab_coeff * self.k2
    rhs_time = (b * phi_hat - c * phi_old_hat) / (2.0 * self.dt)
    forcing0 = -self.lam * stab_coeff * phi_star
    rhs_spatial0 = -self.M * self.k2 * fft2_wrap(forcing0)

    phi0_hat = (rhs_time + rhs_spatial0) / lhs_op
    phiU_hat = (-self.M * self.k2 * fft2_wrap(self.lam * H)) / lhs_op
    phiQ_hat = (-adv_hat) / lhs_op

```

```

phi_lin = xp_backend.real(iff2_wrap(phi0_hat))
phiU = xp_backend.real(iff2_wrap(phiU_hat))
phiQ = xp_backend.real(iff2_wrap(phiQ_hat))

# --- Solve U as a linear function of Q ---
combo0 = a * phi_lin - b * self.phi + c * self.phi_old
I0 = integral(H * combo0)
IU = integral(H * (a * phiU))
IQ = integral(H * (a * phiQ))

denom = a - 0.5 * IU
U0 = (b * self.U - c * self.U_old + 0.5 * I0) / denom
U1 = (0.5 * IQ) / denom

phi0 = phi_lin + U0 * phiU
phi1 = phiQ + U1 * phiU

# --- Build mu components (0 + Q*1) ---
lap_phi0 = xp_backend.real(iff2_wrap(-self.k2 * fft2_wrap(phi0)))
lap_phi1 = xp_backend.real(iff2_wrap(-self.k2 * fft2_wrap(phi1)))
mu0 = self.lam * (-lap_phi0 + H * U0 + stab_coeff * (phi0 - phi_star))
mu1 = self.lam * (-lap_phi1 + H * U1 + stab_coeff * phi1)

# --- Darcy step: velocity split and projection ---
coeff_u = (self.tau * a / (2.0 * self.dt)) + self.alpha
p_hat = fft2_wrap(self.p)
grad_p_x = xp_backend.real(iff2_wrap(1j * self.kx2d * p_hat))
grad_p_y = xp_backend.real(iff2_wrap(1j * self.ky2d * p_hat))
u_rhs0 = (self.tau / (2.0 * self.dt)) * (b * self.u - c * self.u_old) - grad_p_x
v_rhs0 = (self.tau / (2.0 * self.dt)) * (b * self.v - c * self.v_old) - grad_p_y

mu_star_hat = fft2_wrap(mu_star)
grad_mu_star_x = xp_backend.real(iff2_wrap(1j * self.kx2d * mu_star_hat))
grad_mu_star_y = xp_backend.real(iff2_wrap(1j * self.ky2d * mu_star_hat))

u0 = u_rhs0 / coeff_u
v0 = v_rhs0 / coeff_u
u1 = -(phi_star * grad_mu_star_x) / coeff_u
v1 = -(phi_star * grad_mu_star_y) / coeff_u

div_u0_hat = 1j * self.kx2d * fft2_wrap(u0) + 1j * self.ky2d * fft2_wrap(v0)
div_u1_hat = 1j * self.kx2d * fft2_wrap(u1) + 1j * self.ky2d * fft2_wrap(v1)
p_inc0_hat = (self.tau * a / (2.0 * self.dt)) * div_u0_hat / (-self.k2)
p_inc1_hat = (self.tau * a / (2.0 * self.dt)) * div_u1_hat / (-self.k2)
p_inc0_hat[0, 0] = 0.0
p_inc1_hat[0, 0] = 0.0

p_inc0 = xp_backend.real(iff2_wrap(p_inc0_hat))
p_inc1 = xp_backend.real(iff2_wrap(p_inc1_hat))
grad_p_inc0_x = xp_backend.real(iff2_wrap(1j * self.kx2d * p_inc0_hat))
grad_p_inc0_y = xp_backend.real(iff2_wrap(1j * self.ky2d * p_inc0_hat))
grad_p_inc1_x = xp_backend.real(iff2_wrap(1j * self.kx2d * p_inc1_hat))
grad_p_inc1_y = xp_backend.real(iff2_wrap(1j * self.ky2d * p_inc1_hat))

u_corr0 = u0 - (2.0 * self.dt / (self.tau * a)) * grad_p_inc0_x
v_corr0 = v0 - (2.0 * self.dt / (self.tau * a)) * grad_p_inc0_y
u_corr1 = u1 - (2.0 * self.dt / (self.tau * a)) * grad_p_inc1_x
v_corr1 = v1 - (2.0 * self.dt / (self.tau * a)) * grad_p_inc1_y

# --- Q update and final assembly ---
term1_0 = integral(div_u_phi * mu0)

```



```

term1_1 = integral(div_u_phi * mu1)
term2_0 = integral(phi_star * (grad_mu_star_x * u0 + grad_mu_star_y * v0))
term2_1 = integral(phi_star * (grad_mu_star_x * u1 + grad_mu_star_y * v1))
R0 = term1_0 + term2_0
R1 = term1_1 + term2_1
lhs_q = (a / (2.0 * self.dt)) - R1
rhs_q = (b * self.Q - c * self.Q_old) / (2.0 * self.dt) + R0
Q_new = rhs_q / lhs_q

phi_new = phi0 + Q_new * phi1
U_new = U0 + Q_new * U1
mu_new = mu0 + Q_new * mu1

self.u = u_corr0 + Q_new * u_corr1
self.v = v_corr0 + Q_new * v_corr1
self.p = self.p + p_inc0 + Q_new * p_inc1

# --- Persist state ---
self.u_old = u_prev
self.v_old = v_prev
self.mu_old = mu_prev
self.phi_old = self.phi.copy()
self.phi = phi_new
self.mu = mu_new
self.U_old = self.U
self.U = U_new
self.Q_old = self.Q
self.Q = Q_new
self.t += self.dt
self.step_count += 1

def run_case(label, solver, target_times, save_name, invert_colors=False):
    print(f"--- Running {label} ---")
    snapshots = []

    # Capture t=0
    if 0.0 in target_times:
        snapshots.append((0.0, to_cpu(solver.phi.copy())))

    max_time = max(target_times)
    steps = int(max_time / solver.dt) + 50

    for i in range(steps):
        solver.step()

        # Check times
        for target in target_times:
            if abs(solver.t - target) < solver.dt * 0.6:
                print(f"Saving t={target:.2f}")
                snapshots.append((target, to_cpu(solver.phi.copy())))

        if solver.t > max_time + solver.dt:
            break

    # Visualization
    if not snapshots:
        return

    cols = min(len(snapshots), 5) # Figure 4.8 has 5 cols
    rows = (len(snapshots) - 1) // 5 + 1
    fig, axes = plt.subplots(rows, cols, figsize=(3*cols, 3*rows + 0.5))

```

```

if rows == 1 and cols == 1:
    axes = [axes]
elif rows > 1 or cols > 1:
    axes = axes.flatten()

# If fewer snapshots than axes, hide extras
if len(axes) > len(snapshots):
    for j in range(len(snapshots), len(axes)):
        axes[j].axis('off')

for i, (t_val, phi_val) in enumerate(snapshots):
    field = phi_val
    ax = axes[i]
    # Use 'jet' which gives blue (-1) to red (1) with yellow/green interface
    cmap_use = 'jet_r' if invert_colors else 'jet'
    cf = ax.contourf(solver.X.get() if USE_GPU else solver.X,
                    solver.Y.get() if USE_GPU else solver.Y,
                    field,
                    levels=np.linspace(-1.1, 1.1, 50), cmap=cmap_use)

    # Interface line (white)
    ax.contour(solver.X.get() if USE_GPU else solver.X,
               solver.Y.get() if USE_GPU else solver.Y,
               field, levels=[0], colors='white', linewidths=1)
    ax.set_title(f"t={t_val:.1f}")
    ax.axis('off')
    ax.set_aspect('equal')

plt.suptitle(label)
plt.tight_layout()
plt.savefig(f"output/{save_name}.png", dpi=150)
plt.close()
print(f"Saved_{save_name}.png")

if __name__ == "__main__":
    # --- Figure 4.8a ---
    # Parameters from Section 4.3
    # Nx=512, dt=0.001
    print("Initializing_Figure_4.8a...")
    solver48a = CahnHilliardDarcySolver(
        Lx=2*np.pi, Ly=2*np.pi, Nx=512, Ny=512, dt=0.001,
        alpha=100.0, M=1.0, lambda_param=0.01,
        epsilon=0.025, S=10.0, tau=1.0
    )
    solver48a.init_spinodal(phi_avg=0.0, noise_amp=0.001)
    times48a = [0.5, 5.0, 10.0, 15.0, 40.0]
    # Invert colors to match article (swap red/blue mapping)
    run_case("Figure_4.8a: Spinodal_(phi=0)", solver48a, times48a, "Figure_4_8a_Fixed",
            invert_colors=True)

    # --- Figure 4.8b ---
    print("Initializing_Figure_4.8b...")
    solver48b = CahnHilliardDarcySolver(
        Lx=2*np.pi, Ly=2*np.pi, Nx=512, Ny=512, dt=0.001,
        alpha=100.0, M=1.0, lambda_param=0.01,
        epsilon=0.025, S=10.0, tau=1.0
    )
    solver48b.init_spinodal(phi_avg=0.3, noise_amp=0.001)
    times48b = [5.0, 10.0, 15.0, 40.0, 50.0]
    run_case("Figure_4.8b: Spinodal_(phi=0.3)", solver48b, times48b, "Figure_4_8b_Fixed")

```

```

# Convergence tests_figure4.6_gpu.py
"""
Driver to reproduce Fig. 4.6 (DSAV vs EX-SAV vs AV).
GPU-first (CuPy) with CPU fallback; initial condition (4.1),
parameters (4.2), L2 errors vs small-step DSAV reference.
EX-SAV removes Q (Q~n=1); AV removes Q and sets S=0.
"""

import os
import numpy as np
import matplotlib.pyplot as plt

cp = None

# Backend: prefer CuPy if available; else NumPy/scipy.fft
USE_GPU = False
try:
    import cupy as cp
    from cupyx.scipy.fft import fft2 as fft2_gpu, ifft2 as ifft2_gpu, fftfreq as fftfreq_gpu

    xp_backend = cp
    fft2_wrap = fft2_gpu
    ifft2_wrap = ifft2_gpu
    fftfreq_wrap = fftfreq_gpu
    USE_GPU = True
    print("Using CuPy GPU backend")
except ImportError:
    from scipy.fft import fft2 as fft2_cpu, ifft2 as ifft2_cpu, fftfreq as fftfreq_cpu

    xp_backend = np
    fft2_wrap = fft2_cpu
    ifft2_wrap = ifft2_cpu
    fftfreq_wrap = fftfreq_cpu
    print("Using CPU backend")

def to_cpu(arr):
    if USE_GPU and cp is not None:
        return cp.asnumpy(arr)
    return arr

# Directories
SCRIPT_DIR = os.path.dirname(os.path.abspath(__file__))
OUTPUT_DIR = os.path.join(SCRIPT_DIR, "output")
os.makedirs(OUTPUT_DIR, exist_ok=True)

class CahnHilliardDarcySolver:
    """
    Solver with SAV normalization toggle (use_sav). For Figure 4.6 we keep
    use_sav=True and vary use_q (EX-SAV/AV set Q~n=1) and S (AV sets S=0).
    Only the pieces needed for Figure 4.6 are kept.
    """

    def __init__(
        self,
        Lx=2 * np.pi,
        Ly=2 * np.pi,
        Nx=128,
        Ny=128,
        dt=0.001,
    ):

```

```

alpha=100.0,
M=1.0,
lambda_param=0.01,
epsilon=0.05,
S=2.0,
tau=1.0,
B=10.0,
use_sav=True,
use_q=True,
):
    self.Lx, self.Ly = Lx, Ly
    self.Nx, self.Ny = Nx, Ny
    self.dt = dt
    self.alpha = alpha
    self.M = M
    self.lam = lambda_param
    self.eps = epsilon
    self.S = S
    self.tau = tau
    self.B = B
    self.use_sav = use_sav
    self.use_q = use_q

    # Spatial grid
    self.dx = Lx / Nx
    self.dy = Ly / Ny
    self.x = xp_backend.linspace(0, Lx, Nx, endpoint=False)
    self.y = xp_backend.linspace(0, Ly, Ny, endpoint=False)
    self.X, self.Y = xp_backend.meshgrid(self.x, self.y, indexing="ij")

    # Spectral grid (Fourier wavenumbers)
    self.kx = 2 * xp_backend.pi * fftfreq_wrap(Nx, d=self.dx)
    self.ky = 2 * xp_backend.pi * fftfreq_wrap(Ny, d=self.dy)
    self.kx2d, self.ky2d = xp_backend.meshgrid(self.kx, self.ky, indexing="ij")
    self.k2 = self.kx2d**2 + self.ky2d**2
    self.k2[0, 0] = 1e-10 # avoid divide by zero

    # Fields
    self.phi = xp_backend.zeros((Nx, Ny))
    self.phi_old = xp_backend.zeros((Nx, Ny))
    self.u = xp_backend.zeros((Nx, Ny))
    self.v = xp_backend.zeros((Nx, Ny))
    self.mu = xp_backend.zeros((Nx, Ny))
    self.p = xp_backend.zeros((Nx, Ny))
    self.u_old = xp_backend.zeros((Nx, Ny))
    self.v_old = xp_backend.zeros((Nx, Ny))
    self.mu_old = xp_backend.zeros((Nx, Ny))
    self.U = xp_backend.array(0.0) # SAV variable
    self.U_old = xp_backend.array(0.0)
    self.Q = xp_backend.array(1.0)
    self.Q_old = xp_backend.array(1.0)
    self.step_count = 0
    self.t = 0.0

def init_two_circles(self):
    """Initial condition (4.1): two tanh-profile disks."""
    x1, y1 = np.pi - 0.8, np.pi
    x2, y2 = np.pi + 1.7, np.pi
    r1, r2 = 1.4, 0.5
    dist1 = xp_backend.sqrt((self.X - x1) ** 2 + (self.Y - y1) ** 2)
    dist2 = xp_backend.sqrt((self.X - x2) ** 2 + (self.Y - y2) ** 2)
    self.phi = (

```

```

        1.0
        + xp_backend.tanh((r1 - dist1) / (1.5 * self.eps))
        + xp_backend.tanh((r2 - dist2) / (1.5 * self.eps))
    )
    self.phi_old = self.phi.copy()
    self._init_sav()

def _init_sav(self):
    if not self.use_sav:
        # EX-SAV / AV: fix  $Q^n = 1$  (no SAV normalization)
        self.U = 1.0
        self.U_old = self.U
        self.Q = 1.0
        self.Q_old = self.Q
        return
    F = (0.25 / self.eps**2) * (self.phi**2 - 1) ** 2
    E_bulk = xp_backend.sum(F) * self.dx * self.dy
    self.U = xp_backend.sqrt(E_bulk + self.B)
    self.U_old = self.U
    self.Q = 1.0
    self.Q_old = self.Q

def step(self):
    """Perform one SAV time step (Cahn-Hilliard + Darcy)."""
    u_prev = self.u
    v_prev = self.v
    mu_prev = self.mu
    mu_prev_old = self.mu_old
    if self.step_count == 0:
        a, b, c = 2.0, 2.0, 0.0
        u_star = self.u
        v_star = self.v
        mu_star = mu_prev
        phi_star = self.phi
    else:
        a, b, c = 3.0, 4.0, 1.0
        u_star = 2.0 * self.u - self.u_old
        v_star = 2.0 * self.v - self.v_old
        mu_star = 2.0 * mu_prev - mu_prev_old
        phi_star = 2.0 * self.phi - self.phi_old

    dxdy = self.dx * self.dy
    def integral(arr):
        return xp_backend.sum(arr) * dxdy

    f_phi = (1.0 / self.eps**2) * (phi_star**3 - phi_star)
    if self.use_sav:
        F_term = (0.25 / self.eps**2) * (phi_star**2 - 1) ** 2
        E_integral = integral(F_term)
        H = f_phi / xp_backend.sqrt(E_integral + self.B)
    else:
        H = f_phi

    phi_hat = fft2_wrap(self.phi)
    phi_old_hat = fft2_wrap(self.phi_old)
    uxphi_hat = fft2_wrap(u_star * phi_star)
    vyphi_hat = fft2_wrap(v_star * phi_star)
    div_u_phi = xp_backend.real(iff2_wrap(1j * self.kx2d * uxphi_hat + 1j * self.ky2d *
        vyphi_hat))
    adv_hat = fft2_wrap(div_u_phi)

    stab_coeff = self.S / self.eps**2
    lhs_op = (a / (2.0 * self.dt)) + self.M * self.lam * self.k2**2 + self.M * self.lam *

```

```

        stab_coeff * self.k2
    rhs_time = (b * phi_hat - c * phi_old_hat) / (2.0 * self.dt)
    forcing0 = -self.lam * stab_coeff * phi_star
    rhs_spatial0 = -self.M * self.k2 * fft2_wrap(forcing0)

    phi0_hat = (rhs_time + rhs_spatial0) / lhs_op
    phiU_hat = (-self.M * self.k2 * fft2_wrap(self.lam * H)) / lhs_op
    phiQ_hat = (-adv_hat) / lhs_op

    phi_lin = xp_backend.real(iff2_wrap(phi0_hat))
    phiU = xp_backend.real(iff2_wrap(phiU_hat))
    phiQ = xp_backend.real(iff2_wrap(phiQ_hat))

    combo0 = a * phi_lin - b * self.phi + c * self.phi_old
    I0 = integral(H * combo0)
    IU = integral(H * (a * phiU))
    IQ = integral(H * (a * phiQ))

    denom = a - 0.5 * IU
    U0 = (b * self.U - c * self.U_old + 0.5 * I0) / denom
    U1 = (0.5 * IQ) / denom

    phi0 = phi_lin + U0 * phiU
    phi1 = phiQ + U1 * phiU

    lap_phi0 = xp_backend.real(iff2_wrap(-self.k2 * fft2_wrap(phi0)))
    lap_phi1 = xp_backend.real(iff2_wrap(-self.k2 * fft2_wrap(phi1)))
    mu0 = self.lam * (-lap_phi0 + H * U0 + stab_coeff * (phi0 - phi_star))
    mu1 = self.lam * (-lap_phi1 + H * U1 + stab_coeff * phi1)

    coeff_u = (self.tau * a / (2.0 * self.dt)) + self.alpha
    p_hat = fft2_wrap(self.p)
    grad_p_x = xp_backend.real(iff2_wrap(1j * self.kx2d * p_hat))
    grad_p_y = xp_backend.real(iff2_wrap(1j * self.ky2d * p_hat))
    u_rhs0 = (self.tau / (2.0 * self.dt)) * (b * self.u - c * self.u_old) - grad_p_x
    v_rhs0 = (self.tau / (2.0 * self.dt)) * (b * self.v - c * self.v_old) - grad_p_y

    mu_star_hat = fft2_wrap(mu_star)
    grad_mu_star_x = xp_backend.real(iff2_wrap(1j * self.kx2d * mu_star_hat))
    grad_mu_star_y = xp_backend.real(iff2_wrap(1j * self.ky2d * mu_star_hat))

    u0 = u_rhs0 / coeff_u
    v0 = v_rhs0 / coeff_u
    u1 = -(phi_star * grad_mu_star_x) / coeff_u
    v1 = -(phi_star * grad_mu_star_y) / coeff_u

    div_u0_hat = 1j * self.kx2d * fft2_wrap(u0) + 1j * self.ky2d * fft2_wrap(v0)
    div_u1_hat = 1j * self.kx2d * fft2_wrap(u1) + 1j * self.ky2d * fft2_wrap(v1)
    p_inc0_hat = (self.tau * a / (2.0 * self.dt)) * div_u0_hat / (-self.k2)
    p_inc1_hat = (self.tau * a / (2.0 * self.dt)) * div_u1_hat / (-self.k2)
    p_inc0_hat[0, 0] = 0.0
    p_inc1_hat[0, 0] = 0.0

    p_inc0 = xp_backend.real(iff2_wrap(p_inc0_hat))
    p_inc1 = xp_backend.real(iff2_wrap(p_inc1_hat))
    grad_p_inc0_x = xp_backend.real(iff2_wrap(1j * self.kx2d * p_inc0_hat))
    grad_p_inc0_y = xp_backend.real(iff2_wrap(1j * self.ky2d * p_inc0_hat))
    grad_p_inc1_x = xp_backend.real(iff2_wrap(1j * self.kx2d * p_inc1_hat))
    grad_p_inc1_y = xp_backend.real(iff2_wrap(1j * self.ky2d * p_inc1_hat))

    u_corr0 = u0 - (2.0 * self.dt / (self.tau * a)) * grad_p_inc0_x
    v_corr0 = v0 - (2.0 * self.dt / (self.tau * a)) * grad_p_inc0_y

```

```

u_corr1 = u1 - (2.0 * self.dt / (self.tau * a)) * grad_p_inc1_x
v_corr1 = v1 - (2.0 * self.dt / (self.tau * a)) * grad_p_inc1_y

if self.use_q:
    term1_0 = integral(div_u_phi * mu0)
    term1_1 = integral(div_u_phi * mu1)
    term2_0 = integral(phi_star * (grad_mu_star_x * u0 + grad_mu_star_y * v0))
    term2_1 = integral(phi_star * (grad_mu_star_x * u1 + grad_mu_star_y * v1))
    R0 = term1_0 + term2_0
    R1 = term1_1 + term2_1
    lhs_q = (a / (2.0 * self.dt)) - R1
    rhs_q = (b * self.Q - c * self.Q_old) / (2.0 * self.dt) + R0
    Q_new = rhs_q / lhs_q
else:
    Q_new = xp_backend.array(1.0)

phi_new = phi0 + Q_new * phi1
U_new = U0 + Q_new * U1
mu_new = mu0 + Q_new * mu1

self.u = u_corr0 + Q_new * u_corr1
self.v = v_corr0 + Q_new * v_corr1
self.p = self.p + p_inc0 + Q_new * p_inc1

self.u_old = u_prev
self.v_old = v_prev
self.mu_old = mu_prev
self.phi_old = self.phi.copy()
self.phi = phi_new
self.mu = mu_new
self.U_old = self.U
self.U = U_new
self.Q_old = self.Q
self.Q = Q_new
self.t += self.dt
self.step_count += 1

def l2_error(phi_num, phi_ref, dx, dy):
    diff = phi_num - phi_ref
    return np.sqrt(np.sum(diff**2) * dx * dy)

def compute_solution(dt, final_time, base_params, use_sav=True, use_q=True, S_override=None, label=None):
    params = base_params.copy()
    params.update(dict(dt=dt, use_sav=use_sav, use_q=use_q))
    if S_override is not None:
        params["S"] = S_override

    solver = CahnHilliardDarcySolver(**params)
    solver.init_two_circles()
    steps = int(np.ceil(final_time / dt))
    for _ in range(steps):
        solver.step()
    if label:
        print(f"_{label}_t={solver.t:.5e},_steps={steps}")
    phi_cpu = to_cpu(solver.phi)
    return phi_cpu, solver.dx, solver.dy

def run_time_refinement_tests():

```

```

base_params = dict(
    Lx=2 * np.pi,
    Ly=2 * np.pi,
    Nx=128,
    Ny=128,
    alpha=100.0,
    M=1.0,
    lambda_param=0.01,
    epsilon=0.05,
    S=2.0,
    tau=1.0,
    B=10.0,
)

# Use a longer horizon to separate DSAV vs EX-SAV; reference step remains small.
final_time = 0.01
dt_reference = 1e-9
print(f"Computing DSAV reference (dt={dt_reference})...")
phi_ref, dx_ref, dy_ref = compute_solution(
    dt_reference, final_time, base_params, use_sav=True, S_override=2.0, label="DSAV-ref"
)

dt_values = 0.01 / (2 ** np.arange(0, 8))

errors_dsav = []
errors_exsav = []
errors_av = []

for dt in dt_values:
    print(f"Running DSAV dt={dt}...")
    phi_dsav, dx, dy = compute_solution(dt, final_time, base_params, use_sav=True, use_q=True,
                                         S_override=2.0)
    errors_dsav.append(l2_error(phi_dsav, phi_ref, dx, dy))

    print(f"Running EX-SAV dt={dt}...")
    phi_exsav, dx, dy = compute_solution(dt, final_time, base_params, use_sav=True, use_q=False,
                                         S_override=2.0)
    errors_exsav.append(l2_error(phi_exsav, phi_ref, dx, dy))

    print(f"Running AV dt={dt}...")
    phi_av, dx, dy = compute_solution(dt, final_time, base_params, use_sav=True, use_q=False,
                                       S_override=0.0)
    errors_av.append(l2_error(phi_av, phi_ref, dx, dy))

fig, axes = plt.subplots(1, 2, figsize=(12, 4))

# (a) DSAV vs EX-SAV
axes[0].loglog(dt_values, errors_dsav, "rD-", label=r"DSAV:$\phi$")
axes[0].loglog(dt_values, errors_exsav, "bd--", label=r"EX-SAV:$\phi$")
axes[0].set_xlabel("Time Step")
axes[0].set_ylabel(r"$L^2$ Error")
axes[0].set_xlim(1e-5, 1e-2)
axes[0].set_ylim(1e-7, 1e0)
ref_y0_left = errors_dsav[0]
ref_line_left = ref_y0_left * (dt_values / dt_values[0]) ** 2
axes[0].loglog(dt_values, ref_line_left, "b--", label="Ref:slope_2")
axes[0].legend()
axes[0].grid(True, which="both", ls=":")
axes[0].set_title("(a) DSAV and EX-SAV (no Q)")

# (b) DSAV vs AV
axes[1].loglog(dt_values, errors_dsav, "rD-", label=r"DSAV:$\phi$")

```



```

axes[1].loglog(dt_values, errors_av, "bo-", label=r"AV:\phi$")
ref_y0 = errors_dsav[0]
ref_line = ref_y0 * (dt_values / dt_values[0]) ** 2
axes[1].loglog(dt_values, ref_line, "b--", label="Ref:slope_2")
axes[1].set_xlabel("Time_Step")
axes[1].set_ylabel(r"$L^2$Error")
axes[1].set_xlim(1e-5, 1e-2)
axes[1].set_ylim(1e-7, 1e0)
axes[1].legend()
axes[1].grid(True, which="both", ls=":")
axes[1].set_title("(b) DSAV and AV (no S and Q)")

plt.tight_layout()
outfile = os.path.join(OUTPUT_DIR, "Figure_4_6_reproduction.png")
plt.savefig(outfile, dpi=300)
plt.close()
print(f"Saved {outfile}")

if __name__ == "__main__":
    run_time_refinement_tests()

```

A.4. Key Implementation Features

The implementation provides automatic backend selection, allowing seamless execution on GPU or CPU systems. Both versions use identical algorithmic approaches: FFT-based spectral discretization, element-wise operations in Fourier space for linear solves, and fully decoupled time stepping as per Yang's SAV scheme.

A.5. Dependencies

A.5.1. Required (CPU Only)

```

numpy >= 1.20          # NumPy version 1.20 or newer
scipy >= 1.5
matplotlib >= 3.3

```

A.5.2. Optional (GPU Acceleration)

```

cupy >= 10.0    (requires NVIDIA CUDA toolkit)

```

A.5.3. Optional (Performance Enhancement)

`pyfftw >= 0.13` (faster CPU FFT via multithreading)